คู่มือสถาปัตยกรรมและการเขียนโค้ดเชิงวัตถุ (OOAD & OOP)

# คู่มือเจาะลึกและเรียนรู้โค้ด Java OOP 17 คลาส

ระบบบริหารจัดการร้านอาหาร "ข้าวต้มอารมณ์ดี" (Khao Tom Arom Dee)
เจาะลึก 16 คลาสโมเดล + Main Controller ตามมาตรฐาน OOAD P1–P9

### สาระสำคัญที่ครอบคลุมในเอกสารฉบับนี้:

- **สถาปัตยกรรม OOP & Clean Domain Design:** เข้าใจลึกซึ้งถึง Class, Object, Encapsulation, State Management และ Invariants
- **ความสัมพันธ์เชิงโครงสร้าง (Relationships):** จำแนกชัดเจนระหว่าง Association (has-a) และ Composition (owns-a) พร้อมหลักฐานการทำลายวงจรชีวิต (destroy cascade) ทั้ง 5 คู่
- **ผ่าซอร์สโค้ด 16 คลาสโมเดลแบบบรรทัดต่อบรรทัด:** อธิบายบทบาท ฟิลด์ เมธอด Validation กฎการตัดสต็อก การคิดเงิน และระบบแต้มสะสม
- **เจาะลึก Main.java (Driver & Presentation):** ถอดรหัสการทำงาน 8 Use Cases (UC01–UC08), In-Memory Collections และตัวอย่างข้อมูลจริง
- **Life of an Order & Defense Cheatsheet:** ตามรอยการเดินทางของข้อมูลแบบ End-to-End พร้อมคลัง 15 คำถามปราบเซียนในการสอบและนำเสนอ

---

**จัดทำสำหรับ:** นักศึกษาและนักพัฒนาซอฟต์แวร์สาย Object-Oriented Analysis & Design (OOAD)
**ชุดโครงงาน:** ระบบบริหารร้านข้าวต้มอารมณ์ดี (ฉบับ 17 คลาส Java พื้นฐาน)

**วันที่จัดทำ:** พฤศจิกายน 2026
**เวอร์ชัน:** P9 Release Candidate 1.0 (Comprehensive Edition)

# สารบัญเนื้อหา (Table of Contents)


**บทที่ 1: สถาปัตยกรรมและมโนทัศน์พื้นฐาน OOP ของระบบร้านข้าวต้มอารมณ์ดี** หน้า 3

1.1 ภาพรวมระบบและการจำลองการทำงานจริง (8 Use Cases: UC01–UC08)

1.2 มโนทัศน์ OOP ที่ประยุกต์ใช้ในระบบ (Encapsulation, Invariants, Static Scope, Collections)

1.3 Association vs Composition: กฎเกณฑ์และตารางเปรียบเทียบเชิงวิศวกรรม

1.4 หลักฐานเชิงประจักษ์ของ Composition ทั้ง 5 คู่ (destroy cascade)

**บทที่ 2: ผ่าโค้ดเจาะลึก 16 คลาสโมเดลใน src/model (The 16 Model Classes)** หน้า 7

**ส่วนที่ 2.1: โดเมนบุคคลและสถานที่** — Customer, Employee, Table, Member

**ส่วนที่ 2.2: โดเมนเมนู คู่มือ และสูตรอาหาร** — Menu, Recipe, SOP, SOPStep

**ส่วนที่ 2.3: โดเมนคลังวัตถุดิบและการจัดซื้อ** — Ingredient, StockMovement, Supplier, PurchaseOrder

**ส่วนที่ 2.4: โดเมนออเดอร์และการเงิน** — OrderItem, Order, Payment, PointTransaction

**บทที่ 3: เจาะลึก Main.java ตัวควบคุมระบบและการทำงาน 8 Use Cases** หน้า 45

3.1 โครงสร้างภาพรวมและบทบาท Presentation / Controller

3.2 ตัวแปรส่วนกลาง (Shared In-Memory Collections) และพนักงานตัวอย่าง

3.3 การสร้างข้อมูลตั้งต้น createSampleData() (ผัดกะเพราปลา, ยำหมูยอ, สต็อก)

3.4 เครื่องมือและ Helper Functions ในการรับและตรวจสอบอินพุต

3.5 เจาะลึกการทำงานของเมธอด Use Cases ทั้ง 8 งาน (uc01() ถึง uc08()) พร้อมซอร์สโค้ดประกอบ

**บทที่ 4: การเดินทางของข้อมูลแบบครบวงจร (Life of an Order: End-to-End Walkthrough)** หน้า 65

4.1 ตามรอยออเดอร์ตั้งแต่เปิดโต๊ะจนถึงคิดเงินและตัดสต็อก (7 สเต็ปครบวงจร)

4.2 แผนภาพและตาราง State Machine Diagram ของ Order และ OrderItem

**บทที่ 5: คลังข้อสอบและแนวทางตอบคำถามสำหรับนำเสนอ (Defense & Exam Cheatsheet)** หน้า 70

รวบรวม 15 คำถามปราบเซียนที่อาจารย์มักถาม พร้อมคำตอบระดับเกียรตินิยม

**บทที่ 6: บทสรุปและแนวทางการพัฒนาต่อยอดสู่ระดับ Enterprise (Future Roadmap)** หน้า 76

จาก In-Memory สู่ Relational Database (JPA / PostgreSQL) และ Spring Boot Web API

# บทที่ 1: สถาปัตยกรรมและมโนทัศน์พื้นฐาน OOP ของระบบร้านข้าวต้มอารมณ์ดี

## 1.1 ภาพรวมระบบและการจำลองการทำงานจริง (8 Use Cases)

ระบบร้านอาหาร **"ข้าวต้มอารมณ์ดี" (Khao Tom Arom Dee)** เป็นโครงงานพัฒนาระบบตามระเบียบวิธีวิเคราะห์และออกแบบเชิงวัตถุ (Object-Oriented Analysis and Design: OOAD) ตั้งแต่ระยะ P1 ถึง P8 และถูกแปลงเป็นซอร์สโค้ดภาษา Java ในระยะ P9 โดยจำลองการทำงานผ่านคอนโซล (Text Mode Console) เพื่อให้ผู้เรียนสามารถทำความเข้าใจโครงสร้างเชิงวัตถุได้อย่างบริสุทธิ์โดยไม่มีความซับซ้อนของเฟรมเวิร์กเว็บหรือฐานข้อมูลมาบดบัง

ระบบประกอบด้วยงานหลัก 8 ด้าน (Use Cases) ที่ครอบคลุมการดำเนินงานทั้งหมดของร้านอาหาร ได้แก่:

| รหัส UC | ชื่องาน (Use Case) | ผู้ใช้งานหลัก (Actors) | ความรับผิดชอบหลักของระบบ |
|---|---|---|---|
| **UC01** | สั่งอาหารผ่าน QR โต๊ะ | ลูกค้า / พนักงานเสิร์ฟ | สแกนโต๊ะ, ผูกสมาชิก, เลือกเมนูและ Topping, ตรวจสอบสต็อก และยืนยันออเดอร์ |
| **UC02** | จัดการคู่มือ SOP | ผู้จัดการ / เชฟใหญ่ | สร้างคู่มือมาตรฐานการปรุงอาหาร, กำหนดขั้นตอนย่อยและเวลา, เผยแพร่ และปิดใช้งาน |
| **UC03** | ปรุงอาหารและเสิร์ฟ | คนครัว / พนักงานเสิร์ฟ | เลือกสถานีครัว, รับออเดอร์เข้าคิว, เริ่มปรุง, จับเวลาเทียบ SOP, เสิร์ฟถึงโต๊ะ |
| **UC04** | ชำระเงินและออกใบเสร็จ | แคชเชียร์ / ลูกค้า | เลือกโต๊ะที่เสิร์ฟครบ, แลกแต้มส่วนลด, คำนวณเงินทอน, ปิดบิล, แจกแต้มใหม่, ปล่อยโต๊ะว่าง |
| **UC05** | สต็อกและใบสั่งซื้อ | ผู้จัดการ / พนักงานคลัง | ตรวจสต็อกต่ำกว่าเกณฑ์, ออกใบสั่งซื้อ PO ให้ซัพพลายเออร์, รับของเข้าคลัง, ปรับยอดตรวจนับ |
| **UC06** | ภาพรวมและรายงานยอดขาย | ผู้จัดการ / เจ้าของร้าน | สรุปรายได้ตามช่วงวันที่, แยกยอดเงินสดและ QR, วิเคราะห์อันดับเมนูขายดี |
| **UC07** | จัดการเมนูและสูตรอาหาร | ผู้จัดการ / เชฟ | เพิ่มเมนู, ตั้งราคา, สร้างสูตรฉบับร่าง, ยืนยันสูตรสมบูรณ์, แตกเวอร์ชันสูตร, เพิ่มตัวเลือกเสริม |
| **UC08** | สมาชิกและแต้มสะสม | ลูกค้า / แคชเชียร์ / ผู้จัดการ | สมัครสมาชิก, ค้นหาประวัติแต้ม, ผู้จัดการอนุมัติคำขอยกเลิกอาหาร, ปรับกฎแต้มกลางของร้าน |

## 1.2 มโนทัศน์ OOP ที่นำมาใช้ในระบบร้านข้าวต้มอารมณ์ดี

### 1. Encapsulation (การห่อหุ้มและการซ่อนข้อมูล)

ฟิลด์ข้อมูลทั้งหมดใน 16 คลาสโมเดลถูกประกาศเป็น `private` อย่างเคร่งครัด เพื่อป้องกันไม่ให้อ็อบเจกต์ภายนอกหรือคลาส `Main` แก้ไขค่าได้โดยพลการ เช่น ยอดสต็อก `stockQty` ใน `Ingredient` ไม่สามารถถูกเขียนทับด้วยเครื่องหมายเท่ากับ (=) ได้ แต่จะต้องผ่านเมธอดทางธุรกิจ เช่น `deduct()`, `receive()` หรือ `adjustStock()` เท่านั้น ซึ่งแต่ละเมธอดจะมีเงื่อนไขตรวจสอบ (Validation) และจะสร้างประวัติ `StockMovement` เสมอ

### 2. Invariants & State Validation (การรักษาสภาวะความถูกต้องของระบบ)

ทุกอ็อบเจกต์มีหน้าที่รักษากฎความถูกต้องของตนเอง (Invariants) หากมีคำสั่งใดที่ขัดแย้งกับกฎ อ็อบเจกต์จะโยน Exception ออกมาทันที:

- **IllegalArgumentException:** ใช้เมื่อพารามิเตอร์ที่ส่งเข้ามาไม่ถูกต้อง เช่น เบอร์โทรศัพท์ไม่ใช่ 10 หลัก (`!phone.matches("0[0-9]{9}")`), ปริมาณติดลบ, หรือราคาอาหารเท่ากับ 0
- **IllegalStateException:** ใช้เมื่อพยายามทำงานที่ขัดกับสถานะปัจจุบันของอ็อบเจกต์ เช่น พยายามเปิดขายเมนูที่ยังไม่มีสูตรสมบูรณ์ (`currentRecipe() == null`), พยายามยกเลิกอาหารที่เริ่มปรุงไปแล้วโดยไม่ผ่านผู้จัดการ, หรือพยายามรับของซ้ำในใบสั่งซื้อที่รับไปแล้ว

### 3. Class Scope (static) vs Instance Scope

ในคลาส `Member` มีการประยุกต์ใช้ `static` ได้อย่างชัดเจน:

- **Instance Variables:** `memberId`, `name`, `phone`, `points` และ `transactions` เป็นของสมาชิกแต่ละคน (แต่ละอ็อบเจกต์มีค่าของตัวเอง)
- **Static Variables (Class Level):** `bahtPerPoint` (50 บาท/แต้ม), `pointValue` (1 บาท/แต้ม), และ `minRedeem` (10 แต้ม) เป็นกฎกลางของร้านอาหาร เมื่อผู้จัดการแก้ไขผ่าน `Member.setRule()` จะมีผลบังคับใช้กับสมาชิกทุกคนในร้านทันทีโดยไม่ต้องไปแก้ในสมาชิกทีละคน

## 1.3 Association (has-a) vs Composition (owns-a)

หนึ่งในหัวใจสำคัญที่สุดของการออกแบบเชิงวัตถุคือการทำความเข้าใจความสัมพันธ์แบบ **has-a** ซึ่งใน UML และ OOAD แบ่งออกเป็น 2 ประเภทหลัก:

| มิติการเปรียบเทียบ | Association (ความสัมพันธ์แบบรู้จักกัน / has-a) | Composition (ความเป็นเจ้าของส่วนประกอบ / owns-a) |
|---|---|---|
| **สัญลักษณ์ UML** | เส้นตรงธรรมดา หรือเส้นมีหัวลูกศรทิศทางเดียว (——>) | เส้นตรงที่มี **สัญลักษณ์ข้าวหลามตัดทึบ (◆)** อยู่ฝั่งเจ้าของ |
| **ความเป็นเจ้าของ** | ต่างฝ่ายต่างมีชีวิตเป็นของตัวเอง (Loose relationship) | ฝั่ง Composite เป็นเจ้าของส่วนประกอบย่อย (Part-of) โดยสมบูรณ์ |
| **วงจรชีวิต (Lifecycle)** | เป็นอิสระต่อกัน การทำลายอ็อบเจกต์หนึ่งไม่ส่งผลต่ออีกอ็อบเจกต์ | **ผูกติดกัน (Bound lifecycle)** เมื่ออ็อบเจกต์แม่ถูกทำลาย ส่วนย่อยจะถูกทำลายตามไปด้วย |
| **ตัวอย่างในโปรเจกต์** | `Order` อ้างถึง `Table`<br>`Customer` อ้างถึง `Member`<br>`PurchaseOrder` อ้างถึง `Supplier` | `Order` ◆—— `OrderItem`<br>`Menu` ◆—— `Recipe`<br>`SOP` ◆—— `SOPStep`<br>`Ingredient` ◆—— `StockMovement`<br>`Member` ◆—— `PointTransaction` |

## 1.4 หลักฐานเชิงประจักษ์ของ Composition ทั้ง 5 คู่ในโค้ด Java

เนื่องจากภาษา Java มีระบบ Garbage Collection ที่จัดการคืนหน่วยความจำอัตโนมัติ การแสดงความเป็น Composition ในระดับโค้ดจึงแสดงออกผ่าน 3 ปัจจัยหลัก:

1. **การถือครองข้อมูล:** อ็อบเจกต์แม่เก็บ List ของอ็อบเจกต์ลูกไว้ภายใน
2. **การให้กำเนิด:** อ็อบเจกต์แม่มีเมธอดสร้างอ็อบเจกต์ลูกขึ้นมาเอง เช่น `order.addItem(...)` สร้าง `OrderItem`, `sop.addStep(...)` สร้าง `SOPStep`
3. **การทำลายวงจรชีวิต (Lifecycle Cascade):** อ็อบเจกต์แม่มีเมธอด `destroy()` ที่จะวนลูปเรียก `destroy()` ของอ็อบเจกต์ลูกทุกตัวก่อนที่จะเคลียร์ List ทิ้ง

| อ็อบเจกต์แม่ (Composite) | อ็อบเจกต์ลูก (Part-of) | เมธอดสร้างลูก (Creation) | หลักฐานการทำลาย (destroy cascade) ในโค้ดจริง |
|---|---|---|---|
| ORDER | OrderItem | `order.addItem(...)` | `for (OrderItem item : items) item.destroy(); items.clear();` |
| MENU | Recipe | `menu.addRecipe(...)` | `for (Recipe r : recipes) r.destroy(); recipes.clear();` |
| SOP | SOPStep | `sop.addStep(...)` | `for (SOPStep s : steps) s.destroy(); steps.clear();` |
| INGREDIENT | StockMovement | `ingredient.changeStock(...)` | `for (StockMovement m : movements) m.destroy(); movements.clear();` |
| MEMBER | PointTransaction | `member.addTransaction(...)` | `for (PointTransaction t : transactions) t.destroy(); transactions.clear();` |

# บทที่ 2: ผ่าโค้ดเจาะลึก 16 คลาสโมเดลใน src/model

ในโฟลเดอร์ `src/model` ประกอบด้วยคลาสสาธารณะจำนวน 16 คลาสพอดี ไม่มีคลาสซ้อนภายนอก ไม่มีไฟล์ enum แยกย่อย และไม่มีคลาสอำนวยความสะดวกแปลกปลอม เพื่อให้คงความเรียบง่ายตามหลักการเรียนรู้ OOP พื้นฐาน โดยแบ่งออกเป็น 4 โดเมนธุรกิจหลัก ดังนี้:

## 2.1 คลาส Customer (Customer.java)

โดเมนบุคคลและสถานที่ (PEOPLE & TABLE)

**บทบาทหน้าที่ (Responsibility):** ทำหน้าที่เป็นตัวแทนของลูกค้าที่เข้ามาใช้บริการและเปิดคำสั่งซื้อในร้าน โดยลูกค้าทั่วไปสามารถเข้ามาสั่งอาหารได้ทันทีโดยไม่จำเป็นต้องเป็นสมาชิก (member สามารถเป็น null ได้) และสามารถผูกบัญชีสมาชิกในภายหลังได้เมื่อลูกค้าแจ้งเบอร์โทรศัพท์

**ความสัมพันธ์ UML:** `Customer --(0..1)--> Member`

### ตารางฟิลด์และตัวแปรข้อมูล (Fields & Attributes)

| ชนิดข้อมูลและการเข้าถึง | ชื่อตัวแปร (Field) | คำอธิบายหน้าที่ในระบบ |
|---|---|---|
| `private String` | **customerId** | รหัสประจำตัวลูกค้า เช่น 'CUS-1' |
| `private Member` | **member** | ตัวอ้างอิงไปยังอ็อบเจกต์ Member ที่ผูกไว้ (has-a Association) หากไม่ได้เป็นสมาชิกจะมีค่าเป็น null |

### ตารางเมธอดสำคัญ (Key Methods & Operations)

| รูปแบบเมธอด (Signature) | หมวดหมู่การทำงาน | คำอธิบายรายละเอียดการทำงาน |
|---|---|---|
| `Customer(String customerId)` | CONSTRUCTOR | กำหนดรหัสลูกค้า โดยตั้งต้นค่า member เป็น null ตามธรรมชาติของลูกค้าทั่วไป |
| `linkedMember() : Member` | QUERY / ACCESSOR | คืนค่าอ็อบเจกต์ Member ที่ผูกอยู่ เพื่อให้ Order และ Payment ดึงไปใช้คำนวณแต้มและส่วนลด |
| `linkMember(Member member)` | MUTATOR | ผูกบัญชีสมาชิกเข้ากับลูกค้าหลังจากค้นหาพบเบอร์โทรศัพท์ในระบบ |
| `checkCustomerStatus() : boolean` | BUSINESS LOGIC (P7) | ตรวจสอบว่าลูกค้ารายนี้เป็นสมาชิกหรือไม่ โดยคืนค่า true หาก member != null และ false หากยังไม่ได้ผูกสมาชิก |

**จุดสังเกตการออกแบบเชิงวัตถุ (OOP Design Insight):**

ใน Class Diagram P7 เมธอด checkCustomerStatus() คืนค่า boolean เพื่อตอบคำถามว่ามีสถานะสมาชิกหรือไม่ แต่ในขั้นตอนชำระเงินจริง ระบบจำเป็นต้องเข้าถึงแต้มของสมาชิกด้วย ดังนั้นจึงมีการเพิ่มเมธอด linkedMember() และ linkMember() ขึ้นมาเพื่อสนับสนุนการทำงานจริงโดยที่ยังคงรักษา Encapsulation ไว้อย่างสมบูรณ์

**คำถาม-คำตอบยอดฮิตสำหรับการนำเสนอ (Defense Q&A):**

ทำไม Customer ถึงไม่สืบทอด (Inheritance) มาจาก Member? -> เพราะลูกค้าไม่ได้ 'เป็น' สมาชิกเสมอไป ลูกค้า 1 คนอาจเลือกไม่เป็นสมาชิกก็ได้ และหากเป็นสมาชิก ก็เป็นการ 'ถือครองสิทธิ' (Role/Association) ไม่ใช่การสืบทอดพันธุกรรมทางคลาส

## ซอร์สโค้ดฉบับสมบูรณ์ (Full Source Code: Customer.java)

src/model/Customer.java — Java Source Code

```java
package model;

// ลูกค้าไม่จำเป็นต้องเป็นสมาชิก จึงให้ member เป็น null ได้
public class Customer {
    // ลูกค้าทั่วไปมี member เป็น null; ผูกสมาชิกเมื่อแจ้งเบอร์โทร
    private String customerId;
    // has-a (Association): Customer ผูกกับ Member ได้ แต่ไม่ได้เป็นเจ้าของ Member
    private Member member;

    // สร้างลูกค้าใหม่; member เริ่มเป็น null จนกว่าจะผูกสมาชิก
    public Customer(String customerId) {
        this.customerId = customerId;
    }

    // อ่านบัญชีสมาชิกที่ลูกค้าเชื่อมไว้ เพื่อใช้คำนวณแต้มตอนชำระเงิน
    public Member linkedMember() {
        return member;
    }

    // ผูกสมาชิกภายหลังได้ เพราะลูกค้าเริ่มสั่งอาหารโดยยังไม่เป็นสมาชิก
    public void linkMember(Member member) {
        this.member = member;
    }

    // P7: + checkCustomerStatus() : boolean
    // คืน true เมื่อเชื่อมบัญชีสมาชิกแล้ว และคืน false เมื่อยังไม่ได้เชื่อม
    public boolean checkCustomerStatus() {
        if (member == null) {
            return false;
        }
        return true;
    }
}

```

## 2.2 คลาส Employee (Employee.java)

โดเมนบุคคลและสถานที่ (PEOPLE & TABLE)

**บทบาทหน้าที่ (Responsibility):** ทำหน้าที่เก็บข้อมูลของพนักงานร้านอาหาร รวมถึงการควบคุมสิทธิ์การเข้าถึงการทำงานต่าง ๆ ตามบทบาทหน้าที่ (Role-Based Access Control) เช่น ผู้จัดการ, แคชเชียร์, หรือคนครัว และบันทึกสถานีครัวที่รับผิดชอบ

**ความสัมพันธ์ UML:** `Employee`

### ตารางฟิลด์และตัวแปรข้อมูล (Fields & Attributes)

| ชนิดข้อมูลและการเข้าถึง | ชื่อตัวแปร (Field) | คำอธิบายหน้าที่ในระบบ |
|---|---|---|
| `public enum { MANAGER, CASHIER, COOK }` | **Role** | ชุดค่าคงที่กำหนดบทบาทหน้าที่ของพนักงาน |
| `private String` | **employeeId** | รหัสประจำตัวพนักงาน เช่น 'EMP-1' |
| `private String` | **name** | ชื่อ-นามสกุลของพนักงาน เช่น 'ผู้จัดการ', 'คนครัว' |
| `private Role` | **role** | บทบาทหน้าที่ของพนักงานตัวนี้ |
| `private String` | **station** | สถานีครัวที่พนักงานได้รับมอบหมาย เช่น 'ครัวร้อน', 'ครัวยำ' |

### ตารางเมธอดสำคัญ (Key Methods & Operations)

| รูปแบบเมธอด (Signature) | หมวดหมู่การทำงาน | คำอธิบายรายละเอียดการทำงาน |
|---|---|---|
| `Employee(String id, String name, Role role)` | CONSTRUCTOR | กำหนดรหัส ชื่อ และบทบาท enum ตั้งต้น โดยกำหนด station เริ่มต้นเป็น 'ยังไม่กำหนด' |
| `Employee(String id, String name, String role)` | OVERLOADED CONSTRUCTOR | Constructor เสริมที่รับสตริงภาษาไทย ('ผู้จัดการ', 'แคชเชียร์', 'คนครัว') แล้วแปลงเป็น enum อัตโนมัติ |
| `parseRole(String text) : Role` | HELPER (PRIVATE STATIC) | แปลงข้อความภาษาไทยจากหน้าจอเป็นค่า Enum ป้องกันการใส่ข้อมูลผิดพลาด |
| `authenticateRole(Role requiredRole) : boolean` | SECURITY / AUTH | ตรวจสอบว่าพนักงานมีบทบาทตรงกับงานที่จำเป็นต้องใช้สิทธิ์หรือไม่ |
| `assignStationDuty(String station)` | MUTATOR | มอบหมายหน้าที่ประจำสถานีครัวให้แก่คนครัว |
| `name(), roleName(), station()` | ACCESSORS | เมธอดอ่านค่าสำหรับนำไปแสดงผลบนหน้าจอ Text Mode |

**จุดสังเกตการออกแบบเชิงวัตถุ (OOP Design Insight):**
การใช้ enum ซ้อนอยู่ภายในคลาส Employee (Employee.Role) ช่วยจำกัดขอบเขตบทบาทไม่ให้สะกดผิดพลาด และยังคงทำให้โครงสร้างไฟล์ยังคงเป็น 16 คลาสหลักอย่างเคร่งครัดตามข้อกำหนด ไม่มีการแตกไฟล์ enum ย่อยภายนอก

**คำถาม-คำตอบยอดฮิตสำหรับการนำเสนอ (Defense Q&A):**

ทำไมต้องมี authenticateRole()? -> เพื่อป้องกันไม่ให้คนครัวหรือแคชเชียร์มากดอนุมัติคำขอยกเลิกอาหาร หรือแก้กฎแต้มกลางของร้านซึ่งเป็นอำนาจเฉพาะของผู้จัดการเท่านั้น

## ซอร์สโค้ดฉบับสมบูรณ์ (Full Source Code: Employee.java)

**src/model/Employee.java** — Java Source Code

```java
package model;

// พนักงานแต่ละคนมีบทบาท เช่น ผู้จัดการ แคชเชียร์ หรือคนครัว
public class Employee {
    public enum Role { MANAGER, CASHIER, COOK }

    // ข้อมูลประจำตัวพนักงาน; station เปลี่ยนได้เมื่อรับหน้าที่ในครัว
    private String employeeId;
    private String name;
    private Role role;
    private String station;

    // สร้างพนักงานโดยระบุรหัส ชื่อ และบทบาท
    public Employee(String employeeId, String name, Role role) {
        this.employeeId = employeeId;
        this.name = name;
        this.role = role;
        this.station = "ยังไม่กำหนด";
    }

    // รับค่าภาษาไทยจาก Main เพื่อให้อ่านตัวอย่างได้ง่าย
    public Employee(String employeeId, String name, String role) {
        this(employeeId, name, parseRole(role));
    }

    // แปลงคำบทบาทภาษาไทยจากหน้าจอเป็นค่า Role
    private static Role parseRole(String text) {
        if (text.equals("ผู้จัดการ")) return Role.MANAGER;
        if (text.equals("แคชเชียร์")) return Role.CASHIER;
        if (text.equals("คนครัว")) return Role.COOK;
        throw new IllegalArgumentException("บทบาทพนักงานไม่ถูกต้อง");
    }

    // ตรวจสิทธิ์ว่าบทบาทตรงกับงานที่กำหนดหรือไม่
    public boolean authenticateRole(Role requiredRole) {
        return role == requiredRole;
    }

    // บันทึกสถานีครัวที่พนักงานรับผิดชอบ
    public void assignStationDuty(String station) {
        this.station = station;
    }

    // เมธอดด้านล่างให้ Main อ่านรหัส ชื่อ บทบาท และสถานีของพนักงาน

    public String name() { return name; }
    public String roleName() {
        if (role == Role.MANAGER) return "ผู้จัดการ";
        if (role == Role.CASHIER) return "แคชเชียร์";
        return "คนครัว";
    }
    public String station() { return station; }
}

```

## 2.3 คลาส Table (Table.java)

โดเมนบุคคลและสถานที่ (PEOPLE & TABLE)

**บทบาทหน้าที่ (Responsibility):** ทำหน้าที่เป็นตัวแทนของโต๊ะรับประทานอาหารภายในร้าน จัดการรหัสโต๊ะ รหัส QR สำหรับการสั่งอาหาร จำนวนที่นั่ง และสถานะของโต๊ะ (ว่าง หรือ ใช้งาน)

**ความสัมพันธ์ UML:** `Table <--(1)-- Order`

### ตารางฟิลด์และตัวแปรข้อมูล (Fields & Attributes)

| ชนิดข้อมูลและการเข้าถึง | ชื่อตัวแปร (Field) | คำอธิบายหน้าที่ในระบบ |
|---|---|---|
| `private String` | **tableId** | รหัสโต๊ะ เช่น 'T01', 'T02' |
| `private String` | **qrCode** | รหัส QR จำลองประจำโต๊ะ เช่น 'QR-T01' |
| `private int` | **seatCapacity** | จำนวนที่นั่งรองรับ เช่น 4 ที่นั่ง |
| `private String` | **status** | สถานะของโต๊ะ ('ว่าง' หรือ 'ใช้งาน') |
| `private boolean` | **qrScanned** | แฟล็กจำลองว่าโต๊ะนี้ถูกลูกค้าสแกน QR แล้วหรือไม่ |

### ตารางเมธอดสำคัญ (Key Methods & Operations)

| รูปแบบเมธอด (Signature) | หมวดหมู่การทำงาน | คำอธิบายรายละเอียดการทำงาน |
|---|---|---|
| `Table(String tableId, String qrCode, int seatCapacity)` | CONSTRUCTOR | กำหนดค่าเริ่มต้นของโต๊ะ โดยตั้งค่าสถานะเริ่มต้นเป็น 'ว่าง' |
| `scanTableQRCode()` | SIMULATION | จำลองการที่ลูกค้านำมือถือมาสแกน QR Code เพื่อเริ่มกระบวนการสั่งอาหาร |
| `identifyTableNumber() : String` | BUSINESS LOGIC (P7) | คืนรหัสโต๊ะ แต่มีเงื่อนไขตรวจสอบว่าต้องผ่านการสแกน QR มาก่อน (qrScanned == true) มิเช่นนั้นจะโยน IllegalStateException |
| `changeTableStatus(String status)` | STATE MUTATOR | เปลี่ยนสถานะโต๊ะ เช่น เมื่อยืนยันออเดอร์จะเปลี่ยนเป็น 'ใช้งาน' และเมื่อชำระเงินเรียบร้อยจะเปลี่ยนเป็น 'ว่าง' |
| `tableId(), qrCode(), statusText()` | ACCESSORS | อ่านค่าสำหรับแสดงผลบนหน้าจอ |

**จุดสังเกตการออกแบบเชิงวัตถุ (OOP Design Insight):**

ในระบบ Terminal ไม่มีกล้องจริง การจำลองสแกน QR ผ่าน scanTableQRCode() และตรวจสอบใน identifyTableNumber() ช่วยสะท้อน Use Case Diagram และ Sequence Diagram P6 ได้อย่างตรงไปตรงมา

**คำถาม-คำตอบยอดฮิตสำหรับการนำเสนอ (Defense Q&A):**

ทำไม Table ไม่เก็บ Order ไว้ในตัวเอง? -> เพื่อลด Coupling (Loose Coupling) โต๊ะเป็นเพียงสถานที่กายภาพ ไม่ควรผูกมัดกับโครงสร้างบิลเงินหรือรายการอาหาร ให้ Order เป็นฝ่ายชี้มาที่ Table จะยืดหยุ่นกว่า

## ซอร์สโค้ดฉบับสมบูรณ์ (Full Source Code: Table.java)

**src/model/Table.java** — Java Source Code

```java
package model;

// โต๊ะหนึ่งตัวเป็น object ที่มีรหัส QR จำนวนที่นั่ง และสถานะของตัวเอง
public class Table {
    // QR ระบุตัวโต๊ะ ส่วนสถานะใช้บอกว่าว่างหรือกำลังใช้งาน
    private String tableId;
    private String qrCode;
    private int seatCapacity;
    private String status;
    private boolean qrScanned;

    // สร้างโต๊ะพร้อม QR และจำนวนที่นั่ง; สถานะเริ่มเป็นว่าง
    public Table(String tableId, String qrCode) {
        this(tableId, qrCode, 4);
    }

    // Constructor ถูกเรียกเมื่อใช้ new Table(...)
    public Table(String tableId, String qrCode, int seatCapacity) {
        this.tableId = tableId;
        this.qrCode = qrCode;
        this.seatCapacity = seatCapacity;
        this.status = "ว่าง";
    }

    // เมธอดอ่านข้อมูลโต๊ะสำหรับแสดงบนหน้าจอ
    public String tableId() {
        return tableId;
    }

    public String qrCode() {
        return qrCode;
    }

    // Terminal จำลองการสแกน: ไม่มีการเปิดกล้องจริง
    public void scanTableQRCode() {
        qrScanned = true;
    }

    // คืนรหัสโต๊ะเมื่อสแกน QR แล้วเท่านั้น
    public String identifyTableNumber() {
        if (!qrScanned) throw new IllegalStateException("ยังไม่ได้สแกน QR ของโต๊ะ");
        return tableId;
    }


    public String statusText() {
        return status;
    }

    // เปลี่ยนสถานะโต๊ะเมื่อเปิดหรือปิดการใช้งาน
    public void changeTableStatus(String status) {
        this.status = status;
    }
}

```

## 2.4 คลาส Supplier (Supplier.java)

โดเมนคลังวัตถุดิบและการจัดซื้อ (INVENTORY & PURCHASING)

**บทบาทหน้าที่ (Responsibility):** ทำหน้าที่เก็บข้อมูลของผู้จัดจำหน่ายวัตถุดิบ (ซัพพลายเออร์) และรายการวัตถุดิบที่ซัพพลายเออร์รายนั้นสามารถจัดส่งให้ร้านได้ เพื่อใช้ตรวจสอบตอนออกใบสั่งซื้อ (Purchase Order)

**ความสัมพันธ์ UML:** `Supplier --(*)--> Ingredient`

### ตารางฟิลด์และตัวแปรข้อมูล (Fields & Attributes)

| ชนิดข้อมูลและการเข้าถึง | ชื่อตัวแปร (Field) | คำอธิบายหน้าที่ในระบบ |
|---|---|---|
| `private String` | **supplierId** | รหัสประจำตัวผู้จัดจำหน่าย เช่น 'SUP-1' |
| `private String` | **name** | ชื่อบริษัทหรือร้านค้าผู้จัดจำหน่าย เช่น 'เจริญฟาร์มปลาและไข่' |
| `private List<Ingredient>` | **suppliedIngredients** | รายการอ็อบเจกต์วัตถุดิบที่ผู้จัดจำหน่ายรายนี้สามารถส่งได้ (Association) |

### ตารางเมธอดสำคัญ (Key Methods & Operations)

| รูปแบบเมธอด (Signature) | หมวดหมู่การทำงาน | คำอธิบายรายละเอียดการทำงาน |
|---|---|---|
| `Supplier(String supplierId, String name)` | CONSTRUCTOR | กำหนดรหัส ชื่อผู้จัดจำหน่าย และเตรียม List วัตถุดิบ |
| `addIngredient(Ingredient ingredient)` | MUTATOR | เพิ่มวัตถุดิบที่จัดส่งได้เข้าสู่รายการ โดยตรวจสอบป้องกันการเพิ่มซ้ำด้วย contains() |
| `supplies(Ingredient ingredient) : boolean` | VALIDATION (P7) | ตรวจสอบว่าผู้จัดจำหน่ายรายนี้สามารถจัดส่งวัตถุดิบที่ต้องการได้หรือไม่ |
| `name() : String` | ACCESSOR | อ่านชื่อผู้จัดจำหน่าย |

**จุดสังเกตการออกแบบเชิงวัตถุ (OOP Design Insight):**
Supplier ใช้ List ร่วมกับ Ingredient กลางของร้าน ทำให้เมื่อออกใบสั่งซื้อ ระบบสามารถตรวจ supplies(ingredient) ได้ทันทีโดยไม่ต้องเปรียบเทียบแค่ข้อความชื่อ

**คำถาม-คำตอบยอดฮิตสำหรับการนำเสนอ (Defense Q&A):**
ทำไม Supplier ไม่ลบ Ingredient เมื่อ Supplier เลิกกิจการ? -> เพราะเป็น Association ไม่ใช่ Composition วัตถุดิบเป็นทรัพย์สินของร้านอาหาร แม้เปลี่ยนผู้จัดจำหน่าย วัตถุดิบในคลังก็ยังต้องอยู่

## ซอร์สโค้ดฉบับสมบูรณ์ (Full Source Code: Supplier.java)

src/model/Supplier.java — Java Source Code

```java
package model;

import java.util.ArrayList;
import java.util.List;

// ผู้จำหน่ายอ้างถึงวัตถุดิบในร้าน ไม่ได้เป็นเจ้าของอายุของวัตถุดิบ
public class Supplier {
    // เก็บรายการ Ingredient ที่รายนี้ส่งได้ โดยอ้างถึง object ในคลังเดียวกัน
    private String supplierId;
    private String name;
    // has-a (Association): Supplier อ้างถึง Ingredient ที่ส่งได้ ไม่ได้เป็นเจ้าของสต็อก
    private List<Ingredient> suppliedIngredients;

    // สร้างผู้จำหน่ายและเตรียมรายการวัตถุดิบที่ส่งได้
    public Supplier(String supplierId, String name) {
        this.supplierId = supplierId;
        this.name = name;
        this.suppliedIngredients = new ArrayList<Ingredient>();
    }

    // เพิ่มวัตถุดิบที่รายนี้ส่งได้โดยไม่เพิ่มซ้ำ
    public void addIngredient(Ingredient ingredient) {
        if (!suppliedIngredients.contains(ingredient))
            suppliedIngredients.add(ingredient);
    }

    // ตรวจว่าผู้จำหน่ายรายนี้ส่งวัตถุดิบชนิดนี้หรือไม่
    public boolean supplies(Ingredient ingredient) {
        return suppliedIngredients.contains(ingredient);
    }

    // หน้าจออ่านชื่อผู้จำหน่ายผ่านเมธอด เพราะ field เป็น private
    public String name() {
        return name;
    }

}

```

## 2.5 คลาส SOPStep (SOPStep.java)

โดเมนเมนู คู่มือ และสูตรอาหาร (MENU, RECIPE & SOP)

**บทบาทหน้าที่ (Responsibility):** ทำหน้าที่เป็นตัวแทนของขั้นตอนย่อย 1 ขั้นตอนในคู่มือการปฏิบัติงานมาตรฐาน (SOP) เช่น 'เตรียมเนื้อปลาและล้างให้สะอาด' กำหนดเวลามาตรฐาน (นาที) และรองรับการทำลายตามวงจรชีวิตของ SOP

**ความสัมพันธ์ UML:** `SOPStep <--(Composition)-- SOP`

### ตารางฟิลด์และตัวแปรข้อมูล (Fields & Attributes)

| ชนิดข้อมูลและการเข้าถึง | ชื่อตัวแปร (Field) | คำอธิบายหน้าที่ในระบบ |
|---|---|---|
| `private String` | **sopStepId** | รหัสขั้นตอน เช่น 'SOP-1-STP-1' |
| `private int` | **no** | ลำดับของขั้นตอน เริ่มต้นที่ 1 |
| `private String` | **description** | คำอธิบายรายละเอียดการปฏิบัติงาน |
| `private double` | **standardMinutes** | เวลามาตรฐานในการทำงานสำหรับขั้นตอนนี้ (นาที) |
| `private boolean` | **destroyed** | แฟล็กระบุว่าขั้นตอนนี้ถูกทำลายตาม SOP แล้วหรือไม่ |

### ตารางเมธอดสำคัญ (Key Methods & Operations)

| รูปแบบเมธอด (Signature) | หมวดหมู่การทำงาน | คำอธิบายรายละเอียดการทำงาน |
|---|---|---|
| `SOPStep(String id, int no, String desc, double minutes)` | CONSTRUCTOR | กำหนดรหัส ลำดับ และเรียก update() เพื่อตรวจสอบความถูกต้อง |
| `update(String description, double minutes)` | MUTATOR / VALIDATION | แก้ไขคำอธิบายและเวลามาตรฐาน โดยตรวจเงื่อนไข minutes > 0 และเป็นตัวเลขจำกัด |
| `destroy()` | LIFECYCLE (COMPOSITION) | ทำเครื่องหมาย destroyed = true ปลดการใช้งานเมื่อคู่มือแม่ถูกลบ |
| `number(), description(), standardMinutes(), isDestroyed()` | ACCESSORS | อ่านข้อมูลของขั้นตอนย่อย |

**จุดสังเกตการออกแบบเชิงวัตถุ (OOP Design Insight):**
การมีเมธอด destroy() ใน SOPStep เป็นหลักฐานเชิงประจักษ์ของการออกแบบตามหลัก Composition ในภาษา Java เมื่ออ็อบเจกต์ SOP ถูกเรียก destroy() จะทำการวนลูปเรียก destroy() ของทุกขั้นตอนย่อยด้วย

**คำถาม-คำตอบยอดฮิตสำหรับการนำเสนอ (Defense Q&A):**
ทำไม SOPStep จึงไม่ควรอยู่เดี่ยว ๆ โดยไม่มี SOP? -> เพราะขั้นตอนหนึ่งขั้นตอนจะไร้ความหมายหากไม่รู้ว่ากำลังทำเมนูอะไรและเป็นส่วนหนึ่งของคู่มือฉบับใด

## ซอร์สโค้ดฉบับสมบูรณ์ (Full Source Code: SOPStep.java)

src/model/SOPStep.java — Java Source Code

```java
package model;

// หนึ่งขั้นตอน เช่น เตรียมปลา ใช้เวลา 2 นาที
public class SOPStep {
    // หนึ่ง object คือหนึ่งขั้นตอน; no เริ่มจาก 1 ตามลำดับที่เห็นบนหน้าจอ
    private String sopStepId;
    private int no;
    private String description;
    private double standardMinutes;
    private boolean destroyed;

    // สร้างขั้นตอนหนึ่งลำดับพร้อมเวลาเป้าหมาย
    public SOPStep(String sopStepId, int no, String description, double minutes) {
        this.sopStepId = sopStepId;
        this.no = no;
        update(description, minutes);
    }

    public void update(String description, double minutes) {
        if (!Double.isFinite(minutes) || minutes <= 0)
            throw new IllegalArgumentException("เวลาต้องมากกว่า 0");
        this.description = description;
        this.standardMinutes = minutes;
    }

    public void destroy() {
        destroyed = true;
    }

    // เมธอดอ่านลำดับ คำอธิบาย และเวลามาตรฐานของขั้นตอน

    public int number() {
        return no;
    }

    public String description() {
        return description;
    }

    public double standardMinutes() {
        return standardMinutes;
    }

    public boolean isDestroyed() {
        return destroyed;
    }
}

```

## 2.6 คลาส StockMovement (StockMovement.java)

โดเมนคลังวัตถุดิบและการจัดซื้อ (INVENTORY & PURCHASING)

**บทบาทหน้าที่ (Responsibility):** ทำหน้าที่เป็นบันทึกประวัติการเปลี่ยนแปลงสต็อก (Audit Trail) แต่ละครั้ง ว่ามีการรับเข้า, ตัดขาย, คืนสต็อก หรือปรับยอด จำนวนเท่าใด ยอดคงเหลือหลังรายการเท่าใด อ้างอิงเอกสารใด เหตุผลอะไร และใครเป็นผู้ทำรายการ

**ความสัมพันธ์ UML:** `StockMovement <--(Composition)-- Ingredient`

### ตารางฟิลด์และตัวแปรข้อมูล (Fields & Attributes)

| ชนิดข้อมูลและการเข้าถึง | ชื่อตัวแปร (Field) | คำอธิบายหน้าที่ในระบบ |
|---|---|---|
| `public enum { RECEIVE, SALE, RESTORE, ADJUST }` | **MovementType** | ชนิดของการเคลื่อนไหวสต็อก |
| `private String` | **stockMovementId** | รหัสรายการเคลื่อนไหว เช่น 'ING-1-MOV-1' |
| `private LocalDateTime` | **time** | วันและเวลาที่เกิดรายการขึ้นจริง |
| `private MovementType` | **type** | ชนิดรายการ |
| `private double` | **qty** | จำนวนที่เปลี่ยนแปลง (บวกคือเพิ่ม ลบคือลด) |
| `private double` | **balance** | ยอดคงเหลือหลังการทำรายการ |
| `private String` | **reference** | เอกสารอ้างอิง เช่น 'ORD-1', 'PO-1', 'ตรวจนับ' |
| `private String` | **reason** | เหตุผล เช่น 'ตัดตามสูตร', 'รับวัตถุดิบ', 'สินค้าเน่าเสีย' |
| `private Employee` | **doneBy** | พนักงานผู้รับผิดชอบทำรายการ (Association) |
| `private boolean` | **destroyed** | แฟล็กสถานะการทำลายตาม Ingredient |

### ตารางเมธอดสำคัญ (Key Methods & Operations)

| รูปแบบเมธอด (Signature) | หมวดหมู่การทำงาน | คำอธิบายรายละเอียดการทำงาน |
|---|---|---|
| `StockMovement(...)` | CONSTRUCTORS | สร้างประวัติการเคลื่อนไหวและเรียก recordStockMovement |
| `recordStockMovement(...)` | IMMUTABLE STATE RECORDING | บันทึกข้อมูลและตรวจสอบความถูกต้อง โดยมีเงื่อนไขห้ามบันทึกซ้ำ (time != null จะโยน Exception) เพื่อรักษาความถูกต้องของประวัติบัญชี |
| `destroy()` | LIFECYCLE | ทำลายประวัติและตัดการเชื่อมโยงไปยัง Employee |
| `time(), typeText(), quantity(), balance(), reference(), reason(), employee()` | ACCESSORS | เมธอดอ่านค่าสำหรับหน้าจอแสดงประวัติสต็อก |

**จุดสังเกตการออกแบบเชิงวัตถุ (OOP Design Insight):**

ประวัติสต็อกเป็นข้อมูลแบบ Write-Once-Read-Many (WORM) เมื่อถูกสร้างแล้วจะไม่มีเมธอดแก้ไขย้อนหลัง เพื่อป้องกันการทุจริตและการบิดเบือนข้อมูลสต็อกสินค้า

**คำถาม-คำตอบยอดฮิตสำหรับการนำเสนอ (Defense Q&A):**

ทำไม StockMovement ถึงเป็น Composition กับ Ingredient? -> เพราะประวัติการเคลื่อนไหวของเนื้อปลาจะไม่มีความหมายใด ๆ ในระบบ หากตัววัตถุดิบเนื้อปลาถูกลบทิ้งไปจากสารบบของร้าน

## ซอร์สโค้ดฉบับสมบูรณ์ (Full Source Code: StockMovement.java)

src/model/StockMovement.java — Java Source Code

```java
package model;

import java.time.LocalDateTime;

// ประวัติหนึ่งครั้ง: เปลี่ยนเท่าไร เหลือเท่าไร เพราะอะไร และใครทำ
public class StockMovement {
    public enum MovementType { RECEIVE, SALE, RESTORE, ADJUST }

    // ประวัติหนึ่งแถวระบุชนิด จำนวนเปลี่ยน ยอดคงเหลือ และผู้ทำรายการ
    private String stockMovementId;
    private LocalDateTime time;
    private MovementType type;
    private double qty;
    private double balance;
    private String reference;
    private String reason;
    // has-a (Association): ประวัติอ้างถึง Employee ผู้ทำรายการ
    private Employee doneBy;
    private boolean destroyed;

    // สร้างประวัติการเปลี่ยนสต็อกหนึ่งครั้ง
    public StockMovement(String stockMovementId, MovementType type, double qty, double balance,
            String ref, String reason, Employee by) {
        this.stockMovementId = stockMovementId;
        recordStockMovement(type, qty, balance, ref, reason, by);
    }

    // สร้างประวัติการเปลี่ยนสต็อกหนึ่งครั้ง
    public StockMovement(String stockMovementId, String type, double qty, double balance,
            String ref, String reason, Employee by) {
        this(stockMovementId, parseType(type), qty, balance, ref, reason, by);
    }

    // แปลงข้อความภาษาไทยเป็นชนิด MovementType
    private static MovementType parseType(String text) {
        if (text.equals("รับเข้า")) return MovementType.RECEIVE;
        if (text.equals("ขาย")) return MovementType.SALE;
        if (text.equals("คืน")) return MovementType.RESTORE;
        if (text.equals("ปรับยอด")) return MovementType.ADJUST;
        throw new IllegalArgumentException("ชนิดประวัติสต็อกไม่ถูกต้อง");
    }

    // บันทึกได้ครั้งเดียว ประวัติที่สร้างแล้วไม่ควรถูกแก้ย้อนหลัง
    public void recordStockMovement(MovementType type, double qty, double balance,
            String ref, String reason, Employee by) {
        if (time != null) throw new IllegalStateException("บันทึกประวัติซ้ำไม่ได้");
        if (type == null || !Double.isFinite(qty) || !Double.isFinite(balance))
            throw new IllegalArgumentException("ข้อมูลประวัติสต็อกไม่ถูกต้อง");
        this.time = LocalDateTime.now();
        this.type = type;
        this.qty = qty;
        this.balance = balance;
        this.reference = ref;
        this.reason = reason;
        this.doneBy = by;
    }

    public void destroy() {
        destroyed = true;
        doneBy = null;
    }

    // เมธอดอ่านประวัติสต็อกที่บันทึกไว้แล้ว

    public LocalDateTime time() { return time; }
    public String typeText() {
        if (type == MovementType.RECEIVE) return "รับเข้า";
        if (type == MovementType.SALE) return "ขาย";
```

```
69          if (type == MovementType.RESTORE) return "คืน";
70          return "ปรับยอด";
71      }
72      public double quantity() { return qty; }
73      public double balance() { return balance; }
74      public String reference() { return reference; }
75      public String reason() { return reason; }
76      public Employee employee() { return doneBy; }
77      public boolean isDestroyed() { return destroyed; }
78  }
79
```

## 2.7 คลาส PointTransaction (PointTransaction.java)

โดเมนออเดอร์และการเงิน (ORDER LIFECYCLE & FINANCIAL)

**บทบาทหน้าที่ (Responsibility):** ทำหน้าที่เป็นบันทึกประวัติการสะสมและการแลกใช้แต้มสะสมของสมาชิกแต่ละราย เพื่อความโปร่งใสและสามารถตรวจสอบย้อนหลังได้ โดยเชื่อมโยงกับใบชำระเงิน (Payment) ที่เกี่ยวข้อง

**ความสัมพันธ์ UML:** `PointTransaction <--(Composition)-- Member`

### ตารางฟิลด์และตัวแปรข้อมูล (Fields & Attributes)

| ชนิดข้อมูลและการเข้าถึง | ชื่อตัวแปร (Field) | คำอธิบายหน้าที่ในระบบ |
|---|---|---|
| `public enum { EARN, REDEEM }` | **PointType** | ประเภทรายการ: EARN (สะสมแต้ม) หรือ REDEEM (แลกแต้ม) |
| `private String` | **pointTransactionId** | รหัสรายการประวัติแต้ม เช่น 'MEM-1-PTX-1' |
| `private LocalDateTime` | **date** | วันเวลาที่เกิดรายการ |
| `private PointType` | **type** | ชนิดรายการแต้ม |
| `private int` | **points** | จำนวนแต้มที่สะสมเพิ่ม หรือจำนวนแต้มที่ใช้แลกไป |
| `private int` | **balance** | ยอดแต้มคงเหลือสุทธิหลังทำรายการ |
| `private Payment` | **payment** | อ้างอิงใบชำระเงินที่ทำให้เกิดการสะสมหรือแลกแต้ม (Association) |
| `private boolean` | **destroyed** | แฟล็กสถานะการทำลายตาม Member |

### ตารางเมธอดสำคัญ (Key Methods & Operations)

| รูปแบบเมธอด (Signature) | หมวดหมู่การทำงาน | คำอธิบายรายละเอียดการทำงาน |
|---|---|---|
| `PointTransaction(...)` | CONSTRUCTORS | สร้างประวัติแต้มพร้อมบันทึกเวลาปัจจุบัน |
| `recordPointTransaction(...)` | STATE RECORDING | บันทึกข้อมูลแต้มพร้อมเงื่อนไขป้องกันการบันทึกซ้ำ |
| `destroy()` | LIFECYCLE | ทำลายประวัติและเคลียร์การอ้างอิง payment = null |
| `time(), typeText(), points(), balance(), payment()` | ACCESSORS | อ่านข้อมูลประวัติแต้มเพื่อแสดงผล |

> **จุดสังเกตการออกแบบเชิงวัตถุ (OOP Design Insight):**
> PointTransaction ช่วยให้สมาชิกสามารถดู Statement แต้มย้อนหลังได้ว่า ได้รับแต้มจากบิลใบเสร็จใบไหน และเคยแลกส่วนลดไปเมื่อใด

**คำถาม-คำตอบยอดฮิตสำหรับการนำเสนอ (Defense Q&A):**

ทำไม PointTransaction ไม่ถูกเก็บรวมเป็นก้อนเดียวในระดับร้าน? -> เพราะตามหลัก OOAD แต้มเป็นสมบัติส่วนตัวของ Member แต่ละคน การเก็บไว้ใน Member ทำให้เกิด Encapsulation และ Low Coupling

## ซอร์สโค้ดฉบับสมบูรณ์ (Full Source Code: PointTransaction.java)

src/model/PointTransaction.java — Java Source Code

```java
package model;

import java.time.LocalDateTime;

// เก็บประวัติแต้มหนึ่งครั้ง พร้อมใบชำระเงินที่เกี่ยวข้อง (ถ้ามี)
public class PointTransaction {
    public enum PointType { EARN, REDEEM }
    // หนึ่ง object คือแต้มเพิ่มหรือแต้มลดหนึ่งครั้ง พร้อมยอดหลังทำรายการ
    private String pointTransactionId;
    private LocalDateTime date;
    private PointType type;
    private int points;
    private int balance;
    // has-a (Association): ประวัติแต้มอาจอ้างถึง Payment แต่ไม่ได้เป็นเจ้าของใบชำระ
    private Payment payment;
    private boolean destroyed;

    // สร้างประวัติแต้มหนึ่งรายการพร้อมยอดคงเหลือหลังรายการ
    public PointTransaction(String pointTransactionId, PointType type, int points, int balance) {
        this.pointTransactionId = pointTransactionId;
        this.date = LocalDateTime.now();
        this.type = type;
        this.points = points;
        this.balance = balance;
    }

    // สร้างประวัติแต้มหนึ่งรายการพร้อมยอดคงเหลือหลังรายการ
    public PointTransaction(String pointTransactionId, String type, int points, int balance, Payment payment) {
        this(pointTransactionId, type.equals("สะสม") ? PointType.EARN : PointType.REDEEM, points, balance);
        this.payment = payment;
    }

    // บันทึกชนิด จำนวน และใบชำระเงินที่เกี่ยวข้อง
    public void recordPointTransaction(PointType type, int points, int balance, Payment payment) {
        if (this.payment != null) throw new IllegalStateException("บันทึกรายการแต้มแล้ว");
        this.type = type;
        this.points = points;
        this.balance = balance;
        this.payment = payment;
        this.date = LocalDateTime.now();
    }

    public void destroy() {
        destroyed = true;
        payment = null;
    }

    // เมธอดอ่านประวัติแต้ม; typeText() แปลง enum เป็นคำภาษาไทย

    public LocalDateTime time() {
        return date;
    }

    public String typeText() {
        return type == PointType.EARN ? "สะสม" : "แลก";
    }

    public int points() {
        return points;
    }

    public int balance() {
        return balance;
    }

    public Payment payment() {
        return payment;
    }
```

```
69
70      public boolean isDestroyed() {
71          return destroyed;
72      }
73  }
74
```

## 2.8 คลาส Ingredient (Ingredient.java)

โดเมนคลังวัตถุดิบและการจัดซื้อ (INVENTORY & PURCHASING)

**บทบาทหน้าที่ (Responsibility):** ทำหน้าที่เป็นตัวแทนของวัตถุดิบแต่ละชนิดในคลังสินค้า ควบคุมยอดคงเหลือ (stockQty), ยอดจุดสั่งซื้อขั้นต่ำ (minQty), การตัดสต็อกเมื่อขาย, การคืนสต็อกเมื่อยกเลิก, การรับเข้าจากใบสั่งซื้อ, และการปรับยอด พร้อมทั้งเป็นเจ้าของประวัติ StockMovement

**ความสัมพันธ์ UML:** `Ingredient ◆--(*)--> StockMovement`

### ตารางฟิลด์และตัวแปรข้อมูล (Fields & Attributes)

| ชนิดข้อมูลและการเข้าถึง | ชื่อตัวแปร (Field) | คำอธิบายหน้าที่ในระบบ |
|---|---|---|
| `private String` | **ingredientId** | รหัสวัตถุดิบ เช่น 'ING-1' |
| `private String` | **name** | ชื่อวัตถุดิบ เช่น 'เนื้อปลา', 'ไข่ไก่', 'หมูยอ' |
| `private String` | **unit** | หน่วยนับ เช่น 'กก.', 'ฟอง', 'แท่ง' |
| `private double` | **stockQty** | ยอดสต็อกคงเหลือปัจจุบัน |
| `private double` | **minQty** | ยอดขั้นต่ำเตือนสั่งซื้อ (Safety Stock / Reorder Point) |
| `private List<StockMovement>` | **movements** | รายการประวัติการเคลื่อนไหวสต็อก (Composition) |
| `private boolean` | **destroyed** | แฟล็กสถานะการทำลาย |

### ตารางเมธอดสำคัญ (Key Methods & Operations)

| รูปแบบเมธอด (Signature) | หมวดหมู่การทำงาน | คำอธิบายรายละเอียดการทำงาน |
|---|---|---|
| `Ingredient(String id, String name, String unit, double minQty)` | CONSTRUCTOR | สร้างวัตถุดิบ ยอดเริ่มต้นเป็น 0 และเตรียม List สำหรับ movements |
| `hasEnough(double qty) : boolean` | CHECK / INVARIANT | ตรวจสอบว่าสต็อกมีเพียงพอต่อจำนวนที่ต้องการตัดหรือไม่ (มี delta 0.000001 ป้องกันปัญหา floating point) |
| `changeStock(double change, String type, ...)` | CORE HELPER (PRIVATE) | ปรับปรุงยอด stockQty และสร้างอ็อบเจกต์ StockMovement เพิ่มลงใน List ทันที |
| `receive(double qty, String ref, Employee emp)` | BUSINESS LOGIC | รับวัตถุดิบเข้าคลัง เพิ่มยอดและบันทึกประวัติ 'รับเข้า' |
| `deduct(double qty, String ref)` | BUSINESS LOGIC | ตัดสต็อกวัตถุดิบ ตรวจสอบ hasEnough() ก่อนตัดจริง หากไม่พอจะโยน IllegalArgumentException |
| `restore(double qty, String ref)` | BUSINESS LOGIC | คืนสต็อกวัตถุดิบเมื่อมีการยกเลิกอาหารหรือลดจำนวนจาน |
| `adjustStock(double newQty, String reason, Employee by)` | BUSINESS LOGIC | ปรับยอดสต็อกให้ตรงกับการตรวจนับจริง บันทึกเหตุผลและผู้รับผิดชอบ |
| `destroy()` | LIFECYCLE (COMPOSITION) | วนลูปสั่ง movement.destroy() ทุกตัว ล้าง movements.clear() และตั้ง destroyed = true |

**จุดสังเกตการออกแบบเชิงวัตถุ (OOP Design Insight):**
การที่ Ingredient จัดการตัดสต็อกและบันทึก Movement ภายในตัวเอง ทำให้ Main หรือ Order ไม่จำเป็นต้องรู้สูตรคณิตศาสตร์หรือวิธีสร้าง Movement เลย เป็นไปตามหลัก Information Expert ใน GRASP Pattern

**คำถาม-คำตอบยอดฮิตสำหรับการนำเสนอ (Defense Q&A):**
ทำไมยอดเริ่มต้นของ Ingredient ถึงเป็น 0? -> เพราะในทางบัญชีสต็อก วัตถุดิบต้องมียอดเริ่มต้นจากการ 'รับเข้า' อย่างเป็นทางการ (ผ่านใบสั่งซื้อหรือยอดยกมา) เพื่อให้มีหลักฐาน Movement บันทึกไว้ตั้งแต่กิโลกรัมแรก

## ซอร์สโค้ดฉบับสมบูรณ์ (Full Source Code: Ingredient.java)

src/model/Ingredient.java — Java Source Code

```java
package model;

import java.util.ArrayList;
import java.util.List;

// วัตถุดิบรับผิดชอบยอดสต็อกของตัวเอง และเป็นเจ้าของประวัติ StockMovement
public class Ingredient {
    // stockQty คือยอดปัจจุบัน ส่วน movements เก็บเหตุการณ์ทุกครั้งที่ยอดเปลี่ยน
    private String ingredientId;
    private String name;
    private String unit;
    private double stockQty;
    private double minQty;
    // has-a (Composition): Ingredient เป็นเจ้าของ StockMovement ของตนเอง
    // เมื่อลบวัตถุดิบ ประวัติที่เป็นส่วนของมันถูกทำลายตาม
    private List<StockMovement> movements;
    private boolean destroyed;

    // สร้างวัตถุดิบโดยยอดสต็อกเริ่มที่ศูนย์และกำหนดยอดขั้นต่ำ
    public Ingredient(String ingredientId, String name, String unit, double minQty) {
        if (!Double.isFinite(minQty) || minQty < 0)
            throw new IllegalArgumentException("ขั้นต่ำติดลบไม่ได้");
        this.ingredientId = ingredientId;
        this.name = name;
        this.unit = unit;
        this.minQty = minQty;
        this.movements = new ArrayList<StockMovement>();
    }

    // ตรวจว่าสต็อกมีพอต่อจำนวนที่ต้องใช้หรือไม่
    public boolean hasEnough(double qty) {
        return stockQty + 0.000001 >= qty;
    }

    // รับของ ตัดขาย และคืนของ ใช้เมธอดเดียวกันบันทึกยอดกับประวัติ
    private void changeStock(double change, String type, String reference,
            String reason, Employee employee) {
        if (destroyed)
            throw new IllegalStateException("วัตถุดิบนี้ถูกลบแล้ว");
        stockQty = Math.round((stockQty + change) * 1000.0) / 1000.0;
        String id = ingredientId + "-MOV-" + (movements.size() + 1);
        StockMovement movement = new StockMovement(id, type, change, stockQty,
                reference, reason, employee);
        movements.add(movement);
    }

    // เพิ่มยอดรับเข้าคลังและสร้างประวัติการเคลื่อนไหว
    public void receive(double quantity, String reference, Employee employee) {
        if (!Double.isFinite(quantity) || quantity < 0)
            throw new IllegalArgumentException("จำนวนรับติดลบไม่ได้");
        changeStock(quantity, "รับเข้า", reference, "รับวัตถุดิบ", employee);
    }

    // ชื่อ operation ตาม P7; งานรับ PO ที่ระบุพนักงานใช้ receive(..., employee)
    public void increaseStock(double qty, String ref) {
        receive(qty, ref, null);
    }

    // ตรวจยอดก่อนตัดตามออเดอร์ แล้วสร้างประวัติการขาย
    public void deduct(double qty, String ref) {
        if (!Double.isFinite(qty) || qty < 0 || !hasEnough(qty)) {
            throw new IllegalArgumentException("วัตถุดิบไม่พอ: " + name);
        }
        changeStock(-qty, "ขาย", ref, "ตัดตามสูตร", null);
    }

    // คืนสต็อกเมื่อยกเลิกหรือแก้จำนวนอาหาร
    public void restore(double qty, String ref) {
```

```java
        if (!Double.isFinite(qty) || qty < 0)
            throw new IllegalArgumentException("จำนวนคืนติดลบไม่ได้");
        changeStock(qty, "คืน", ref, "ยกเลิกหรือแก้จำนวนอาหาร", null);
    }

    // ปรับยอดให้ตรงกับการตรวจนับ โดยต้องบอกเหตุผล
    public void adjustStock(double newQty, String reason, Employee by) {
        if (!Double.isFinite(newQty) || newQty < 0 || reason.trim().isEmpty()) {
            throw new IllegalArgumentException("ยอดต้องไม่ติดลบและต้องมีเหตุผล");
        }
        changeStock(newQty - stockQty, "ปรับยอด", "ตรวจนับ", reason, by);
    }

    // แก้ชื่อ หน่วย และยอดขั้นต่ำของวัตถุดิบ
    public void update(String name, String unit, double minQty) {
        if (!Double.isFinite(minQty) || minQty < 0)
            throw new IllegalArgumentException("ขั้นต่ำติดลบไม่ได้");
        this.name = name;
        this.unit = unit;
        this.minQty = minQty;
    }

    // Composition: เมื่อลบวัตถุดิบ ประวัติที่เป็นส่วนของวัตถุดิบนี้ถูกทำลายตาม
    public void destroy() {
        for (int i = 0; i < movements.size(); i++)
            movements.get(i).destroy();
        movements.clear();
        destroyed = true;
    }

    // เมธอดอ่านข้อมูลสต็อกและประวัติ โดยไม่แก้ยอดจากภายนอกโดยตรง
    public String ingredientId() {
        return ingredientId;
    }

    public String name() {
        return name;
    }

    public String unit() {
        return unit;
    }

    public double stockQty() {
        return stockQty;
    }

    public double minQty() {
        return minQty;
    }

    public List<StockMovement> movements() {
        return movements;
    }

    public boolean isDestroyed() {
        return destroyed;
    }
}

```

## 2.9 คลาส Recipe (Recipe.java)

โดเมนเมนู คู่มือ และสูตรอาหาร (MENU, RECIPE & SOP)

**บทบาทหน้าที่ (Responsibility):** ทำหน้าที่เก็บสูตรอาหารหนึ่งเวอร์ชัน โดยระบุว่าเมนูนี้ใช้วัตถุดิบอะไรบ้างและปริมาณเท่าใดต่อ 1 จาน มีสถานะเป็นฉบับร่าง (DRAFT) หรือฉบับสมบูรณ์ (COMPLETE) และรองรับการแตกเวอร์ชันใหม่ (Versioning)

**ความสัมพันธ์ UML:** `Recipe ◆<--(Composition)-- Menu, Recipe --(*)--> Ingredient`

### ตารางฟิลด์และตัวแปรข้อมูล (Fields & Attributes)

| ชนิดข้อมูลและการเข้าถึง | ชื่อตัวแปร (Field) | คำอธิบายหน้าที่ในระบบ |
|---|---|---|
| `public enum { DRAFT, COMPLETE }` | **RecipeStatus** | สถานะของสูตร: DRAFT (กำลังแก้ไข) หรือ COMPLETE (เสร็จสมบูรณ์พร้อมใช้) |
| `private String` | **recipeId** | รหัสสูตร เช่น 'RCP-1-v1' |
| `private int` | **version** | หมายเลขเวอร์ชันของสูตร เช่น 1, 2 |
| `private RecipeStatus` | **status** | สถานะสูตรปัจจุบัน |
| `private List<Ingredient>` | **ingredients** | รายการวัตถุดิบที่ใช้ (Association ชี้ไปที่ Ingredient ในคลัง) |
| `private List<Double>` | **quantities** | รายการปริมาณที่ใช้ต่อ 1 จาน (จับคู่ตาม index เดียวกับ ingredients) |
| `private boolean` | **destroyed** | แฟล็กสถานะการทำลาย |

### ตารางเมธอดสำคัญ (Key Methods & Operations)

| รูปแบบเมธอด (Signature) | หมวดหมู่การทำงาน | คำอธิบายรายละเอียดการทำงาน |
|---|---|---|
| `Recipe(String recipeId, int version)` | CONSTRUCTOR | สร้างสูตรฉบับร่าง (DRAFT) เวอร์ชันที่กำหนด และเตรียม List ทั้งสองชุด |
| `addItem(Ingredient ing, double qty)` | MUTATOR / VALIDATION | เพิ่มวัตถุดิบและปริมาณ ทำได้เฉพาะสถานะ DRAFT หากมีวัตถุดิบเดิมอยู่แล้วจะบวกปริมาณเพิ่ม |
| `changeQty(Ingredient ing, double qty)` | MUTATOR | เปลี่ยนปริมาณวัตถุดิบ ทำได้เฉพาะสูตรฉบับร่าง DRAFT เท่านั้น |
| `complete()` | LIFECYCLE STATE TRANSITION | ยืนยันความสมบูรณ์ของสูตร เปลี่ยนสถานะเป็น COMPLETE โดยตรวจว่าต้องมีวัตถุดิบอย่างน้อย 1 ชนิด |
| `createNewVersion() : Recipe` | VERSIONING | สร้างสูตรฉบับร่างเวอร์ชันถัดไป (v+1) โดยคัดลอกส่วนผสมเดิมทั้งหมดมาให้ เพื่อให้เชฟปรับปรุงสูตรใหม่โดยไม่กระทบสูตรเก่า |
| `destroy()` | LIFECYCLE | เคลียร์รายการวัตถุดิบและปริมาณทิ้งทั้งหมด |

> **จุดสังเกตการออกแบบเชิงวัตถุ (OOP Design Insight):**
> สูตรที่ COMPLETE แล้วจะถูกล็อกไม่ให้แก้ไขตรง ๆ (Immutability Pattern) หากต้องการปรับสูตร ต้องเรียก createNewVersion() เพื่อสร้างเวอร์ชันใหม่เท่านั้น เพื่อป้องกันการกระทบยอดอาหารที่อยู่ในครัว

**คำถาม-คำตอบยอดฮิตสำหรับการนำเสนอ (Defense Q&A):**

ทำไมใช้ List สองชุด (ingredients และ quantities) แทนที่จะใช้ Map? -> เพื่อให้การเรียนรู้ OOP ในระดับพื้นฐานเข้าใจง่าย ลำดับของ List ทั้งสองตรงกัน และสามารถอธิบายผ่าน index เดียวกันได้ชัดเจน

## ซอร์สโค้ดฉบับสมบูรณ์ (Full Source Code: Recipe.java)

src/model/Recipe.java — Java Source Code

```java
package model;

import java.util.ArrayList;
import java.util.List;

// สูตรหนึ่งเวอร์ชัน: วัตถุดิบหนึ่งชนิดจับคู่กับปริมาณต่อหนึ่งจาน
public class Recipe {
    public enum RecipeStatus { DRAFT, COMPLETE }

    // ingredients และ quantities จับคู่กันด้วย index เดียวกัน
    private String recipeId;
    private int version;
    private RecipeStatus status;
    // has-a (Association): Recipe อ้างถึง Ingredient ในคลัง แต่ไม่ได้เป็นเจ้าของวัตถุดิบ
    private List<Ingredient> ingredients;
    private List<Double> quantities;
    private boolean destroyed;

    // สร้างสูตรฉบับร่างหนึ่งเวอร์ชัน
    public Recipe(String recipeId, int version) {
        this.recipeId = recipeId;
        this.version = version;
        this.status = RecipeStatus.DRAFT;
        this.ingredients = new ArrayList<Ingredient>();
        this.quantities = new ArrayList<Double>();
    }

    // เพิ่มวัตถุดิบกับปริมาณที่ตำแหน่งเดียวกัน ถ้ามีอยู่แล้วให้บวกปริมาณ
    public void addItem(Ingredient ingredient, double qty) {
        if (status != RecipeStatus.DRAFT || destroyed)
            throw new IllegalStateException("แก้ได้เฉพาะสูตรร่างที่ยังไม่ลบ");
        if (ingredient == null || !Double.isFinite(qty) || qty <= 0)
            throw new IllegalArgumentException("วัตถุดิบและปริมาณไม่ถูกต้อง");
        int index = ingredients.indexOf(ingredient);
        if (index >= 0)
            quantities.set(index, quantities.get(index) + qty);
        else {
            ingredients.add(ingredient);
            quantities.add(qty);
        }
    }

    // ชื่อเดิมยังใช้ใน Main และแบบทดสอบ
    public void addIngredient(Ingredient ingredient, double quantity) {
        addItem(ingredient, quantity);
    }

    // เปลี่ยนปริมาณได้เฉพาะสูตรร่าง
    public void changeQty(Ingredient ingredient, double qty) {
        if (status != RecipeStatus.DRAFT)
            throw new IllegalStateException("สร้างเวอร์ชันใหม่ก่อนแก้สูตรสมบูรณ์");
        int index = ingredients.indexOf(ingredient);
        if (index < 0 || !Double.isFinite(qty) || qty <= 0)
            throw new IllegalArgumentException("วัตถุดิบหรือปริมาณไม่ถูกต้อง");
        quantities.set(index, qty);
    }

    // แปลงตำแหน่งในหน้าจอเป็นวัตถุดิบแล้วเปลี่ยนปริมาณ
    public void changeQuantity(int index, double quantity) {
        changeQty(ingredients().get(index), quantity);
    }

    // ทำสูตรให้สมบูรณ์เมื่อมีวัตถุดิบอย่างน้อยหนึ่งชนิด
    public void complete() {
        if (ingredients.isEmpty())
            throw new IllegalStateException("สูตรต้องมีวัตถุดิบ");
        status = RecipeStatus.COMPLETE;
    }
```

```
69
70      // สูตรใหม่มีรายการของตัวเอง แต่ยังอ้างวัตถุดิบในคลังตัวเดียวกัน
71      public Recipe createNewVersion(String id) {
72          Recipe next = new Recipe(id, version + 1);
73          for (int i = 0; i < ingredients.size(); i++)
74              next.addItem(ingredients.get(i), quantities.get(i));
75          return next;
76      }
77
78      // คัดลอกส่วนผสมไปเป็นสูตรร่างเวอร์ชันถัดไป
79      public Recipe createNewVersion() {
80          return createNewVersion(recipeId + "-v" + (version + 1));
81      }
82
83      public void destroy() {
84          ingredients.clear();
85          quantities.clear();
86          destroyed = true;
87      }
88
89      // เมธอดอ่านสูตร; รายการที่คืนออกไปเป็นสำเนาเพื่อกันการแก้จากภายนอก
90
91      public int version() { return version; }
92      public String statusText() { return status == RecipeStatus.DRAFT ? "ร่าง" : "สมบูรณ์"; }
93      public ArrayList<Ingredient> ingredients() { return new ArrayList<Ingredient>(ingredients); }
94      public double quantity(int index) { return quantities.get(index); }
95      public boolean isDestroyed() { return destroyed; }
96  }
97
```

## 2.10 คลาส SOP (SOP.java)

โดเมนเมนู คู่มือ และสูตรอาหาร (MENU, RECIPE & SOP)

**บทบาทหน้าที่ (Responsibility):** ทำหน้าที่เป็นคู่มือการปฏิบัติงานมาตรฐาน (Standard Operating Procedure) ของเมนูอาหารฉบับหนึ่ง ควบคุมขั้นตอนการปรุงตามลำดับ คำนวณเวลารวมมาตรฐาน และบริหารวงจรชีวิตของคู่มือ (DRAFT, PUBLISHED, DISABLED)

**ความสัมพันธ์ UML:** `SOP ◆--(*)--> SOPStep, SOP --(1)--> Menu`

### ตารางฟิลด์และตัวแปรข้อมูล (Fields & Attributes)

| ชนิดข้อมูลและการเข้าถึง | ชื่อตัวแปร (Field) | คำอธิบายหน้าที่ในระบบ |
|---|---|---|
| `public enum { DRAFT, PUBLISHED, DISABLED }` | **SopStatus** | สถานะคู่มือ: DRAFT (ร่าง), PUBLISHED (เผยแพร่ใช้งาน), DISABLED (ปิดใช้งานชั่วคราว) |
| `private String` | **sopId** | รหัสคู่มือ เช่น 'SOP-1' |
| `private String` | **name** | ชื่อคู่มือ เช่น 'SOP ผัดกะเพราปลา' |
| `private SopStatus` | **status** | สถานะของคู่มือ |
| `private Menu` | **menu** | เมนูอาหารที่คู่มือฉบับนี้ใช้ควบคุม (Association) |
| `private List<SOPStep>` | **steps** | รายการขั้นตอนการปฏิบัติงาน (Composition) |
| `private boolean` | **destroyed** | แฟล็กสถานะการทำลาย |

### ตารางเมธอดสำคัญ (Key Methods & Operations)

| รูปแบบเมธอด (Signature) | หมวดหมู่การทำงาน | คำอธิบายรายละเอียดการทำงาน |
|---|---|---|
| `SOP(String sopId, String name, Menu menu)` | CONSTRUCTOR | สร้างคู่มือฉบับร่าง (DRAFT) ผูกกับ Menu |
| `addStep(String description, double minutes)` | COMPOSITION CREATOR | สร้างอ็อบเจกต์ SOPStep ใหม่ กำหนดเลขลำดับอัตโนมัติ และเก็บเข้า List steps |
| `getTotalMinutes() : double` | AGGREGATION / CALCULATION | วนลูปคำนวณผลรวมเวลามาตรฐานของทุกขั้นตอน เพื่อใช้เป็นเป้าหมายเวลาในครัว |
| `publish()` | STATE TRANSITION | เผยแพร่คู่มือเพื่อให้คนครัวนำไปใช้ได้ ต้องเป็นฉบับร่างและมีขั้นตอนอย่างน้อย 1 ขั้น |
| `disable() / enable()` | STATE MUTATORS | ปิดใช้ หรือเปิดใช้คู่มือที่เคยเผยแพร่แล้ว |
| `isDeletable() : boolean` | BUSINESS RULE | ลบคู่มือได้เฉพาะเมื่อยังคงเป็นฉบับร่าง (DRAFT) เท่านั้น |
| `destroy()` | LIFECYCLE (COMPOSITION) | สั่ง destroy() ขั้นตอนย่อยทุกตัว ล้าง List และตัดการเชื่อมโยง menu = null |

**จุดสังเกตการออกแบบเชิงวัตถุ (OOP Design Insight):**

SOP ทำให้ครัวมีมาตรฐาน เวลาปรุงจริงใน OrderItem จะถูกนำมาเปรียบเทียบกับ getTotalMinutes() ของ SOP เพื่อประเมินประสิทธิภาพของคนครัว

**คำถาม-คำตอบยอดฮิตสำหรับการนำเสนอ (Defense Q&A):**

ทำไมลบ SOP แล้วไม่ลบ Menu ตามไปด้วย? -> เพราะ SOP เป็นเพียงคู่มือที่กำกับเมนู (Association) เมนูอาหารยังคงมีอยู่และเปิดขายได้ตามปกติแม้คู่มือฉบับนั้นจะถูกลบ

## ซอร์สโค้ดฉบับสมบูรณ์ (Full Source Code: SOP.java)

**src/model/SOP.java** — Java Source Code

```java
package model;

import java.util.ArrayList;
import java.util.List;

// คู่มือหนึ่งฉบับผูกกับหนึ่งเมนู และเป็นเจ้าของขั้นตอนย่อย
public class SOP {
    public enum SopStatus { DRAFT, PUBLISHED, DISABLED }

    // SOP ผูกกับ Menu หนึ่งรายการ และเก็บขั้นตอนตามลำดับ
    private String sopId;
    private String name;
    private SopStatus status;
    // has-a (Association): SOP ใช้ Menu เดิมของร้าน ไม่ได้ทำลายเมนูเมื่อคู่มือถูกลบ
    private Menu menu;
    // has-a (Composition): SOP เป็นเจ้าของ SOPStep; ลบ SOP แล้วลบขั้นตอนด้วย
    private List<SOPStep> steps;
    private boolean destroyed;

    // สร้างคู่มือฉบับร่างที่ผูกกับเมนูหนึ่งรายการ
    public SOP(String sopId, String name, Menu menu) {
        this.sopId = sopId;
        this.name = name;
        this.menu = menu;
        this.status = SopStatus.DRAFT;
        this.steps = new ArrayList<SOPStep>();
    }

    // เพิ่มขั้นตอนพร้อมลำดับและเวลามาตรฐาน
    public void addStep(String description, double minutes) {
        int number = steps.size() + 1;
        SOPStep step = new SOPStep(sopId + "-STP-" + number, number, description, minutes);
        steps.add(step);
    }

    // รวมเวลาของทุกขั้นตอนเพื่อใช้เทียบเวลาปรุง
    public double getTotalMinutes() {
        double total = 0;
        for (int i = 0; i < steps.size(); i++)
            total += steps.get(i).standardMinutes();
        return total;
    }

    // เผยแพร่ได้เมื่อฉบับร่างมีขั้นตอนแล้ว
    public void publish() {
        if (status != SopStatus.DRAFT || steps.isEmpty()) {
            throw new IllegalStateException("เผยแพร่ได้เฉพาะฉบับร่างที่มีขั้นตอน");
        }
        status = SopStatus.PUBLISHED;
    }

    // ปิดใช้คู่มือที่เผยแพร่แล้ว
    public void disable() {
        if (status != SopStatus.PUBLISHED)
            throw new IllegalStateException("ต้องเผยแพร่ก่อน");
        status = SopStatus.DISABLED;
    }

    // เปิดใช้คู่มือที่ถูกปิด
    public void enable() {
        if (status != SopStatus.DISABLED)
            throw new IllegalStateException("ต้องปิดใช้อยู่ก่อน");
        status = SopStatus.PUBLISHED;
    }

    // อนุญาตให้ลบได้เฉพาะคู่มือฉบับร่าง
    public boolean isDeletable() {
        return status == SopStatus.DRAFT;
```

```
    }

    public void destroy() {
        for (int i = 0; i < steps.size(); i++)
            steps.get(i).destroy();
        steps.clear();
        menu = null;
        destroyed = true;
    }

    // เมธอดอ่านข้อมูลคู่มือและขั้นตอนสำหรับแสดงผล

    public String name() {
        return name;
    }

    // แก้ชื่อคู่มือ
    public void update(String name) {
        this.name = name;
    }

    public String statusText() {
        if (status == SopStatus.DRAFT) return "ร่าง";
        if (status == SopStatus.PUBLISHED) return "เผยแพร่";
        return "ปิดใช้";
    }

    public Menu menu() {
        return menu;
    }

    public List<SOPStep> steps() {
        return steps;
    }

    public boolean isDestroyed() {
        return destroyed;
    }
}

```

## 2.11 คลาส Menu (Menu.java)

โดเมนเมนู คู่มือ และสูตรอาหาร (MENU, RECIPE & SOP)

**บทบาทหน้าที่ (Responsibility):** ทำหน้าที่เป็นตัวแทนของเมนูอาหารในร้าน จัดการราคาขาย หมวดหมู่อาหาร สถานีครัวที่ปรุง สถานะการเปิด/ปิดขาย (ON_SALE/OFF_SALE) ตัวเลือกเสริม (Options เช่น เพิ่มไข่ดาว) และเป็นเจ้าของสูตร Recipe ทุกเวอร์ชัน

**ความสัมพันธ์ UML:** `Menu ◆--(*)--> Recipe`

### ตารางฟิลด์และตัวแปรข้อมูล (Fields & Attributes)

| ชนิดข้อมูลและการเข้าถึง | ชื่อตัวแปร (Field) | คำอธิบายหน้าที่ในระบบ |
|---|---|---|
| `public enum { ON_SALE, OFF_SALE }` | **SaleStatus** | สถานะการขาย: ON_SALE (เปิดขาย) หรือ OFF_SALE (ปิดขาย) |
| `private String` | **menuId** | รหัสเมนู เช่น 'M01', 'M02' |
| `private String` | **name** | ชื่อเมนูอาหาร เช่น 'ผัดกะเพราปลา', 'ยำหมูยอ/แหนม' |
| `private double` | **price** | ราคาขายพื้นฐานต่อจาน (บาท) |
| `private String` | **category** | หมวดหมู่อาหาร เช่น 'จานเดียว', 'กับข้าว', 'เครื่องดื่ม' |
| `private String` | **station** | สถานีครัวที่รับผิดชอบ เช่น 'ครัวร้อน', 'ครัวยำ' |
| `private SaleStatus` | **status** | สถานะการขายปัจจุบัน |
| `private boolean` | **spiceSelectable** | สามารถเลือกระดับความเผ็ดได้หรือไม่ |
| `private boolean` | **everOrdered** | เคยมีลูกค้าสั่งเมนูนี้ไปแล้วหรือไม่ (ใช้ล็อกห้ามลบ) |
| `private List<Recipe>` | **recipes** | รายการสูตรอาหารทุกเวอร์ชันของเมนูนี้ (Composition) |
| `private List<String>, List<Double>` | **options, optionPrices** | ชื่อตัวเลือกเสริมและราคาเพิ่มตามลำดับ index |
| `private Lists` | **extraOptionIndexes, extraIngredients, extraQuantities** | การผูกตัวเลือกเสริมเข้ากับวัตถุดิบและปริมาณที่ต้องใช้เพิ่ม |

### ตารางเมธอดสำคัญ (Key Methods & Operations)

| รูปแบบเมธอด (Signature) | หมวดหมู่การทำงาน | คำอธิบายรายละเอียดการทำงาน |
|---|---|---|
| `Menu(...)` | CONSTRUCTORS | สร้างเมนูอาหาร โดยตั้งสถานะเริ่มต้นเป็นปิดขาย (OFF_SALE) จนกว่าจะมีสูตรสมบูรณ์ |
| `addRecipe(Recipe recipe)` | COMPOSITION LINKER | ผูกสูตรอาหารเข้ากับเมนู |
| `currentRecipe() : Recipe` | BUSINESS LOGIC | เลือกสูตรที่มีสถานะ 'สมบูรณ์' (COMPLETE) และมีหมายเลขเวอร์ชันสูงสุดเพื่อนำไปใช้ปรุง |
| `openSale()` | STATE TRANSITION | เปิดขายเมนู มีเงื่อนไขเข้มงวด: ต้องมีสูตรที่สมบูรณ์แล้วอย่างน้อย 1 สูตร และเมนูยังไม่ถูกลบ |
| `closeSale()` | STATE MUTATOR | ปิดขายเมนูชั่วคราว เช่น เมื่อวัตถุดิบหมด |
| `addOption(String name, double price)` | OPTION MANAGEMENT | เพิ่มตัวเลือกเสริม เช่น 'เพิ่มไข่ดาว' ราคา 15 บาท |
| `addOptionIngredient(int optIdx, Ingredient ing, double qty)` | OPTION INVENTORY | ผูกว่าตัวเลือกเสริมนั้นต้องตัดวัตถุดิบใดเพิ่ม (เช่น ไข่ไก่ 1 ฟอง) |
| `isDeletable() : boolean` | BUSINESS RULE | ตรวจสอบว่าสามารถลบเมนูได้หรือไม่ (ถ้า everOrdered == true จะห้ามลบ) |
| `destroy()` | LIFECYCLE (COMPOSITION) | สั่ง destroy() สูตรทุกเวอร์ชัน เคลียร์ตัวเลือกเสริม และตั้ง destroyed = true |

**จุดสังเกตการออกแบบเชิงวัตถุ (OOP Design Insight):**
การไม่ยอมให้เปิดขาย (openSale) จนกว่าจะมีสูตรสมบูรณ์ เป็น Business Invariant ที่ยอดเยี่ยม ป้องกันไม่ให้ออเดอร์หลุดเข้าไปในระบบโดยที่ครัวไม่รู้วิธีทำและไม่รู้ว่าจะตัดสต็อกวัตถุดิบใด

**คำถาม-คำตอบยอดฮิตสำหรับการนำเสนอ (Defense Q&A):**
ทำไมเก็บตัวเลือกเสริม (Options) ไว้ใน Menu เลย ไม่สร้างคลาส Option แยก? -> เพื่อควบคุมจำนวนคลาสให้คงไว้ที่ 16 คลาสโมเดลหลักตามข้อกำหนดสถาปัตยกรรม โดยเก็บเป็นข้อมูลโครงสร้างภายใน

## ซอร์สโค้ดฉบับสมบูรณ์ (Full Source Code: Menu.java)

src/model/Menu.java — Java Source Code

```java
package model;

import java.util.ArrayList;
import java.util.List;

// เมนูเป็นเจ้าของสูตรหลายเวอร์ชัน ตัวเลือกเสริมเก็บเป็นข้อมูลภายใน ไม่สร้างคลาสที่ 17
public class Menu {
    public enum SaleStatus { ON_SALE, OFF_SALE }

    // เมนูหนึ่งรายการมีสูตรได้หลายเวอร์ชัน และตัวเลือกมีราคาเพิ่มของตัวเอง
    private String menuId;
    private String name;
    private double price;
    private String category;
    private String station;
    private SaleStatus status;
    private boolean spiceSelectable;
    private boolean everOrdered;
    private boolean destroyed;
    // has-a (Composition): Menu เป็นเจ้าของ Recipe; เมื่อลบ Menu จะลบสูตรด้วย
    private List<Recipe> recipes;
    // List สองชุดนี้จับคู่ด้วย index เดียวกัน เช่น ชื่อที่ 0 มีราคาที่ 0
    private List<String> options;
    private List<Double> optionPrices;
    // สามรายการนี้จับคู่กัน: ตัวเลือกเลขใด ใช้วัตถุดิบอะไร ปริมาณเท่าไร
    private ArrayList<Integer> extraOptionIndexes;
    private ArrayList<Ingredient> extraIngredients;
    private ArrayList<Double> extraQuantities;

    // สร้างเมนูใหม่ โดยเริ่มปิดขายจนกว่าจะมีสูตรสมบูรณ์
    public Menu(String menuId, String name, double price) {
        this(menuId, name, price, "ยังไม่ระบุ", "ครัวร้อน");
    }

    // สร้างเมนูใหม่ โดยเริ่มปิดขายจนกว่าจะมีสูตรสมบูรณ์
    public Menu(String id, String name, double price, String category, String station) {
        this.menuId = id;
        update(name, price, category, station);
        this.status = SaleStatus.OFF_SALE;
        this.recipes = new ArrayList<Recipe>();
        this.options = new ArrayList<String>();
        this.optionPrices = new ArrayList<Double>();
        this.extraOptionIndexes = new ArrayList<Integer>();
        this.extraIngredients = new ArrayList<Ingredient>();
        this.extraQuantities = new ArrayList<Double>();
    }

    // แก่ชื่อ ราคา หมวด และสถานีครัว โดยราคาต้องมากกว่าศูนย์
    public void update(String name, double price, String category, String station) {
        if (!Double.isFinite(price) || price <= 0)
            throw new IllegalArgumentException("ราคาต้องมากกว่า 0");
        this.name = name;
        this.price = price;
        this.category = category;
        this.station = station;
    }

    // เก็บสูตรอีกหนึ่งเวอร์ชันไว้กับเมนู
    public void addRecipe(Recipe recipe) {
        recipes.add(recipe);
    }

    // เลือกสูตรสมบูรณ์ที่มีเลขเวอร์ชันสูงสุด
    public Recipe currentRecipe() {
        Recipe latest = null;
        for (int i = 0; i < recipes.size(); i++) {
            Recipe recipe = recipes.get(i);
            if (recipe.statusText().equals("สมบูรณ์")) {
```

```
                if (latest == null || recipe.version() > latest.version())
                    latest = recipe;
            }
        }
        return latest;
    }

    // เปิดขายได้เมื่อมีสูตรสมบูรณ์และเมนูยังไม่ถูกลบ
    public void openSale() {
        if (destroyed || currentRecipe() == null) {
            throw new IllegalStateException("ต้องมีสูตรสมบูรณ์และเมนูยังไม่ถูกลบ");
        }
        status = SaleStatus.ON_SALE;
    }

    // ปิดขายเมนูชั่วคราว
    public void closeSale() {
        status = SaleStatus.OFF_SALE;
    }

    // จำว่าเคยยืนยันสั่งเมนูนี้แล้ว เพื่อป้องกันการลบข้อมูลเก่า
    public void noteOrdered() {
        everOrdered = true;
    }

    // ลบเมนูได้เฉพาะเมื่อยังไม่เคยถูกสั่งจริง
    public boolean isDeletable() {
        return !everOrdered;
    }

    // เพิ่มชื่อตัวเลือกและราคาเพิ่ม
    public void addOption(String name, double price) {
        if (!Double.isFinite(price) || price < 0)
            throw new IllegalArgumentException("ราคาตัวเลือกติดลบไม่ได้");
        int index = options.indexOf(name);
        if (index >= 0)
            optionPrices.set(index, price);
        else {
            options.add(name);
            optionPrices.add(price);
        }
    }

    // ผูกตัวเลือกกับวัตถุดิบและปริมาณที่ใช้เพิ่ม
    public void addOptionIngredient(int optionIndex, Ingredient ingredient, double quantity) {
        if (optionIndex < 0 || optionIndex >= options.size() || !Double.isFinite(quantity) || quantity <= 0) {
            throw new IllegalArgumentException("ตัวเลือกหรือปริมาณไม่ถูกต้อง");
        }
        extraOptionIndexes.add(optionIndex);
        extraIngredients.add(ingredient);
        extraQuantities.add(quantity);
    }

    public void destroy() {
        for (int i = 0; i < recipes.size(); i++)
            recipes.get(i).destroy();
        recipes.clear();
        options.clear();
        optionPrices.clear();
        extraOptionIndexes.clear();
        extraIngredients.clear();
        extraQuantities.clear();
        destroyed = true;
    }

    // เมธอดอ่านข้อมูลเมนูและตัวเลือก; สถานะ enum ถูกแปลงเป็นค่าไทย
    public String menuId() {
        return menuId;
    }

    public String name() {
        return name;
    }
```

```
142
143     public double price() {
144         return price;
145     }
146
147     public String category() {
148         return category;
149     }
150
151     public String station() {
152         return station;
153     }
154
155     public String statusText() {
156         return status == SaleStatus.ON_SALE ? "เปิดขาย" : "ปิดขาย";
157     }
158
159     public boolean isSpiceSelectable() {
160         return spiceSelectable;
161     }
162
163     public void allowSpiceSelection(boolean value) {
164         spiceSelectable = value;
165     }
166
167     public ArrayList<Recipe> recipes() {
168         return new ArrayList<Recipe>(recipes);
169     }
170
171     public ArrayList<String> optionNames() {
172         return new ArrayList<String>(options);
173     }
174
175     public double optionPrice(int index) {
176         return optionPrices.get(index);
177     }
178
179     public int extraCount() {
180         return extraIngredients.size();
181     }
182
183     public int extraOptionIndex(int index) {
184         return extraOptionIndexes.get(index);
185     }
186
187     public Ingredient extraIngredient(int index) {
188         return extraIngredients.get(index);
189     }
190
191     public double extraQuantity(int index) {
192         return extraQuantities.get(index);
193     }
194
195     public boolean isDestroyed() {
196         return destroyed;
197     }
198 }
199
```

## 2.12 คลาส PurchaseOrder (PurchaseOrder.java) โดเมนคลังวัตถุดิบและการจัดซื้อ (INVENTORY & PURCHASING)

**บทบาทหน้าที่ (Responsibility):** ทำหน้าที่เป็นใบสั่งซื้อวัตถุดิบจากผู้จัดจำหน่าย (PO) ติดตามสถานะตั้งแต่ฉบับร่าง (DRAFT) สั่งซื้อแล้ว (ORDERED) จนถึงตรวจรับสินค้าเข้าคลังจริง (RECEIVED) พร้อมปรับเพิ่มยอดสต็อกอัตโนมัติ

**ความสัมพันธ์ UML:** `PurchaseOrder --(1)--> Supplier, PurchaseOrder --(*)--> Ingredient`

### ตารางฟิลด์และตัวแปรข้อมูล (Fields & Attributes)

| ชนิดข้อมูลและการเข้าถึง | ชื่อตัวแปร (Field) | คำอธิบายหน้าที่ในระบบ |
|---|---|---|
| `public enum { DRAFT, ORDERED, RECEIVED }` | **PoStatus** | สถานะใบสั่งซื้อ: DRAFT (ร่าง), ORDERED (ส่งคำสั่งซื้อแล้ว), RECEIVED (ตรวจรับเข้าคลังแล้ว) |
| `private String` | **poId** | รหัสใบสั่งซื้อ เช่น 'PO-1' |
| `private PoStatus` | **status** | สถานะใบสั่งซื้อปัจจุบัน |
| `private LocalDateTime` | **createdAt** | วันและเวลาที่ออกใบสั่งซื้อ |
| `private Supplier` | **supplier** | ผู้จัดจำหน่ายที่ออกใบสั่งซื้อให้ (Association) |
| `private Employee` | **createdBy** | พนักงานผู้เปิดใบสั่งซื้อ (Association) |
| `private List<Ingredient>` | **ingredients** | รายการวัตถุดิบที่สั่งซื้อ |
| `private List<Double>` | **orderedQty** | จำนวนที่สั่งซื้อตามลำดับ index |
| `private List<Double>` | **receivedQty** | จำนวนที่ได้รับจริงตามลำดับ index |

### ตารางเมธอดสำคัญ (Key Methods & Operations)

| รูปแบบเมธอด (Signature) | หมวดหมู่การทำงาน | คำอธิบายรายละเอียดการทำงาน |
|---|---|---|
| `PurchaseOrder(String poId, Supplier supplier, Employee by)` | CONSTRUCTOR | สร้างใบสั่งซื้อสถานะ DRAFT พร้อมจดเวลาปัจจุบันและเตรียม List |
| `addItem(Ingredient ing, double qty)` | VALIDATION / MUTATOR | เพิ่มรายการวัตถุดิบที่ต้องการสั่ง โดยตรวจสอบก่อนว่า supplier.supplies(ing) หรือไม่ |
| `markOrdered()` | STATE TRANSITION | เปลี่ยนสถานะจาก DRAFT เป็น ORDERED เมื่อมีรายการสั่งอย่างน้อย 1 รายการ |
| `receive(List<Double> qtys)` | ATOMIC RECEIVING & STOCKING | ตรวจรับสินค้าจริง: ตรวจสอบความถูกต้องของตัวเลขทุกรายการให้ครบก่อน แล้วจึงวนลูปเรียก ing.receive(qty, poId, createdBy) เพื่อเพิ่มสต็อกเข้าคลังและเปลี่ยนสถานะเป็น RECEIVED |
| `poId(), supplier(), statusText(), ingredients(), orderedQty(), receivedQty()` | ACCESSORS | อ่านข้อมูลใบสั่งซื้อเพื่อแสดงผล |

**จุดสังเกตการออกแบบเชิงวัตถุ (OOP Design Insight):**

ในเมธอด receive() มีการใช้รูปแบบ Two-Pass Validation (ตรวจผ่านทุกตัวก่อน ค่อยเริ่มอัปเดตสต็อก) ป้องกันปัญหาข้อมูลสต็อกไม่สอดคล้องกัน (Inconsistent State) หากเกิดข้อผิดพลาดกลางคัน

**คำถาม-คำตอบยอดฮิตสำหรับการนำเสนอ (Defense Q&A):**

ทำไมต้องมีสถานะ ORDERED คั่นกลางระหว่าง DRAFT กับ RECEIVED? -> เพราะในโลกความเป็นจริง เมื่อส่งใบสั่งซื้อไปแล้ว ร้านต้องรอให้ซัพพลายเออร์นำของมาส่ง ซึ่งจำนวนที่ส่งจริงอาจไม่เท่ากับที่สั่ง (Short shipment) จึงต้องมีการกรอก receivedQty ตอนรับจริง

## ซอร์สโค้ดฉบับสมบูรณ์ (Full Source Code: PurchaseOrder.java)

**src/model/PurchaseOrder.java** — Java Source Code

```java
package model;

import java.time.LocalDateTime;
import java.util.ArrayList;
import java.util.List;

// ใบสั่งซื้อเปลี่ยนสถานะจาก ร่าง → สั่งแล้ว → รับแล้ว
public class PurchaseOrder {
    public enum PoStatus { DRAFT, ORDERED, RECEIVED }

    // List ทั้งสามชุดใช้ index เดียวกัน: วัตถุดิบที่ 0 / สั่งที่ 0 / รับที่ 0
    private String poId;
    private PoStatus status;
    private LocalDateTime createdAt;
    // has-a (Association): PO อ้างถึง Ingredient, Supplier และ Employee ในระบบ
    private List<Ingredient> ingredients;
    private List<Double> orderedQty;
    private List<Double> receivedQty;
    private Supplier supplier;
    private Employee createdBy;

    // สร้างใบสั่งซื้อฉบับร่างพร้อมผู้จำหน่ายและผู้สร้าง
    public PurchaseOrder(String poId, Supplier supplier, Employee by) {
        this.poId = poId;
        this.supplier = supplier;
        this.createdBy = by;
        this.status = PoStatus.DRAFT;
        this.createdAt = LocalDateTime.now();
        this.ingredients = new ArrayList<Ingredient>();
        this.orderedQty = new ArrayList<Double>();
        this.receivedQty = new ArrayList<Double>();
    }

    // เพิ่มจำนวนสั่งของวัตถุดิบที่ผู้จำหน่ายรายนี้ส่งได้
    public void addItem(Ingredient ingredient, double qty) {
        if (status != PoStatus.DRAFT)
            throw new IllegalStateException("แก้ได้เฉพาะใบสั่งซื้อร่าง");
        if (!supplier.supplies(ingredient))
            throw new IllegalArgumentException("ผู้จำหน่ายไม่ส่งวัตถุดิบนี้");
        if (!Double.isFinite(qty) || qty <= 0)
            throw new IllegalArgumentException("จำนวนต้องมากกว่า 0");
        int index = ingredients.indexOf(ingredient);
        if (index >= 0)
            orderedQty.set(index, orderedQty.get(index) + qty);
        else {
            ingredients.add(ingredient);
            orderedQty.add(qty);
            receivedQty.add(0.0);
        }
    }

    // เปลี่ยนจากร่างเป็นสั่งแล้วเมื่อมีรายการ
    public void markOrdered() {
        if (status != PoStatus.DRAFT || orderedQty.isEmpty())
            throw new IllegalStateException("ใบสั่งซื้อต้องเป็นร่างและมีรายการ");
        status = PoStatus.ORDERED;
    }

    // ตรวจจำนวนรับให้ครบก่อนเพิ่มยอดวัตถุดิบจริง
    public void receive(List<Double> qtys) {
        if (status != PoStatus.ORDERED)
            throw new IllegalStateException("รับของซ้ำไม่ได้");
        if (qtys == null || qtys.size() != ingredients.size())
            throw new IllegalArgumentException("ต้องระบุครบทุกวัตถุดิบ");
        // ตรวจให้ครบก่อนเปลี่ยนยอด เพื่อไม่ให้รับเข้าบางรายการเมื่อข้อมูลผิด
        for (int i = 0; i < ingredients.size(); i++) {
            Double qty = qtys.get(i);
            if (qty == null || qty < 0 || !Double.isFinite(qty))
```

```
                throw new IllegalArgumentException("จำนวนรับต้องเป็นตัวเลขไม่ติดลบ");
            }
            // ข้อมูลทุกแถวผ่านแล้วจึงรับเข้าคลังทีละชนิด
            for (int i = 0; i < ingredients.size(); i++) {
                double qty = qtys.get(i);
                receivedQty.set(i, qty);
                ingredients.get(i).receive(qty, poId, createdBy);
            }
            status = PoStatus.RECEIVED;
        }

        // เมธอดอ่านใบสั่งซื้อ; รายการที่คืนออกไปเป็นสำเนาเพื่อกันการแก้ตรง ๆ
        public String poId() { return poId; }
        public Supplier supplier() { return supplier; }
        public String statusText() {
            if (status == PoStatus.DRAFT) return "ร่าง";
            if (status == PoStatus.ORDERED) return "สั่งแล้ว";
            return "รับแล้ว";
        }
        public LocalDateTime createdAt() { return createdAt; }
        public ArrayList<Ingredient> ingredients() { return new ArrayList<Ingredient>(ingredients); }
        public double orderedQty(int index) { return orderedQty.get(index); }
        public double receivedQty(int index) { return receivedQty.get(index); }
    }

```

## 2.13 คลาส OrderItem (OrderItem.java)

โดเมนออเดอร์และการเงิน (ORDER LIFECYCLE & FINANCIAL)

**บทบาทหน้าที่ (Responsibility):** ทำหน้าที่เป็นตัวแทนของอาหาร 1 บรรทัดในออเดอร์ (เช่น ผัดกะเพราปลา 2 จาน เพิ่มไข่ดาว) ทำการ Snapshot ราคาและสูตร ณ เวลาสั่ง ควบคุมวงจรชีวิตการปรุงในครัว และจัดการระบบคำขอแก้ไข/ยกเลิกอาหารส่งให้ผู้จัดการพิจารณา

**ความสัมพันธ์ UML:** `OrderItem ◆<--(Composition)-- Order, OrderItem --(1)--> Menu`

### ตารางฟิลด์และตัวแปรข้อมูล (Fields & Attributes)

| ชนิดข้อมูลและการเข้าถึง | ชื่อตัวแปร (Field) | คำอธิบายหน้าที่ในระบบ |
|---|---|---|
| `public enum { WAITING, COOKING, READY, SERVED, CANCELLED }` | **ItemStatus** | สถานะการปรุงอาหาร |
| `public enum { CANCEL, CHANGE_NOTE }` | **RequestType** | ชนิดคำขอแก้ไข: ขอยกเลิก หรือ ขอแก้หมายเหตุ |
| `public enum { NONE, PENDING, APPROVED, REJECTED }` | **RequestStatus** | สถานะคำขอ: ไม่มี, รออนุมัติ, อนุมัติแล้ว, ปฏิเสธ |
| `private String` | **orderItemId** | รหัสรายการอาหาร เช่น 'ORD-1-ITEM-1' |
| `private Menu, Recipe` | **menu, recipe** | เมนูและสูตรอาหารที่ใช้อ้างอิง (Association) |
| `private int` | **qty** | จำนวนจานที่สั่ง |
| `private double` | **price** | ราคาต่อหน่วยที่บันทึก Snapshot ไว้ (รวมค่าตัวเลือกเสริม) |
| `private String` | **spiceLevel, note, option** | ระดับความเผ็ด หมายเหตุพิเศษ และข้อความตัวเลือกเสริม |
| `private ItemStatus` | **status** | สถานะการปรุงปัจจุบัน (เริ่มต้น WAITING) |
| `private LocalDateTime` | **startTime, endTime** | เวลาเริ่มปรุง และเวลาปรุงเสร็จจริง |
| `private double` | **standardMinutes** | เวลามาตรฐานตาม SOP (นาที) |
| `private Employee` | **handledBy** | คนครัวผู้ปรุง หรือบริกรผู้เสิร์ฟ |
| `private Enums/Strings` | **requestType, requestStatus, requestReason, decisionReason** | ข้อมูลการยื่นและพิจารณาคำขอแก้ไขอาหาร |
| `private boolean` | **stockDeducted** | แฟล็กจำว่ารายการนี้ถูกตัดสต็อกจริงไปแล้วหรือไม่ เพื่อความถูกต้องตอนคืนสต็อก |
| `private Lists` | **optionNames, ingredients, quantities** | รายการ Snapshot ส่วนผสมและปริมาณต่อจาน |

### ตารางเมธอดสำคัญ (Key Methods & Operations)

| รูปแบบเมธอด (Signature) | หมวดหมู่การทำงาน | คำอธิบายรายละเอียดการทำงาน |
|---|---|---|
| `OrderItem(...)` | CONSTRUCTOR | Snapshot ราคาจาก menu.price() และคัดลอก ingredients กับ quantities จาก currentRecipe() มาเก็บไว้เองทันที |
| `addOption(int index)` | OPTION CALCULATOR | เพิ่มตัวเลือกเสริม บวกราคาเพิ่ม และดึงวัตถุดิบเสริมของตัวเลือกนั้นมาใส่ใน Snapshot |
| `amountPerDish(Ingredient ing) : double` | CALCULATION | รวมปริมาณวัตถุดิบทั้งหมดต่อ 1 จาน (รวมทั้งจากสูตรหลักและตัวเลือกเสริม) |
| `startCooking(Employee cook, double stdMin)` | KITCHEN WORKFLOW | เปลี่ยนสถานะเป็น COOKING บันทึกเวลาเริ่ม startTime และเวลามาตรฐาน SOP |
| `markReady()` | KITCHEN WORKFLOW | เปลี่ยนสถานะเป็น READY บันทึก endTime และตรวจห้ามข้ามคำขอที่รออนุมัติ |
| `serve(Employee by)` | KITCHEN WORKFLOW | เปลี่ยนสถานะเป็น SERVED (เสิร์ฟแล้ว) บันทึกพนักงาน |
| `cookingMinutes() / compareTime()` | KPI ANALYSIS | คำนวณเวลาที่ใช้ปรุงจริง และเปรียบเทียบกับเวลามาตรฐาน SOP (ถ้าเป็นบวกคือปรุงช้ากว่าเกณฑ์) |
| `cancel() / forceCancel()` | CANCELLATION & STOCK RESTORE | ยกเลิกรายการอาหาร หากเคยตัดสต็อกไปแล้ว (stockDeducted == true) จะคืนสต็อก ing.restore() ทันที |
| `requestChange(...)` | MANAGER APPROVAL WORKFLOW | ยื่นคำขอยกเลิกหรือเปลี่ยนหมายเหตุขณะอาหารกำลังปรุง สถานะเปลี่ยนเป็น PENDING |
| `decideRequest(boolean approve, ...)` | MANAGER APPROVAL WORKFLOW | ผู้จัดการพิจารณาอนุมัติ (ทำการยกเลิกจริง) หรือปฏิเสธคำขอ |
| `destroy()` | LIFECYCLE | เคลียร์รายการ Snapshot ทั้งหมดและตัดการเชื่อมโยง |

**จุดสังเกตการออกแบบเชิงวัตถุ (OOP Design Insight):**

การ Snapshot ราคาและสูตรไว้ใน OrderItem เป็นหลักการสำคัญของระบบค้าปลีกและร้านอาหาร เพราะหากในอนาคตผู้จัดการเปลี่ยนราคาเมนูจาก 69 เป็น 80 บาท หรือเปลี่ยนสูตรอาหาร บิลเก่าของลูกค้าที่สั่งไปแล้วจะต้องคงยอดเดิมไว้เสมอ

**คำถาม-คำตอบยอดฮิตสำหรับการนำเสนอ (Defense Q&A):**

ทำไมลูกค้าถึงกดยกเลิกอาหารเองไม่ได้เมื่ออาหารอยู่ในสถานะ COOKING? -> เพราะครัวได้ลงมือปรุงและใช้วัตถุดิบไปแล้ว หากลูกค้ายกเลิกโดยพลการร้านจะสูญเสียต้นทุน จึงต้องส่งคำขอให้ผู้จัดการพิจารณา (requestChange) เป็นกรณีพิเศษ

## ซอร์สโค้ดฉบับสมบูรณ์ (Full Source Code: OrderItem.java)

src/model/OrderItem.java — Java Source Code

```java
package model;

import java.time.LocalDateTime;
import java.time.Duration;
import java.util.ArrayList;

// อาหารหนึ่งรายการ เก็บราคาและสูตร ณ ตอนสั่ง เพื่อไม่ให้เมนูที่แก้ภายหลังเปลี่ยนบิลเก่า
public class OrderItem {
    public enum ItemStatus { WAITING, COOKING, READY, SERVED, CANCELLED }
    public enum RequestType { CANCEL, CHANGE_NOTE }
    public enum RequestStatus { NONE, PENDING, APPROVED, REJECTED }
    // ราคาและส่วนผสมเป็นสำเนาตอนสั่ง บิลเก่าจึงไม่เปลี่ยนเมื่อแก้ Menu/Recipe
    private String orderItemId;
    // has-a (Association): อาหารอ้างถึง Menu และ Recipe ที่มีอยู่แล้ว
    // OrderItem เก็บสำเนาราคา/ปริมาณ แต่ไม่ทำลาย Menu หรือ Recipe
    private Menu menu;
    private Recipe recipe;
    private int qty;
    private double price;
    private String spiceLevel;
    private String note;
    private String option;
    private ItemStatus status;
    private LocalDateTime startTime;
    private LocalDateTime endTime;
    private double standardMinutes;
    // has-a (Association): พนักงานผู้จัดการรายการ ไม่ได้ถูก OrderItem เป็นเจ้าของ
    private Employee handledBy;
    private String requestReason;
    private RequestType requestType;
    private RequestStatus requestStatus;
    private String decisionReason;
    private String requestedNote;
    private LocalDateTime requestedAt;
    private String decidedBy;
    // ธงนี้ช่วยคืนสต็อกเฉพาะรายการที่เคยตัดจริง
    private boolean stockDeducted;
    private boolean destroyed;
    private ArrayList<String> optionNames;
    private ArrayList<Ingredient> ingredients;
    private ArrayList<Double> quantities;

    // สร้างอาหารหนึ่งบรรทัดและคัดลอกราคา/สูตร ณ เวลาที่สั่ง
    public OrderItem(String orderItemId, Menu menu, int qty, Recipe recipe) {
        this(orderItemId, menu, qty, "ไม่ระบุ", "");
        if (recipe != null && recipe != this.recipe)
            throw new IllegalArgumentException("สูตรไม่ตรงกับเมนูที่เลือก");
    }

    // สร้างอาหารหนึ่งบรรทัดและคัดลอกราคา/สูตร ณ เวลาที่สั่ง
    public OrderItem(String orderItemId, Menu menu, int qty, String spiceLevel, String note) {
        if (qty <= 0)
            throw new IllegalArgumentException("จำนวนต้องมากกว่า 0");
        this.orderItemId = orderItemId;
        this.menu = menu;
        this.recipe = menu.currentRecipe();
        if (recipe == null)
            throw new IllegalArgumentException("เมนูยังไม่มีสูตรสมบูรณ์");
        this.qty = qty;
        this.price = menu.price();
        this.spiceLevel = spiceLevel;
        this.note = note;
        this.status = ItemStatus.WAITING;
        this.requestStatus = RequestStatus.NONE;
        this.optionNames = new ArrayList<String>();
        this.ingredients = new ArrayList<Ingredient>();
        this.quantities = new ArrayList<Double>();
        for (int i = 0; i < recipe.ingredients().size(); i++) {
```

```
            ingredients.add(recipe.ingredients().get(i));
            quantities.add(recipe.quantity(i));
        }
    }

    // เพิ่มตัวเลือก ราคา และวัตถุดิบที่ใช้เพิ่มก่อนส่งครัว
    public void addOption(int index) {
        if (stockDeducted)
            throw new IllegalStateException("แก้ตัวเลือกหลังส่งครัวไม่ได้");
        String name = menu.optionNames().get(index);
        if (optionNames.contains(name))
            throw new IllegalArgumentException("เลือกตัวเลือกนี้แล้ว");
        optionNames.add(name);
        option = String.join(", ", optionNames);
        price += menu.optionPrice(index);
        for (int i = 0; i < menu.extraCount(); i++) {
            if (menu.extraOptionIndex(i) == index) {
                ingredients.add(menu.extraIngredient(i));
                quantities.add(menu.extraQuantity(i));
            }
        }
    }

    // รวมปริมาณของวัตถุดิบเดียวกันทั้งสูตรและตัวเลือก โดยคิดต่อหนึ่งจาน
    public double amountPerDish(Ingredient ingredient) {
        double total = 0;
        for (int i = 0; i < ingredients.size(); i++) {
            if (ingredients.get(i) == ingredient)
                total += quantities.get(i);
        }
        return total;
    }

    // เข้าถึงได้จาก Order ใน package model เพื่อให้ Order คุมการเปลี่ยนจำนวน
    void changeQuantity(int qty) {
        this.qty = qty;
    }

    // จำว่าสต็อกถูกตัดแล้วและเมนูนี้เคยถูกสั่ง
    void markConfirmed() {
        stockDeducted = true;
        menu.noteOrdered();
    }

    // เปลี่ยนจากรอปรุงเป็นกำลังปรุง และจดเวลาเริ่ม
    public void startCooking(Employee cook, double standardMinutes) {
        if (!stockDeducted || status != ItemStatus.WAITING)
            throw new IllegalStateException("ต้องยืนยันและยังรอปรุงอยู่");
        this.status = ItemStatus.COOKING;
        this.startTime = LocalDateTime.now();
        this.standardMinutes = standardMinutes;
        this.handledBy = cook;
    }

    // เปลี่ยนจากรอปรุงเป็นกำลังปรุง และจดเวลาเริ่ม
    public void startCooking(Employee cook) {
        startCooking(cook, 0);
    }

    // จดเวลาจบเมื่อปรุงเสร็จ และห้ามข้ามค่าขอที่รออนุมัติ
    public void markReady() {
        if (requestStatus == RequestStatus.PENDING)
            throw new IllegalStateException("ให้ผู้จัดการพิจารณาค่าขอก่อนปรุงเสร็จ");
        if (status != ItemStatus.COOKING)
            throw new IllegalStateException("ต้องกำลังปรุง");
        status = ItemStatus.READY;
        endTime = LocalDateTime.now();
    }

    // เปลี่ยนจากพร้อมเสิร์ฟเป็นเสิร์ฟแล้ว
    public void serve(Employee by) {
        if (status != ItemStatus.READY)
            throw new IllegalStateException("ต้องพร้อมเสิร์ฟ");
```

```
        status = ItemStatus.SERVED;
        handledBy = by;
    }

    // หาเวลาที่จริงจากเวลาเริ่มถึงเวลาจบ
    public double cookingMinutes() {
        if (startTime == null || endTime == null)
            return 0;
        return Duration.between(startTime, endTime).getSeconds() / 60.0;
    }

    // เทียบเวลาจริงกับเวลามาตรฐาน; ค่าบวกหมายถึงเกินเวลา
    public int compareTime() {
        return (int) Math.round(cookingMinutes() - standardMinutes);
    }

    // เปลี่ยนสถานะเฉพาะเส้นทางที่ไม่ข้ามกฎการปรุง
    public void changeStatus(ItemStatus s) {
        if (s == ItemStatus.CANCELLED) { cancel(); return; }
        if (s == ItemStatus.READY) { markReady(); return; }
        throw new IllegalStateException("ใช้ขั้นตอนปรุงและเสิร์ฟที่กำหนดเพื่อเปลี่ยนสถานะ");
    }

    // ยกเลิกอาหารที่ยังรอปรุง
    public void cancel() {
        if (status != ItemStatus.WAITING)
            throw new IllegalStateException("เริ่มปรุงแล้วต้องส่งคำขอผู้จัดการ");
        forceCancel();
    }

    // คืนสต็อกเฉพาะรายการที่เคยตัดจริง แล้วตั้งสถานะยกเลิก
    private void forceCancel() {
        if (status == ItemStatus.CANCELLED)
            return;
        // คืนเฉพาะสต็อกที่เคยตัดจริง ตะกร้าที่ยังไม่ยืนยันจึงไม่ทำให้สต็อกเพี้ยน
        if (stockDeducted) {
            for (int i = 0; i < ingredients.size(); i++) {
                ingredients.get(i).restore(quantities.get(i) * qty, orderItemId);
            }
            stockDeducted = false;
        }
        status = ItemStatus.CANCELLED;
    }

    // บันทึกคำขอและเหตุผลเพื่อรอผู้จัดการพิจารณา
    public void requestChange(String type, String reason, String newNote) {
        if (status != ItemStatus.COOKING || requestStatus == RequestStatus.PENDING) {
            throw new IllegalStateException("ขอได้เฉพาะรายการกำลังปรุงที่ไม่มีคำขอค้าง");
        }
        if (!type.equals("ยกเลิก") && !type.equals("แก้หมายเหตุ"))
            throw new IllegalArgumentException("ชนิดคำขอไม่ถูกต้อง");
        if (reason.trim().isEmpty())
            throw new IllegalArgumentException("ต้องระบุเหตุผล");
        requestType = type.equals("ยกเลิก") ? RequestType.CANCEL : RequestType.CHANGE_NOTE;
        requestReason = reason;
        requestedNote = newNote;
        requestStatus = RequestStatus.PENDING;
        decisionReason = "";
        requestedAt = LocalDateTime.now();
    }

    // บันทึกคำขอและเหตุผลเพื่อรอผู้จัดการพิจารณา
    public void requestChange(RequestType type, String reason) {
        requestChange(type == RequestType.CANCEL ? "ยกเลิก" : "แก้หมายเหตุ", reason, note);
    }

    // ตรวจบทบาทผู้จัดการแล้วอนุมัติหรือปฏิเสธคำขอ
    public void decideRequest(boolean approve, String reason, Employee manager) {
        if (!manager.roleName().equals("ผู้จัดการ"))
            throw new IllegalArgumentException("ต้องเป็นผู้จัดการ");
        if (requestStatus != RequestStatus.PENDING)
            throw new IllegalStateException("ไม่มีคำขอค้าง");
        if (approve) {
```

```
            if (requestType == RequestType.CANCEL)
                forceCancel();
            else
                note = requestedNote;
            requestStatus = RequestStatus.APPROVED;
        } else {
            if (reason.trim().isEmpty())
                throw new IllegalArgumentException("ต้องระบุเหตุผลปฏิเสธ");
            requestStatus = RequestStatus.REJECTED;
        }
        decisionReason = manager.name() + ": " + reason;
        decidedBy = manager.name();
    }

    // ชื่อสั้นตาม P7 สำหรับอนุมัติคำขอ
    public void approve(Employee by) { decideRequest(true, "", by); }

    // ชื่อสั้นตาม P7 สำหรับปฏิเสธพร้อมเหตุผล
    public void reject(Employee by, String why) { decideRequest(false, why, by); }

    public void destroy() {
        ingredients.clear();
        quantities.clear();
        optionNames.clear();
        menu = null;
        recipe = null;
        handledBy = null;
        destroyed = true;
    }

    // เมธอดอ่านข้อมูลรายการอาหารและสถานะสำหรับหน้าจอ

    public Menu menu() {
        return menu;
    }

    public Recipe recipe() {
        return recipe;
    }

    public int quantity() {
        return qty;
    }

    // คืนราคารวมของรายการนี้: ราคาต่อจานคูณจำนวน
    public double price() {
        return price * qty;
    }

    public String spice() {
        return spiceLevel;
    }

    public String note() {
        return note;
    }

    public void changeNote(String note) {
        this.note = note;
    }

    public String statusText() {
        switch (status) {
            case WAITING: return "รอปรุง";
            case COOKING: return "กำลังปรุง";
            case READY: return "พร้อมเสิร์ฟ";
            case SERVED: return "เสิร์ฟแล้ว";
            default: return "ยกเลิก";
        }
    }

    public ArrayList<String> optionNames() {
        return optionNames;
```

```
288     }
289
290     public ArrayList<Ingredient> ingredients() {
291         return ingredients;
292     }
293
294     public double standardMinutes() {
295         return standardMinutes;
296     }
297
298     public String requestStatus() {
299         switch (requestStatus) {
300             case NONE: return "ไม่มี";
301             case PENDING: return "รออนุมัติ";
302             case APPROVED: return "อนุมัติ";
303             default: return "ปฏิเสธ";
304         }
305     }
306
307     public String requestReason() {
308         return requestReason;
309     }
310
311     public String requestType() {
312         if (requestType == null) return null;
313         return requestType == RequestType.CANCEL ? "ยกเลิก" : "แก้หมายเหตุ";
314     }
315
316     public String decisionReason() {
317         return decisionReason;
318     }
319
320
321
322     public boolean isDestroyed() {
323         return destroyed;
324     }
325 }
326
```

## 2.14 คลาส Order (Order.java)

โดเมนออเดอร์และการเงิน (ORDER LIFECYCLE & FINANCIAL)

**บทบาทหน้าที่ (Responsibility):** ทำหน้าที่เป็นตัวแทนของคำสั่งซื้อทั้งใบ (บิลอาหาร) ควบคุมสถานะภาพรวมของออเดอร์ จัดการตะกร้าสินค้า (CART) คำนวณยอดเงินรวม รวมรายการวัตถุดิบที่ต้องใช้และตัดสต็อกแบบอะตอมมิกเมื่อยืนยัน (CONFIRMED) ตลอดจนประสานงานระหว่างครัวและการชำระเงิน

**ความสัมพันธ์ UML:** `Order ◆ --(*)--> OrderItem, Order --(1)--> Table, Order --(1)--> Customer`

### ตารางฟิลด์และตัวแปรข้อมูล (Fields & Attributes)

| ชนิดข้อมูลและการเข้าถึง | ชื่อตัวแปร (Field) | คำอธิบายหน้าที่ในระบบ |
|---|---|---|
| `public enum { CART, CONFIRMED, COOKING, READY, SERVED, PAID, CANCELLED }` | **OrderStatus** | สถานะภาพรวมของออเดอร์ |
| `private String` | **orderId** | รหัสออเดอร์ เช่น 'ORD-1' |
| `private Table` | **table** | โต๊ะอาหารที่เปิดออเดอร์นี้ (Association) |
| `private Customer` | **customer** | ลูกค้าผู้สั่งอาหาร (Association) |
| `private Payment` | **payment** | ใบชำระเงินเมื่อเช็คบิลแล้ว (Association) |
| `private OrderStatus` | **status** | สถานะออเดอร์ปัจจุบัน |
| `private double` | **totalAmount** | ยอดเงินรวมล่าสุด |
| `private LocalDateTime` | **createdAt** | วันเวลาที่เปิดบิล |
| `private List<OrderItem>` | **items** | รายการอาหารทั้งหมดในบิลนี้ (Composition) |
| `private int` | **itemNumber** | ตัวนับลำดับไอเทมในบิล |
| `private boolean` | **destroyed** | แฟล็กสถานะการทำลาย |

### ตารางเมธอดสำคัญ (Key Methods & Operations)

| รูปแบบเมธอด (Signature) | หมวดหมู่การทำงาน | คำอธิบายรายละเอียดการทำงาน |
|---|---|---|
| `Order(String orderId, Table table, Customer customer)` | CONSTRUCTOR | สร้างออเดอร์ใหม่ในสถานะตะกร้า (CART) พร้อมจดเวลาเปิดบิล |
| `addItem(Menu menu, int qty, String spice, String note) : OrderItem` | COMPOSITION CREATOR | สร้าง OrderItem ใหม่เก็บไว้ในรายการ ทำได้เฉพาะสถานะ CART และเมนูต้องเปิดขายอยู่ |
| `removeItem(int index)` | CART MANAGEMENT | ลบอาหารออกจากตะกร้าก่อนยืนยัน สั่ง item.destroy() และนำออกจาก List |
| `calculateTotal() : double` | FINANCIAL CALCULATION | คำนวณผลรวมราคาของรายการอาหารทั้งหมดที่ไม่ถูกยกเลิก (statusText != 'ยกเลิก') |
| `allIngredients() / requiredQuantity()` | INVENTORY AGGREGATION (PRIVATE) | รวมรายชื่อวัตถุดิบและคำนวณปริมาณที่ต้องใช้ทั้งหมดของทั้งบิล เพื่อนำไปตรวจสต็อก |
| `confirm()` | ATOMIC STOCK CHECK & DEDUCT | หัวใจสำคัญ: ตรวจสอบวัตถุดิบทุกชนิดให้ครบว่าพอหรือไม่ (Loop 1) หากพอครบทุกตัว จึงวนลูปตัดสต็อกจริง (Loop 2) แล้วเปลี่ยนสถานะเป็น CONFIRMED และเปลี่ยนโต๊ะเป็น 'ใช้งาน' |
| `changeQuantity(int index, int newQty)` | DIFFERENTIAL STOCK UPDATE | ปรับจำนวนจานอาหารที่รอปรุง หากเพิ่มจานจะตรวจและตัดสต็อกส่วนต่าง หากลดจานจะคืนสต็อกส่วนต่าง |
| `refreshStatus()` | STATUS ROLLUP | สรุปสถานะของออเดอร์ทั้งใบโดยดูจากสถานะของ OrderItem แต่ละตัว (เช่น ถ้าปรุงเสร็จครบทุกจานจะเป็น READY ถ้าเสิร์ฟครบจะเป็น SERVED) |
| `isServed() : boolean` | PAYMENT PREREQUISITE | ตรวจสอบว่าอาหารที่สั่งทั้งหมดได้รับการเสิร์ฟครบถ้วนแล้วหรือยัง เพื่อเป็นเงื่อนไขก่อนคิดเงิน |
| `markPaid(Payment payment)` | SETTLEMENT | บันทึกการชำระเงินและเปลี่ยนสถานะเป็น PAID |
| `cancel()` | CANCELLATION | ยกเลิกออเดอร์ทั้งใบ (ทำได้เฉพาะเมื่อยังไม่มีรายการใดเริ่มปรุง) |
| `destroy()` | LIFECYCLE (COMPOSITION) | สั่ง item.destroy() ทุกรายการ ล้าง items.clear() และตัดการเชื่อมโยงทั้งหมด |

**จุดสังเกตการออกแบบเชิงวัตถุ (OOP Design Insight):**
ใน confirm() การแบ่งเป็น 2 Loop (Loop ตรวจเช็กก่อน แล้วค่อย Loop ตัดสต็อก) เป็นรูปแบบ Two-Phase Commit แบบย่อ เพื่อการันตีความเป็น Atomic Transaction ป้องกันกรณีตัดปลาไปแล้ว แต่พบว่าไข่ไก่ไม่พอ ซึ่งจะทำให้สต็อกปลาหายไปฟรี ๆ

**คำถาม-คำตอบยอดฮิตสำหรับการนำเสนอ (Defense Q&A):**
ทำไม Order ถึงไม่เก็บสถานะโต๊ะไว้เอง ทำไมต้องไปสั่ง table.changeTableStatus('ใช้งาน')? -> เพื่อให้โต๊ะตัวจริงถูกเปลี่ยนสถานะในระบบกลาง และหน้าจอเปิดโต๊ะใหม่จะไม่เปิดซ้ำโต๊ะที่มีคนนั่งอยู่

## ซอร์สโค้ดฉบับสมบูรณ์ (Full Source Code: Order.java)

src/model/Order.java — Java Source Code

```java
package model;

import java.time.LocalDateTime;
import java.util.ArrayList;
import java.util.List;

// ออเดอร์เป็นเจ้าของ OrderItem และอ้างถึงโต๊ะ ลูกค้า และการชำระเงิน
public class Order {
    public enum OrderStatus { CART, CONFIRMED, COOKING, READY, SERVED, PAID, CANCELLED }
    // Order เป็นเจ้าของรายการอาหาร แต่เพียงอ้างถึง Table, Customer และ Payment
    private String orderId;
    // has-a (Association): ออเดอร์อ้างถึงโต๊ะ ลูกค้า และใบชำระเงิน
    // object เหล่านี้ยังมีชีวิตได้ แม้ออเดอร์นี้ถูกลบ
    private Table table;
    private Customer customer;
    private Payment payment;
    private OrderStatus status;
    private double totalAmount;
    private LocalDateTime createdAt;
    // has-a (Composition): Order เป็นเจ้าของ OrderItem หลายรายการ
    // destroy() ของ Order จะเรียก destroy() ของแต่ละ OrderItem
    private List<OrderItem> items;
    private int itemNumber;
    private boolean destroyed;

    // สร้างออเดอร์ในสถานะตะกร้า และจดเวลาที่สร้าง
    public Order(String orderId, Table table) {
        this(orderId, table, new Customer("CUSTOMER-" + orderId));
    }

    // สร้างออเดอร์ในสถานะตะกร้า และจดเวลาที่สร้าง
    public Order(String orderId, Table table, Customer customer) {
        this.orderId = orderId;
        this.table = table;
        this.customer = customer;
        this.status = OrderStatus.CART;
        this.createdAt = LocalDateTime.now();
        this.items = new ArrayList<OrderItem>();
    }

    // เพิ่ม OrderItem เมื่อเมนูเปิดขายและออเดอร์ยังเป็นตะกร้า
    public OrderItem addItem(Menu menu, int qty, String spice, String note) {
        if (status != OrderStatus.CART)
            throw new IllegalStateException("เพิ่มอาหารได้เฉพาะในตะกร้า");
        if (!menu.statusText().equals("เปิดขาย"))
            throw new IllegalArgumentException("เมนูปิดขาย");
        itemNumber++;
        OrderItem item = new OrderItem(orderId + "-ITEM-" + itemNumber, menu, qty, spice, note);
        items.add(item);
        return item;
    }

    // ลบรายการจากตะกร้าก่อนยืนยัน
    public void removeItem(int index) {
        if (status != OrderStatus.CART)
            throw new IllegalStateException("ลบได้เฉพาะตะกร้า");
        items.get(index).destroy();
        items.remove(index);
    }

    // ลบรายการจากตะกร้าก่อนยืนยัน
    public void removeItem(OrderItem item) {
        int index = items.indexOf(item);
        if (index < 0) throw new IllegalArgumentException("ไม่พบรายการในออเดอร์");
        removeItem(index);
    }

    // ผูกสมาชิกกับลูกค้าในออเดอร์ก่อนยืนยัน
```

```
    public void attachMember(Member member) {
        if (status != OrderStatus.CART) throw new IllegalStateException("แนบสมาชิกได้ก่อนยืนยันเท่านั้น");
        customer.linkMember(member);
    }

    // รวมราคาของรายการที่ยังไม่ยกเลิก
    public double calculateTotal() {
        double total = 0;
        // รายการที่ยกเลิกไม่อยู่ในยอดบิล แม้ยังเก็บไว้เพื่อดูประวัติ
        for (int i = 0; i < items.size(); i++) {
            if (!items.get(i).statusText().equals("ยกเลิก"))
                total += items.get(i).price();
        }
        totalAmount = total;
        return totalAmount;
    }

    // รวบรวมชื่อวัตถุดิบโดยไม่ซ้ำ เพื่อให้ตรวจยอดรวมของหลายจานได้
    private ArrayList<Ingredient> allIngredients() {
        ArrayList<Ingredient> result = new ArrayList<Ingredient>();
        for (int i = 0; i < items.size(); i++) {
            OrderItem item = items.get(i);
            if (!item.statusText().equals("ยกเลิก")) {
                for (int j = 0; j < item.ingredients().size(); j++) {
                    Ingredient ingredient = item.ingredients().get(j);
                    if (!result.contains(ingredient))
                        result.add(ingredient);
                }
            }
        }
        return result;
    }

    // คำนวณจำนวนวัตถุดิบรวมของทุกรายการในออเดอร์
    private double requiredQuantity(Ingredient ingredient) {
        double total = 0;
        for (int i = 0; i < items.size(); i++) {
            OrderItem item = items.get(i);
            if (!item.statusText().equals("ยกเลิก")) {
                total += item.amountPerDish(ingredient) * item.quantity();
            }
        }
        return total;
    }

    // ตรวจวัตถุดิบทุกชนิดให้พอครบก่อนตัดสต็อกและส่งครัว
    public void confirm() {
        if (status != OrderStatus.CART || items.isEmpty())
            throw new IllegalStateException("ตะกร้าต้องมีอาหารและยังไม่ยืนยัน");
        ArrayList<Ingredient> ingredients = allIngredients();
        // ตรวจให้ครบทุกชนิดก่อน แล้วค่อยตัด
        // เพื่อไม่ให้ตัดไปครึ่งหนึ่งแล้วพบว่าอีกชนิดไม่พอ
        for (int i = 0; i < ingredients.size(); i++) {
            Ingredient ingredient = ingredients.get(i);
            if (!ingredient.hasEnough(requiredQuantity(ingredient))) {
                throw new IllegalArgumentException("วัตถุดิบไม่พอ: " + ingredient.name());
            }
        }
        for (int i = 0; i < ingredients.size(); i++) {
            Ingredient ingredient = ingredients.get(i);
            ingredient.deduct(requiredQuantity(ingredient), orderId);
        }
        // หลังตัดครบแล้วจึงเปลี่ยนรายการเป็นงานของครัว
        for (int i = 0; i < items.size(); i++)
            items.get(i).markConfirmed();
        status = OrderStatus.CONFIRMED;
        table.changeTableStatus("ใช้งาน");
    }

    // แก้จำนวนอาหารและตัดหรือคืนสต็อกเฉพาะส่วนต่าง
    public void changeQuantity(int index, int newQuantity) {
        OrderItem item = items.get(index);
        if (newQuantity <= 0 || !item.statusText().equals("รอปรุง") || payment != null) {
```

```
            throw new IllegalArgumentException("แก้ได้เฉพาะรายการรอปรุงที่ยังไม่ชำระ และจำนวนต้องมากกว่า 0");
        }
        if (status == OrderStatus.CART) {
            item.changeQuantity(newQuantity);
            return;
        }
        int difference = newQuantity - item.quantity();
        // เพิ่มจานต้องเช็กสต็อกส่วนต่าง; ลดจานจะคืนเฉพาะส่วนต่าง
        ArrayList<Ingredient> ingredients = allIngredients();
        if (difference > 0) {
            for (int i = 0; i < ingredients.size(); i++) {
                Ingredient ingredient = ingredients.get(i);
                if (!ingredient.hasEnough(item.amountPerDish(ingredient) * difference)) {
                    throw new IllegalArgumentException("วัตถุดิบไม่พอ: " + ingredient.name());
                }
            }
        }
        for (int i = 0; i < ingredients.size(); i++) {
            Ingredient ingredient = ingredients.get(i);
            double change = item.amountPerDish(ingredient) * difference;
            if (change > 0)
                ingredient.deduct(change, orderId);
            else if (change < 0)
                ingredient.restore(-change, orderId);
        }
        item.changeQuantity(newQuantity);
    }

    // ยกเลิกรายการที่ยังไม่ชำระ แล้วอัปเดตสถานะออเดอร์
    public void cancelItem(int index) {
        if (payment != null)
            throw new IllegalStateException("ชำระแล้ว แก้รายการไม่ได้");
        items.get(index).cancel();
        refreshStatus();
    }

    // คืน true เมื่อรายการที่ไม่ถูกยกเลิกเสิร์ฟครบทุกจาน
    public boolean isServed() {
        boolean hasFood = false;
        for (int i = 0; i < items.size(); i++) {
            String itemStatus = items.get(i).statusText();
            if (!itemStatus.equals("ยกเลิก")) {
                hasFood = true;
                if (!itemStatus.equals("เสิร์ฟแล้ว"))
                    return false;
            }
        }
        return hasFood;
    }

    // สถานะออเดอร์สรุปจากอาหารที่ยังไม่ยกเลิกทั้งหมด
    public void refreshStatus() {
        if (status == OrderStatus.CART || status == OrderStatus.PAID)
            return;
        int active = 0;
        int ready = 0;
        int served = 0;
        boolean cooking = false;
        for (int i = 0; i < items.size(); i++) {
            String value = items.get(i).statusText();
            if (!value.equals("ยกเลิก"))
                active++;
            if (value.equals("กำลังปรุง"))
                cooking = true;
            if (value.equals("พร้อมเสิร์ฟ"))
                ready++;
            if (value.equals("เสิร์ฟแล้ว"))
                served++;
        }
        if (active == 0)
            status = OrderStatus.CANCELLED;
        else if (served == active)
            status = OrderStatus.SERVED;
```

```
215         else if (ready + served == active)
216             status = OrderStatus.READY;
217         else if (cooking)
218             status = OrderStatus.COOKING;
219         else
220             status = OrderStatus.CONFIRMED;
221     }
222
223     // ผูก Payment ที่ยืนยันแล้วและเปลี่ยนสถานะเป็นชำระแล้ว
224     void markPaid(Payment payment) {
225         this.payment = payment;
226         status = OrderStatus.PAID;
227     }
228
229     // ยกเลิกทั้งออเดอร์เมื่อยังไม่มีรายการเริ่มปรุง
230     public void cancel() {
231         if (payment != null) throw new IllegalStateException("ชำระแล้ว ยกเลิกไม่ได้");
232         for (OrderItem item : items) {
233             if (!item.statusText().equals("รอปรุง") && !item.statusText().equals("ยกเลิก"))
234                 throw new IllegalStateException("มีรายการเริ่มปรุงแล้ว ให้ส่งคำขอผู้จัดการ");
235         }
236         for (OrderItem item : items) if (!item.statusText().equals("ยกเลิก")) item.cancel();
237         status = OrderStatus.CANCELLED;
238     }
239
240     // จำกัดการเปลี่ยนสถานะให้ผ่านขั้นตอนที่มีกฎรองรับ
241     public void changeStatus(OrderStatus s) {
242         if (s == OrderStatus.CANCELLED) { cancel(); return; }
243         if (s == status) return;
244         throw new IllegalStateException("สถานะเปลี่ยนผ่านขั้นตอนของรายการอาหารและการชำระเงิน");
245     }
246
247     public void destroy() {
248         for (int i = 0; i < items.size(); i++)
249             items.get(i).destroy();
250         items.clear();
251         table = null;
252         customer = null;
253         payment = null;
254         destroyed = true;
255     }
256
257     // เมธอดอ่านออเดอร์; statusText() แปลง enum เป็นข้อความที่ผู้ใช้เห็น
258     public String orderId() {
259         return orderId;
260     }
261
262     public Table table() {
263         return table;
264     }
265
266     public Customer customer() {
267         return customer;
268     }
269
270     public Member linkedMember() {
271         return customer.linkedMember();
272     }
273
274     public Payment payment() {
275         return payment;
276     }
277
278     public String statusText() {
279         switch (status) {
280             case CART: return "ตะกร้า";
281             case CONFIRMED: return "ยืนยันแล้ว";
282             case COOKING: return "กำลังปรุง";
283             case READY: return "พร้อมเสิร์ฟ";
284             case SERVED: return "เสิร์ฟแล้ว";
285             case PAID: return "ชำระแล้ว";
286             default: return "ยกเลิก";
287         }
```

```
288     }
289
290     // อ่านยอดรวมล่าสุดโดยคำนวณใหม่ก่อนคืนค่า
291
292     public LocalDateTime createdAt() {
293         return createdAt;
294     }
295
296     public List<OrderItem> items() {
297         return items;
298     }
299
300     public boolean isDestroyed() {
301         return destroyed;
302     }
303 }
304
```

## 2.15 คลาส Member (Member.java)

โดเมนบุคคลและสถานที่ (PEOPLE & TABLE)

**บทบาทหน้าที่ (Responsibility):** ทำหน้าที่เก็บข้อมูลสมาชิกของร้านอาหาร (ชื่อ, เบอร์โทรศัพท์ 10 หลัก, ยอดแต้มสะสม), จัดการการคำนวณแต้มสะสมจากการรับประทานอาหาร, การแลกแต้มเป็นส่วนลดเงินสด, เป็นเจ้าของประวัติ PointTransaction และควบคุมกฎการคิดแต้มระดับร้าน (Static Rules)

**ความสัมพันธ์ UML:** `Member ◆--(*)--> PointTransaction`

### ตารางฟิลด์และตัวแปรข้อมูล (Fields & Attributes)

| ชนิดข้อมูลและการเข้าถึง | ชื่อตัวแปร (Field) | คำอธิบายหน้าที่ในระบบ |
|---|---|---|
| `private String` | **memberId** | รหัสสมาชิก เช่น 'MEM-1' |
| `private String` | **name** | ชื่อ-นามสกุลของสมาชิก |
| `private String` | **phone** | เบอร์โทรศัพท์ 10 หลักขึ้นต้นด้วย 0 (มี Regex ตรวจสอบ) |
| `private int` | **points** | ยอดแต้มสะสมปัจจุบันของสมาชิกรายนี้ |
| `private List<PointTransaction>` | **transactions** | รายการประวัติการได้/ใช้แต้มของสมาชิก (Composition) |
| `private boolean` | **destroyed** | แฟล็กสถานะการทำลาย |
| `private static int` | **bahtPerPoint** | กฎกลาง: ยอดซื้อกี่บาทได้ 1 แต้ม (ค่าเริ่มต้น 50 บาท/แต้ม) |
| `private static double` | **pointValue** | กฎกลาง: 1 แต้มมีมูลค่าส่วนลดกี่บาท (ค่าเริ่มต้น 1 บาท/แต้ม) |
| `private static int` | **minRedeem** | กฎกลาง: ขั้นต่ำในการแลกแต้มกี่แต้ม (ค่าเริ่มต้น 10 แต้ม) |

### ตารางเมธอดสำคัญ (Key Methods & Operations)

| รูปแบบเมธอด (Signature) | หมวดหมู่การทำงาน | คำอธิบายรายละเอียดการทำงาน |
|---|---|---|
| `Member(String memberId, String name, String phone)` | CONSTRUCTOR | สร้างสมาชิกใหม่ ตรวจสอบเบอร์โทร และเตรียม List ประวัติแต้ม |
| `changePhone(String phone)` | VALIDATION / MUTATOR | เปลี่ยนเบอร์โทรศัพท์ ตรวจด้วย Regular Expression: '0[0-9]{9}' หากไม่ถูกต้องจะโยน IllegalArgumentException |
| `earnPoints(double netAmount, Payment payment) : int` | LOYALTY BUSINESS LOGIC | คำนวณแต้มใหม่: netAmount / bahtPerPoint (ปัดเศษทิ้ง) บวกแต้มเพิ่ม และสร้าง PointTransaction ประเภท 'สะสม' |
| `redeemPoints(int use, Payment payment)` | LOYALTY BUSINESS LOGIC | แลกแต้ม: ตรวจสอบ use >= minRedeem และ use <= points หักแต้ม และสร้าง PointTransaction ประเภท 'แลก' |
| `setRule(int b, double v, int m)` | CLASS-LEVEL CONFIGURATION (PUBLIC STATIC) | ผู้จัดการแก้ไขกฎแต้มกลางของร้าน มีผลต่อสมาชิกทุกคนทันที |
| `validateRedeem(int points, double amount) : boolean` | VALIDATION HELPER (PUBLIC STATIC) | ตรวจเงื่อนไขแต้มขั้นต่ำและมูลค่าส่วนลดไม่เกินยอดบิล |
| `destroy()` | LIFECYCLE (COMPOSITION) | สั่ง destroy() ประวัติแต้มทุกตัว ล้าง List และตั้ง destroyed = true |

**จุดสังเกตการออกแบบเชิงวัตถุ (OOP Design Insight):**

การใช้คีย์เวิร์ด static สำหรับ bahtPerPoint, pointValue, minRedeem เป็นตัวอย่างที่สมบูรณ์แบบของการแบ่งแยกระหว่าง Instance Variable (แต้มของใครของมัน) กับ Class Variable (กฎเกณฑ์ที่ร้านใช้ร่วมกันทุกคน)

**คำถาม-คำตอบยอดฮิตสำหรับการนำเสนอ (Defense Q&A):**

ทำไมยอดแต้มที่ได้ถึงแปลงเป็น int (ปัดเศษทิ้ง)? -> เพราะแต้มสะสมไม่มีหน่วยเป็นทศนิยม เศษของเงินที่ไม่ครบ 50 บาท (ตามอัตรา 50 บาท/แต้ม) จะไม่ถูกนำมาคิดเป็นแต้ม

## ซอร์สโค้ดฉบับสมบูรณ์ (Full Source Code: Member.java)

src/model/Member.java — Java Source Code

```java
package model;

import java.util.ArrayList;
import java.util.List;

// ข้อมูลสมาชิกเป็นของแต่ละ object แต่กฎแต้มเป็น static จึงใช้ร่วมกันทั้งร้าน
public class Member {
    // ข้อมูลและประวัติเป็นของสมาชิกแต่ละคน
    private String memberId;
    private String name;
    private String phone;
    private int points;
    // has-a (Composition): Member เป็นเจ้าของประวัติ PointTransaction
    // เมื่อทำลาย Member จะทำลายประวัติแต้มของสมาชิกคนนี้ด้วย
    private List<PointTransaction> transactions;
    private boolean destroyed;
    // กฎ static ใช้ร่วมกันทุกสมาชิก เมื่อผู้จัดการเปลี่ยนจึงมีผลทั้งร้าน
    private static int bahtPerPoint = 50;
    private static double pointValue = 1;
    private static int minRedeem = 10;

    // สร้างสมาชิกพร้อมตรวจรูปแบบเบอร์โทร
    public Member(String memberId, String name, String phone) {
        this.memberId = memberId;
        this.name = name;
        changePhone(phone);
        this.transactions = new ArrayList<PointTransaction>();
    }

    // คำนวณแต้มจากยอดสุทธิ เพิ่มยอด และบันทึกประวัติ
    public int earnPoints(double netAmount, Payment payment) {
        if (!Double.isFinite(netAmount) || netAmount < 0)
            throw new IllegalArgumentException("ยอดติดลบไม่ได้");
        int earned = (int) (netAmount / bahtPerPoint);
        if (earned > 0) {
            points += earned;
            addTransaction("สะสม", earned, payment);
        }
        return earned;
    }

    // คำนวณแต้มจากยอดสุทธิ เพิ่มยอด และบันทึกประวัติ
    public int earnPoints(double amount) {
        return earnPoints(amount, null);
    }

    // ตรวจขั้นต่ำและแต้มคงเหลือก่อนหักแต้ม
    public void redeemPoints(int use, Payment payment) {
        if (use < minRedeem || use > points)
            throw new IllegalArgumentException("แต้มไม่พอหรือไม่ถึงขั้นต่ำ");
        points -= use;
        addTransaction("แลก", use, payment);
    }

    // ตรวจขั้นต่ำและแต้มคงเหลือก่อนหักแต้ม
    public void redeemPoints(int points) {
        redeemPoints(points, null);
    }

    // ตรวจเกณฑ์ส่วนลดกลางของร้าน; จำนวนแต้มคงเหลืออยู่ที่สมาชิกแต่ละคน
    public static boolean validateRedeem(int points, double amount) {
        return points >= minRedeem && Double.isFinite(amount)
                && amount >= 0 && points * pointValue <= amount;
    }

    private void addTransaction(String type, int amount, Payment payment) {
        String id = memberId + "-PTX-" + (transactions.size() + 1);
        transactions.add(new PointTransaction(id, type, amount, points, payment));
```

```
    }

    // เปลี่ยนกฎกลางของร้านที่สมาชิกทุกคนใช้ร่วมกัน
    public static void setRule(int b, double v, int m) {
        if (b <= 0 || !Double.isFinite(v) || v <= 0 || m <= 0)
            throw new IllegalArgumentException("ค่ากฎแต้มต้องมากกว่า 0");
        bahtPerPoint = b;
        pointValue = v;
        minRedeem = m;
    }

    public void destroy() {
        for (int i = 0; i < transactions.size(); i++)
            transactions.get(i).destroy();
        transactions.clear();
        destroyed = true;
    }

    // เมธอดอ่านข้อมูลสมาชิกและกฎแต้มเพื่อแสดงหน้าจอ
    public String memberId() {
        return memberId;
    }

    public String name() {
        return name;
    }

    // เปลี่ยนชื่อสมาชิกหลังสมัคร
    public void rename(String name) {
        this.name = name;
    }

    public String phone() {
        return phone;
    }

    // ตรวจเบอร์ 10 หลักก่อนเปลี่ยนข้อมูลสมาชิก
    public void changePhone(String phone) {
        if (!phone.matches("0[0-9]{9}"))
            throw new IllegalArgumentException("เบอร์โทรต้องมี 10 หลักขึ้นต้นด้วย 0");
        this.phone = phone;
    }

    public int points() {
        return points;
    }

    public List<PointTransaction> transactions() {
        return transactions;
    }

    public static int bahtPerPoint() {
        return bahtPerPoint;
    }

    public static double pointValue() {
        return pointValue;
    }

    public static int minimumRedeem() {
        return minRedeem;
    }

    public boolean isDestroyed() {
        return destroyed;
    }
}

```

## 2.16 คลาส Payment (Payment.java)

โดเมนออเดอร์และการเงิน (ORDER LIFECYCLE & FINANCIAL)

**บทบาทหน้าที่ (Responsibility):** ทำหน้าที่บริหารการชำระเงิน ออกใบเสร็จรับเงิน คำนวณส่วนลดจากการแลกแต้มสมาชิก คำนวณยอดเงินสุทธิ คำนวณเงินทอน และยืนยันการปิดบิลพร้อมบันทึกแต้มสะสมใหม่เข้าสู่บัญชีสมาชิกแบบเบ็ดเสร็จ

**ความสัมพันธ์ UML:** `Payment --(1)--> Order, Payment --(1)--> Employee`

### ตารางฟิลด์และตัวแปรข้อมูล (Fields & Attributes)

| ชนิดข้อมูลและการเข้าถึง | ชื่อตัวแปร (Field) | คำอธิบายหน้าที่ในระบบ |
|---|---|---|
| `public enum { CASH, QR }` | **PayMethod** | วิธีการชำระเงิน: CASH (เงินสด) หรือ QR (สแกนจ่าย) |
| `private String` | **paymentId** | รหัสการชำระเงิน เช่น 'PAY-1' |
| `private Order` | **order** | ออเดอร์ที่ทำการชำระเงิน (Association) |
| `private Employee` | **receivedBy** | แคชเชียร์ผู้รับเงิน (Association) |
| `private PayMethod` | **method** | ช่องทางการชำระเงิน |
| `private double` | **received** | จำนวนเงินที่รับจากลูกค้า |
| `private double` | **change** | จำนวนเงินทอน |
| `private double` | **discount** | มูลค่าส่วนลดที่ได้จากการแลกแต้ม |
| `private double` | **netAmount** | ยอดเงินสุทธิที่ต้องชำระหลังหักส่วนลด |
| `private int` | **pointsToRedeem** | จำนวนแต้มที่ลูกค้าขอใช้แลกส่วนลด |
| `private int` | **earnedPoints** | จำนวนแต้มใหม่ที่ลูกค้าได้รับจากบิลนี้ |
| `private LocalDateTime` | **time** | เวลาที่ยืนยันการชำระเงินสำเร็จ |
| `private String` | **receiptNo** | เลขที่ใบเสร็จ เช่น 'RCT-PAY-1' |
| `private boolean` | **confirmed** | แฟล็กยืนยันว่าการชำระเงินเสร็จสมบูรณ์แล้ว |

### ตารางเมธอดสำคัญ (Key Methods & Operations)

| รูปแบบเมธอด (Signature) | หมวดหมู่การทำงาน | คำอธิบายรายละเอียดการทำงาน |
|---|---|---|
| `Payment(String paymentId, Order order, Employee by)` | CONSTRUCTOR | สร้างใบชำระเงิน ผูกกับออเดอร์และแคชเชียร์ พร้อมสร้าง receiptNo |
| `calculateDiscount(int points) : double` | DISCOUNT CALCULATOR | คำนวณส่วนลดจากแต้ม ตรวจสอบสิทธิ์และจำนวนแต้มคงเหลือของสมาชิก |
| `calculateChange() : double` | CHANGE CALCULATOR | คำนวณเงินทอน: received - netAmount (ปัดทศนิยม 2 ตำแหน่ง) |
| `pay(String method, double received, int points)` | PREPARATION & VALIDATION | คำนวณยอดเงิน เตรียมบิล ตรวจสอบเงินรับว่าพอหรือไม่ แต่ยังไม่หักแต้มและยังไม่ปิดออเดอร์ |
| `confirm()` | ATOMIC SETTLEMENT | ยืนยันการชำระเงินจริง: หักแต้มที่แลก (redeemPoints), แจกแต้มใหม่ (earnPoints), บันทึกเวลาชำระเงิน, ตั้ง confirmed = true และสั่ง order.markPaid(this) ปิดออเดอร์ทันที ป้องกันการยืนยันซ้ำ |
| `receiptNo(), method(), received(), change(), discount(), netAmount(), earnedPoints()` | ACCESSORS | อ่านข้อมูลสำหรับพิมพ์ใบเสร็จและทำรายงานยอดขาย |

**จุดสังเกตการออกแบบเชิงวัตถุ (OOP Design Insight):**

การแยกเป็น 2 ขั้นตอนระหว่าง pay() (เตรียมคำนวณและแสดงตัวอย่างบิล) กับ confirm() (ยืนยันตัดเงินและแต้มจริง) ช่วยให้ผู้ใช้สามารถตรวจสอบความถูกต้องบนหน้าจอก่อนยืนยันขั้นสุดท้ายได้อย่างปลอดภัย

**คำถาม-คำตอบยอดฮิตสำหรับการนำเสนอ (Defense Q&A):**

ทำไม Payment ไม่เป็น Composition กับ Order? -> ในสถาปัตยกรรมนี้ Payment ถูกออกแบบให้เป็นเอกสารทางการเงินอิสระ (Association) แม้ Order จะถูกเคลียร์หรือยกเลิก ข้อมูลการเงินใน Payment ยังสามารถใช้สอบทานบัญชีในรายงาน UC06 ได้

## ซอร์สโค้ดฉบับสมบูรณ์ (Full Source Code: Payment.java)

src/model/Payment.java — Java Source Code

```java
package model;

import java.time.LocalDateTime;

// Payment คำนวณเงินและแต้ม ส่วน Main รับค่าตอบจากผู้ใช้และแสดงใบเสร็จ
public class Payment {
    public enum PayMethod { CASH, QR }
    // เก็บยอดและวิธีชำระของหนึ่งบิล; confirm() จะเปลี่ยนข้อมูลสมาชิกและออเดอร์
    private String paymentId;
    // has-a (Association): Payment อ้างถึง Order และ Employee ผู้รับเงิน
    private Order order;
    private Employee receivedBy;
    private PayMethod method;
    private double received;
    private double change;
    private double discount;
    private double netAmount;
    private int pointsToRedeem;
    private int earnedPoints;
    private LocalDateTime time;
    private String receiptNo;
    private boolean confirmed;

    // ผูกการชำระกับออเดอร์และผู้รับเงิน พร้อมสร้างเลขใบเสร็จ
    public Payment(String paymentId, Order order, Employee by) {
        this.paymentId = paymentId;
        this.order = order;
        this.receivedBy = by;
        this.receiptNo = "RCT-" + paymentId;
    }

    // ตรวจสิทธิ์แลกแต้มและคืนมูลค่าส่วนลด
    public double calculateDiscount(int points) {
        if (points < 0) throw new IllegalArgumentException("แต้มต้องไม่ติดลบ");
        Member member = order.linkedMember();
        if (points > 0 && (member == null || points < Member.minimumRedeem()
                || points > member.points()
                || points * Member.pointValue() > order.calculateTotal()))
            throw new IllegalArgumentException("สมาชิกหรือจำนวนแต้มไม่ผ่านเงื่อนไข");
        pointsToRedeem = points;
        return points * Member.pointValue();
    }

    // คำนวณเงินทอนจากเงินรับลบยอดสุทธิ
    public double calculateChange() {
        if (method == null) throw new IllegalStateException("ยังไม่ได้เลือกวิธีชำระ");
        return Math.round((received - netAmount) * 100.0) / 100.0;
    }

    // ตรวจวิธีชำระ แต้ม และเงินรับ แล้วคำนวณยอดสุทธิ
    public void pay(PayMethod method, double received) {
        if (method == null) throw new IllegalArgumentException("ต้องเลือกวิธีชำระ");
        pay(method == PayMethod.CASH ? "เงินสด" : "QR", received, pointsToRedeem);
    }

    // ตรวจวิธีชำระ แต้ม และเงินรับ แล้วคำนวณยอดสุทธิ
    public void pay(String method, double received, int points) {
        // pay() คำนวณและเตรียมบิลก่อน ยังไม่หักแต้มและยังไม่ปิดออเดอร์
        if (confirmed || order.payment() != null || !order.isServed()) {
            throw new IllegalStateException("ต้องเสิร์ฟครบและยังไม่ชำระ");
        }
        if (!method.equals("เงินสด") && !method.equals("QR"))
            throw new IllegalArgumentException("ช่องทางไม่ถูกต้อง");
        if (points < 0 || !Double.isFinite(received))
            throw new IllegalArgumentException("เงินหรือแต้มไม่ถูกต้อง");
        Member member = order.linkedMember();
        if (points > 0) {
            if (member == null || points < Member.minimumRedeem() || points > member.points()) {
```

```
                throw new IllegalArgumentException("สมาชิกหรือจำนวนแต้มไม่ผ่านเงื่อนไข");
            }
        }
        double total = order.calculateTotal();
        double reduction = points * Member.pointValue();
        if (reduction > total)
            throw new IllegalArgumentException("ส่วนลดเกินยอดบิล");
        double due = Math.round((total - reduction) * 100.0) / 100.0;
        if (method.equals("เงินสด") && received < due)
            throw new IllegalArgumentException("เงินไม่พอ");
        this.method = method.equals("เงินสด") ? PayMethod.CASH : PayMethod.QR;
        this.pointsToRedeem = points;
        this.discount = reduction;
        this.netAmount = due;
        if (method.equals("QR"))
            this.received = due;
        else
            this.received = received;
        this.change = calculateChange();
    }

    // หักแต้มที่ใช้ เพิ่มแต้มที่ได้ และปิดออเดอร์ครั้งเดียว
    public void confirm() {
        if (confirmed || order.payment() != null || method == null || !order.isServed()) {
            throw new IllegalStateException("ยังไม่พร้อมชำระ หรือยืนยันไปแล้ว");
        }
        Member member = order.linkedMember();
        // เปลี่ยนยอดแต้มจริงที่นี่ เมื่อผู้ใช้ยืนยันชำระแล้วเท่านั้น
        if (pointsToRedeem > 0) {
            if (member == null || pointsToRedeem > member.points() || pointsToRedeem < Member.minimumRedeem())
{
                throw new IllegalStateException("แต้มเปลี่ยน กรุณาคำนวณการชำระใหม่");
            }
            member.redeemPoints(pointsToRedeem, this);
        }
        if (member != null)
            earnedPoints = member.earnPoints(netAmount, this);
        time = LocalDateTime.now();
        confirmed = true;
        order.markPaid(this);
    }

    // เมธอดอ่านยอดชำระที่คำนวณแล้วเพื่อแสดงใบเสร็จและรายงาน

    public String receiptNo() {
        return receiptNo;
    }

    public Order order() {
        return order;
    }

    public Employee employee() {
        return receivedBy;
    }

    public String method() {
        if (method == null) return null;
        return method == PayMethod.CASH ? "เงินสด" : "QR";
    }

    public double received() {
        return received;
    }

    public double change() {
        return change;
    }

    public double discount() {
        return discount;
    }

```

```
    public double netAmount() {
        return netAmount;
    }

    public int pointsToRedeem() {
        return pointsToRedeem;
    }

    public int earnedPoints() {
        return earnedPoints;
    }

    public LocalDateTime time() {
        return time;
    }

    public boolean isConfirmed() {
        return confirmed;
    }

    // สร้างข้อความใบเสร็จหลังยืนยันการชำระ
    public String printReceipt() {
        if (!confirmed) throw new IllegalStateException("ต้องยืนยันการชำระก่อนพิมพ์ใบเสร็จ");
        return "ใบเสร็จ " + receiptNo + "\nออเดอร์ " + order.orderId()
                + "\nยอดสุทธิ " + String.format("%.2f", netAmount)
                + " บาท\nรับเงิน " + String.format("%.2f", received)
                + " บาท\nเงินทอน " + String.format("%.2f", change) + " บาท";
    }
}

```

# บทที่ 3: เจาะลึก Main.java ตัวควบคุมระบบและการทำงาน 8 Use Cases

## 3.1 สถาปัตยกรรมและบทบาทของ Main.java

ไฟล์ `src/Main.java` มีความยาว 1,445 บรรทัด ทำหน้าที่เป็นทั้ง **Presentation Layer (ส่วนติดต่อผู้ใช้แบบคอนโซล Text Mode)** และ **Application Orchestrator (ตัวควบคุมการไหลของระบบ)** ที่คอยรับคำสั่งจากแป้นพิมพ์ ส่งต่อให้อ็อบเจกต์ในแพ็กเกจ `model` ทำงาน และนำผลลัพธ์มาแสดงผล

เหตุผลที่ `Main.java` มีความยาวมาก เกิดจากการที่ระบบต้องรวมหน้าจอเมนูและลอจิกการรับคำตอบของ Use Cases ทั้ง 8 งาน (UC01–UC08) ไว้ในไฟล์เดียวตามข้อกำหนด โดยไม่สร้างคลาสสนับสนุนในโฟลเดอร์ภายนอก (ไม่มี `ui/`, `data/` หรือ `util/`) เพื่อให้ไฟล์โมเดลทั้ง 16 คลาสคงความบริสุทธิ์ของกฎทางธุรกิจอย่างสมบูรณ์

> **หลักการแบ่งหน้าที่ (Separation of Concerns):**
> **Main** สนใจเรื่อง *"ผู้ใช้เลือกเมนูใด และจะพิมพ์อะไรออกทางหน้าจอ"* ในขณะที่ **model** สนใจเรื่อง *"สต็อกมีพอไหม, เงินทอนเท่าไร, สถานะเปลี่ยนได้หรือไม่"* ทำให้โค้ดมีความเป็นโมดูลาร์สูง แม้ `Main` จะยาวแต่ก็อ่านง่ายเพราะแบ่งเป็นเมธอด `uc01()` ถึง `uc08()` ชัดเจน

## 3.2 ฐานข้อมูลจำลองส่วนกลาง (Shared In-Memory Collections)

ด้านบนของคลาส `Main` มีการประกาศตัวแปรส่วนกลางแบบ `static ArrayList` หลายชุด ซึ่งทำหน้าที่เสมือนฐานข้อมูลจำลอง (In-Memory Database) ที่ทุก Use Case ใช้งานร่วมกันตลอดเวลาที่โปรแกรมเปิดอยู่:

| ตัวแปรใน Main | ชนิดข้อมูล | หน้าที่และความสำคัญ |
|---|---|---|
| `tables` | `ArrayList<Table>` | เก็บโต๊ะอาหาร T01 ถึง T04 เพื่อใช้สแกนเปิดออเดอร์ใน UC01 และเคลียร์โต๊ะใน UC04 |
| `menus` | `ArrayList<Menu>` | เก็บเมนูอาหาร เช่น ผัดกะเพราปลา, ยำหมูยอ/แหนม สำหรับสั่งอาหารและคำนวณราคา |
| `ingredients` | `ArrayList<Ingredient>` | คลังวัตถุดิบกลาง (เนื้อปลา, ไข่ไก่, หมูยอ ฯลฯ) ที่ถูกตัดสต็อกใน UC01 และรับเข้าใน UC05 |
| `members` | `ArrayList<Member>` | ทะเบียนสมาชิกของร้าน สำหรับค้นหาด้วยเบอร์โทรใน UC01/UC08 และใช้แลกแต้มใน UC04 |
| `suppliers` | `ArrayList<Supplier>` | รายชื่อผู้จัดจำหน่ายวัตถุดิบ สำหรับการออกใบสั่งซื้อ PO ใน UC05 |
| `sops` | `ArrayList<SOP>` | คู่มือการปรุงอาหารมาตรฐาน สำหรับให้คนครัวดูเวลาและขั้นตอนใน UC03 |
| `orders` | `ArrayList<Order>` | ประวัติและคิวออเดอร์ทั้งหมดในร้าน ตั้งแต่ตะกร้า ปรุง เสิร์ฟ จนถึงชำระเงิน |
| `purchaseOrders` | `ArrayList<PurchaseOrder>` | รายการใบสั่งซื้อวัตถุดิบทั้งหมดของร้าน |
| `manager, cashier, cook` | `Employee` | พนักงานตัวแทน 3 บทบาทหลักที่ใช้แสดงและตรวจสอบสิทธิ์ในระบบ |

## 3.3 การสร้างข้อมูลตั้งต้น createSampleData()

เมื่อโปรแกรมเริ่มทำงาน เมธอด `main()` จะเรียก `createSampleData()` เป็นลำดับแรก เพื่อจำลองสภาพแวดล้อมจริงของร้านข้าวต้มอารมณ์ดี:

**รายละเอียดข้อมูลตัวอย่าง P4:**

- **โต๊ะอาหาร (4 โต๊ะ):** T01 (4 ที่นั่ง), T02 (4 ที่นั่ง), T03 (2 ที่นั่ง), T04 (6 ที่นั่ง)
- **วัตถุดิบเริ่มต้น:** เนื้อปลา 5.3 กก., ไข่ไก่ 3 ฟอง, หมูยอ 2 แท่ง, แหนม 2 แท่ง, น้ำยำ 2 ลิตร
- **เมนูอาหาร:** ผัดกะเพราปลา (69 บาท) พร้อมออปชันเพิ่มไข่ดาว (15 บาท), ยำหมูยอ/แหนม (65 บาท)
- **สมาชิกตัวอย่าง:** คุณอารมณ์ดี แจ่มใส (0812345678) แต้มสะสม 46 แต้ม

**src/Main.java → createSampleData()** — **Pre-seeded Business Data**

```
    static void createSampleData() {
        for (int i = 1; i <= 8; i++) {
            String tableId = String.format("TBL-%03d", i);
            tables.add(new Table(tableId, "QR-" + tableId, 4));
        }
        // ข้อมูลสาธิต P4: เมื่อสั่งกะเพราปลา 2 จานกับย่า 1 จาน ยอดเป็น 233 บาท
        Ingredient fish = addIngredient("เนื้อปลา", "กก.", 5.3, 2);
        Ingredient rice = addIngredient("ข้าวสาร", "กก.", 50.24, 20);
        Ingredient basil = addIngredient("ใบกะเพรา", "กำ", 6.4, 10);
        Ingredient egg = addIngredient("ไข่ไก่", "ฟอง", 3, 30);
        Ingredient oil = addIngredient("น้ำมัน", "ลิตร", 8.06, 3);
        Ingredient herb = addIngredient("มะดูมแห้ง", "กก.", 2, 1);
        Ingredient ham = addIngredient("หมูยอ", "กก.", 1, 0.5);
        Ingredient pork = addIngredient("หมูเนื้อแดง", "กก.", 5, 2);

        Menu porridge = new Menu("MNU-001", "ข้าวต้มปลา", 80, "ต้ม/นึ่ง", "ครัวร้อน");
        Recipe porridgeRecipe = new Recipe(newId("RCP"), 1);
        porridgeRecipe.addIngredient(fish, 0.15);
        porridgeRecipe.addIngredient(rice, 0.1);
        porridgeRecipe.complete();
        porridge.addRecipe(porridgeRecipe);
        porridge.openSale();
        menus.add(porridge);

        Menu garlic = new Menu("MNU-018", "หมูกระเทียม", 120, "ผัด", "ครัวร้อน");
        Recipe garlicRecipe = new Recipe(newId("RCP"), 1);
        garlicRecipe.addIngredient(pork, 0.2);
        garlicRecipe.addIngredient(oil, 0.02);
        garlicRecipe.complete();
        garlic.addRecipe(garlicRecipe);
        garlic.openSale();
        menus.add(garlic);

        Menu stir = new Menu("MNU-029", "ผัดกะเพราปลา", 69, "ผัด", "ครัวร้อน");
        stir.allowSpiceSelection(true);
        Recipe stirRecipe = new Recipe(newId("RCP"), 1);
        stirRecipe.addIngredient(fish, 0.15);
        stirRecipe.addIngredient(rice, 0.12);
        stirRecipe.addIngredient(basil, 0.2);
        stirRecipe.addIngredient(oil, 0.02);
        stirRecipe.complete();
        stir.addRecipe(stirRecipe);
        stir.addOption("เพิ่มไข่ดาว", 15);
        stir.addOptionIngredient(0, egg, 1);
        stir.addOptionIngredient(0, oil, 0.01);
        stir.addOption("เพิ่มข้าว", 10);
        stir.addOptionIngredient(1, rice, 0.12);
        stir.openSale();
        menus.add(stir);

        Menu yum = new Menu("MNU-007", "ย่าหมูยอ/แหนม", 65, "ย่า", "ครัวเย็น");
        yum.allowSpiceSelection(true);
        Recipe yumRecipe = new Recipe(newId("RCP"), 1);
        yumRecipe.addIngredient(ham, 0.1);
        yumRecipe.complete();
        yum.addRecipe(yumRecipe);
        yum.openSale();
        menus.add(yum);

        Menu drink = new Menu("MNU-033", "น้ำมะดูม", 45, "เครื่องดื่ม", "บาร์น้ำ");
        Recipe drinkRecipe = new Recipe(newId("RCP"), 1);
        drinkRecipe.addIngredient(herb, 0.05);
        drinkRecipe.complete();
        drink.addRecipe(drinkRecipe);
        drink.openSale();
        menus.add(drink);

        createSop(porridge, "เตรียมปลาและข้าว", 2, "ต้มและจัดชาม", 5);
        createSop(garlic, "เตรียมหมูและกระเทียม", 2, "ผัดและจัดจาน", 5);
        createSop(stir, "เตรียมวัตถุดิบ", 2, "ผัดและจัดจาน", 5);
        createSop(yum, "เตรียมหมูยอและน้ำย่า", 2, "คลุกและจัดจาน", 3);
```

```
        createSop(drink, "ต้มมะดูมและกรอง", 3, "เติมน้ำแข็งและเสิร์ฟ", 1);

        Supplier supplier = new Supplier(newId("SUP"), "ร้านวัตถุดิบชุมชน");
        for (int i = 0; i < ingredients.size(); i++)
            supplier.addIngredient(ingredients.get(i));
        suppliers.add(supplier);
        Member p4Member = new Member("MBR-0001", "Human", "0999999999");
        p4Member.earnPoints(2300, null);
        members.add(p4Member);
        Member member = new Member(newId("MBR"), "สมชาย", "0812345678");
        member.earnPoints(2300, null); // ข้อมูลประวัติเริ่มต้น: 2300 / 50 = 46 แต้ม
        members.add(member);
    }
```

## 3.4 เครื่องมือรับคำสั่งและดักจับความผิดพลาด (Input Helpers)

ใน `Main.java` มีเมธอดสำหรับอ่านและตรวจสอบอินพุตจากผู้ใช้ เพื่อป้องกันไม่ให้เกิด `InputMismatchException` หรือโปรแกรมแครชเมื่อผู้ใช้พิมพ์ข้อมูลผิด:

- `readText(String question)`: อ่านข้อความ 1 บรรทัด พร้อมตัดช่องว่าง (trim)
- `readRequired(String question)`: วนลูปถามซ้ำจนกว่าผู้ใช้จะพิมพ์ข้อความที่ไม่ใช่ค่าว่าง
- `readInt(String question, int min, int max)`: อ่านตัวเลขจำนวนเต็มในช่วง [min, max] หากพิมพ์ตัวอักษรจะพิมพ์แจ้งเตือนและให้กรอกใหม่
- `readNumber(String question, double min, double max)`: อ่านตัวเลขทศนิยมในช่วงที่กำหนด
- `confirm(String question)`: รับคำตอบ 1 (ใช่) หรือ 0 (ไม่ใช่) คืนค่าเป็น `boolean`
- `chooseMenu()`, `chooseIngredient()`, `chooseOrder()`: แสดงรายการอ็อบเจกต์เป็นเมนูตัวเลข 1, 2, 3... และแปลงค่าที่ผู้ใช้เลือกกลับเป็น Index ของ `ArrayList` (ใช้ `choice - 1`)

> **กลไก Global Exception Handling ใน main():**
> ในลูปเมนูหลักของ `main()` มีการครอบบล็อก `try-catch` รับ `IllegalArgumentException` และ `IllegalStateException` เอาไว้เสมอ เมื่อคลาสใน `model` ตรวจพบว่าสต็อกไม่พอหรือเงินไม่พอและโยน Exception ออกมา `Main` จะจับข้อความไปพิมพ์เตือนบนหน้าจออย่างสุภาพ เช่น *"ทำรายการไม่ได้: วัตถุดิบไม่พอ: เนื้อปลา"* โดยไม่ทำให้โปรแกรมปิดตัวลงกลางคัน

## 3.5 ผ่าโค้ดการทำงานของ Use Cases (uc01 ถึง uc08)

### 3.5.1 UC01: สั่งอาหารผ่าน QR โต๊ะและติดตามออเดอร์ (uc01 & addFood)

**ขั้นตอนการทำงาน:** สแกนโต๊ะ (`scanTableQRCode`) → สร้าง Customer → ค้นหาและผูกสมาชิก → สร้าง Order สถานะ CART → วนลูปเพิ่มอาหารและ Topping ผ่าน `addFood` → ยืนยันออเดอร์ `order.confirm()` ตรวจสต็อกครบทุกตัวแล้วตัดสต็อกอะตอมมิก → โต๊ะเปลี่ยนเป็น "ใช้งาน"

**src/Main.java → uc01() & addFood()** — **Order Placement Workflow**

```java
    static void uc01() {
        screen("ข้าวต้มอารมณ์ดี - สั่งอาหารผ่าน QR Code", "ลูกค้า - แคชเชียร์ - ผู้จัดการ");
        System.out.println("จำลองการสั่งอาหาร ปรับแต่งเมนู ส่งเข้าครัว และติดตามสถานะ");
        line();
        System.out.println("[1] สแกน QR Code และสั่งอาหารใหม่");
        System.out.println("[2] ติดตามสถานะและจัดการออเดอร์");
        System.out.println("[0] ย้อนกลับหน้าหลัก");
        int choice = readInt("เลือกงานสั่งอาหาร", 0, 2);
        if (choice == 0)
            return;
        if (choice == 2) {
            manageOrder();
            return;
        }
        for (int i = 0; i < tables.size(); i++) {
            System.out.println("[" + (i + 1) + "] " + tables.get(i).tableId() + " - " + tables.get(i).statusTex
t());
        }
        int tableChoice = readInt("เลือก QR ของโต๊ะ (0 กลับ)", 0, tables.size());
        if (tableChoice == 0)
            return;
        Table table = tables.get(tableChoice - 1);
        table.scanTableQRCode();
        screen("สแกน QR Code ประจำโต๊ะ", "ลูกค้า");
        System.out.println("ระบบระบุโต๊ะ " + table.identifyTableNumber() + " จาก " + table.qrCode());
        screen("สิทธิพิเศษสมาชิกและสะสมแต้ม", "ลูกค้า");
        System.out.println(
                "ทุก " + Member.bahtPerPoint() + " บาท = 1 แต้ม - 1 แต้ม = " + Member.pointValue() + " บาท");
        Customer customer = new Customer(newId("CUS"));
        String phone = readText("เบอร์สมาชิก (Enter ข้าม)");
        if (!phone.isEmpty()) {
            Member member = findMember(phone);
            if (member == null)
                System.out.println("ไม่พบสมาชิก บิลนี้ยังไม่ผูกสมาชิก");
            else {
                customer.linkMember(member);
                System.out.println("ระบบผูกเบอร์ " + phone + " (" + member.name() + " - "
                        + member.memberId() + ") กับออเดอร์เพื่อสะสมแต้มตอนชำระเงิน");
            }
        }
        Order order = new Order(newOrderId(), table, customer);
        System.out.println("สร้างตะกร้า " + cartId(order) + " ให้อัตโนมัติ");
        boolean editing = true;
        while (editing) {
            showOrder(order);
            System.out.println("[1] เลือกหมวดและเพิ่มอาหาร");
            System.out.println("[2] แก้จำนวนจาน");
            System.out.println("[3] ลบรายการ");
            System.out.println("[4] ยืนยันส่งออเดอร์เข้าครัว");
            System.out.println("[0] ทิ้งตะกร้าและกลับหน้าหลัก");
            int action = readInt("เลือกตัวเลือกในตะกร้า", 0, 4);
            if (action == 0) {
                order.destroy();
                return;
            }
            if (action == 1)
                addFood(order);
            else if (action == 2) {
                int index = chooseItem(order);
                if (index >= 0)
                    order.changeQuantity(index, readInt("จำนวนใหม่", 1, 100));
            } else if (action == 3) {
                int index = chooseItem(order);
                if (index >= 0)
                    order.removeItem(index);
            } else if (action == 4) {
                // ถ้าสต็อกไม่พอ ให้กลับมาแก้ตะกร้าเดิมได้
                try {
                    screen("ยืนยันการสั่งอาหารเข้าครัว", "ลูกค้า");
                    System.out.printf("สรุปรายการ %d รายการ รวม %.2f บาท%n", order.items().size(),
                            order.calculateTotal());
```

```
                    order.confirm();
                    orders.add(order);
                    System.out.println("วัตถุดิบเพียงพอ - ส่งครัวแล้ว " + order.orderId());
                    System.out.println("ตัดสต็อกตามสูตรทันที - สต็อกคงเหลือ:");
                    for (int i = 0; i < ingredients.size(); i++) {
                        Ingredient ingredient = ingredients.get(i);
                        System.out.printf("%s = %.3f %s%n", ingredient.name(),
                                ingredient.stockQty(), ingredient.unit());
                    }
                    int hot = 0;
                    int cold = 0;
                    int bar = 0;
                    for (int i = 0; i < order.items().size(); i++) {
                        String station = order.items().get(i).menu().station();
                        if (station.equals("ครัวร้อน"))
                            hot++;
                        else if (station.equals("ครัวเย็น"))
                            cold++;
                        else if (station.equals("บาร์น้ำ"))
                            bar++;
                    }
                    System.out.println("ส่งรายการเข้าจอ KDS: ครัวร้อน " + hot + " - ครัวเย็น "
                            + cold + " - บาร์น้ำ " + bar + " รายการ");
                    showOrder(order);
                    editing = false;
                } catch (IllegalArgumentException e) {
                    System.out.println(e.getMessage());
                } catch (IllegalStateException e) {
                    System.out.println(e.getMessage());
                }
            }
        }
    }
```

**src/Main.java → addFood()** — **Add Item & Option Logic**

```java
    static void addFood(Order order) {
        Menu menu = chooseMenuForOrder(order);
        if (menu == null)
            return;
        if (!menu.statusText().equals("เปิดขาย")) {
            System.out.println("เมนูนี้ปิดขาย");
            return;
        }
        screen("ปรับแต่งเมนูอาหาร", "ลูกค้า");
        System.out.printf("%s %s - %s - ราคา %.2f บาท%n", menu.menuId(), menu.name(), menu.station(),
                menu.price());
        int quantity = readInt("จำนวนจาน", 1, 100);
        String spice = "ไม่ระบุ";
        if (menu.isSpiceSelectable()) {
            System.out.println("ระดับความเผ็ด: [1] ไม่เผ็ด [2] เผ็ดน้อย [3] เผ็ดปกติ [4] เผ็ดมาก");
            String[] levels = { "ไม่เผ็ด", "เผ็ดน้อย", "เผ็ดปกติ", "เผ็ดมาก" };
            spice = levels[readInt("ระดับความเผ็ด [1-4]", 1, 4) - 1];
        }
        for (int i = 0; i < menu.optionNames().size(); i++) {
            System.out.printf("[%d] %s +%.2f บาท%n", i + 1, menu.optionNames().get(i), menu.optionPrice(i));
        }
        String options = "0";
        if (!menu.optionNames().isEmpty())
            options = readText("ตัวเลือกเพิ่มเติม (เลขหลายอันคั่นด้วย , หรือ 0)");
        String note = readText("หมายเหตุ (Enter ไม่มี)");
        OrderItem item = order.addItem(menu, quantity, spice, note);
        String[] chosen = options.split(",");
        for (int i = 0; i < chosen.length; i++) {
            try {
                int optionNumber = Integer.parseInt(chosen[i].trim());
                if (optionNumber >= 1 && optionNumber <= menu.optionNames().size())
                    item.addOption(optionNumber - 1);
            } catch (NumberFormatException e) {
                System.out.println("ข้ามตัวเลือกที่ไม่ใช่ตัวเลข: " + chosen[i]);
            }
        }
        System.out.printf("ราคารายการ: %.2f บาท%n", item.price());
        if (!confirm("ใส่ตะกร้าหรือไม่")) {
            order.removeItem(order.items().size() - 1);
            return;
        }
        System.out.println("เพิ่มลงตะกร้าแล้ว (" + order.items().size() + " รายการ)");
    }
```

### 3.5.2 UC02: จัดการคู่มือ SOP (uc02)

**ขั้นตอนการทำงาน:** สร้าง SOP ใหม่ ผูกกับเมนู → เพิ่มขั้นตอนย่อยทีละข้อพร้อมเวลามาตรฐาน → คำนวณเวลารวมด้วย `getTotalMinutes()` → เผยแพร่คู่มือ `publish()` เพื่อให้ครัวนำไปใช้ หรือปิดใช้ `disable()`

src/Main.java → uc02() SOP Management

```java
    static void uc02() {
        screen("ข้าวต้มอารมณ์ดี - จัดการ SOP (Standard Operating Procedure)", "ผู้จัดการ");
        System.out.println("[1] ดูรายการ SOP ทั้งหมด");
        System.out.println("[2] สร้าง SOP ใหม่และกำหนดขั้นตอน");
        System.out.println("[3] แก้ไข SOP ฉบับร่าง");
        System.out.println("[4] เผยแพร่ SOP ให้ครัวใช้");
        System.out.println("[5] แก้ไข SOP ที่เผยแพร่แล้ว");
        System.out.println("[6] ปิดใช้งานหรือเปิดใช้งาน SOP อีกครั้ง");
        System.out.println("[7] ลบ SOP ฉบับร่าง");
        System.out.println("[0] ย้อนกลับหน้าหลัก");
        int action = readInt("เลือกงาน SOP", 0, 7);
        if (action == 0)
            return;
        if (action == 1) {
            screen("รายการคู่มือขั้นตอนการปรุงอาหาร (SOP ทั้งหมด)", "ผู้จัดการ");
            for (int i = 0; i < sops.size(); i++) {
                SOP sop = sops.get(i);
                System.out.println(sop.name() + " - " + sop.menu().name() + " - " + sop.statusText());
            }
            return;
        }
        if (action == 2) {
            Menu menu = chooseMenu();
            if (menu == null)
                return;
            if (findSop(menu) != null) {
                System.out.println("เมนูนี้มี SOP แล้ว");
                return;
            }
            SOP sop = new SOP(newId("SOP"), readRequired("ชื่อ SOP"), menu);
            do {
                sop.addStep(readRequired("รายละเอียดขั้นตอน"), readNumber("เวลามาตรฐาน (นาที)", 0.01, 1440));
            } while (confirm("เพิ่มอีกขั้นตอนหรือไม่"));
            if (confirm("บันทึกร่างหรือไม่"))
                sops.add(sop);
            else
                sop.destroy();
            return;
        }
        SOP sop = chooseSop();
        if (sop == null)
            return;
        if (action == 3 || action == 5) {
            if (action == 3 && !sop.statusText().equals("ร่าง")) {
                System.out.println("เมนูนี้ไม่ใช่ SOP ฉบับร่าง");
                return;
            }
            if (action == 5 && !sop.statusText().equals("เผยแพร่")) {
                System.out.println("เมนูนี้ยังไม่เผยแพร่");
                return;
            }
            showSop(sop);
            if (confirm("แก้ไขหรือไม่")) {
                if (isCooking(sop.menu())) {
                    System.out.println("ยังมีอาหารของเมนูนี้กำลังปรุง");
                    return;
                }
                String name = readRequired("ชื่อ SOP ใหม่");
                int step = readInt("ขั้นที่จะแก้ (0 เพิ่มขั้นตอน)", 0, sop.steps().size());
                String description = readRequired("คำอธิบาย");
                double minutes = readNumber("เวลามาตรฐาน (นาที)", 0.01, 1440);
                if (confirm("บันทึกหรือไม่")) {
                    sop.update(name);
                    if (step == 0)
                        sop.addStep(description, minutes);
                    else
                        sop.steps().get(step - 1).update(description, minutes);
                }
            }
        } else if (action == 4)
            sop.publish();
```

```
        else if (action == 6) {
            if (sop.statusText().equals("เผยแพร่")) {
                if (isCooking(sop.menu())) {
                    System.out.println("กำลังมีอาหารปรุงตามคู่มือนี้");
                    return;
                }
                sop.disable();
            } else if (sop.statusText().equals("ปิดใช้"))
                sop.enable();
            else
                System.out.println("ฉบับร่างยังปิดใช้งานไม่ได้");
        } else if (action == 7) {
            if (!sop.isDeletable()) {
                System.out.println("ลบได้เฉพาะร่าง ให้ปิดใช้แทน");
                return;
            }
            if (confirm("ลบคู่มือและขั้นตอนทั้งหมดหรือไม่")) {
                sop.destroy();
                sops.remove(sop);
                return;
            }
        }
        showSop(sop);
    }
```

### 3.5.3 UC03: ปรุงอาหารและเสิร์ฟ (uc03)

**ขั้นตอนการทำงาน:** คนครัวเลือกสถานีครัว → ดึงคิว OrderItem → เรียก `item.startCooking()` บันทึกเวลาเริ่ม → เมื่อทำเสร็จ เรียก `item.markReady()` บันทึกเวลาจบและคำนวณเปรียบเทียบกับ SOP → บริกรเรียก `item.serve()` เปลี่ยนเป็นเสิร์ฟแล้ว → สั่ง `order.refreshStatus()`

src/Main.java → uc03() Kitchen & Serving Workflow

```java
    static void uc03() {
        screen("ข้าวต้มอารมณ์ดี - ปรุงอาหารตาม SOP และงานเสิร์ฟ", "คนครัว - แคชเชียร์");
        System.out.println("คนครัวเลือกสถานี ดูคิว ปรุงตาม SOP และส่งอาหารไปเสิร์ฟ");
        line();
        System.out.println("[1] เลือกสถานีและดูคิวออเดอร์");
        System.out.println("[2] เริ่มปรุงอาหารตาม SOP");
        System.out.println("[3] บันทึกปรุงเสร็จและเทียบเวลา");
        System.out.println("[4] ดูรายการอาหารพร้อมเสิร์ฟ");
        System.out.println("[5] ยืนยันเสิร์ฟอาหารถึงโต๊ะ");
        System.out.println("[0] ย้อนกลับหน้าหลัก");
        int action = readInt("เลือกงานครัวและเสิร์ฟ", 0, 5);
        if (action == 0)
            return;
        String station = "";
        if (action <= 3) {
            station = chooseStation();
            cook.assignStationDuty(station);
            screen("จอครัว KDS - " + station, "คนครัว");
        } else
            screen("รายการอาหารพร้อมเสิร์ฟ", "แคชเชียร์");
        ArrayList<Order> queueOrders = new ArrayList<Order>();
        ArrayList<OrderItem> queueItems = new ArrayList<OrderItem>();
        for (int i = 0; i < orders.size(); i++) {
            Order order = orders.get(i);
            for (int j = 0; j < order.items().size(); j++) {
                OrderItem item = order.items().get(j);
                boolean matches = false;
                if (action == 1 && item.menu().station().equals(station)
                        && !item.statusText().equals("ยกเลิก") && !item.statusText().equals("เสิร์ฟแล้ว"))
                    matches = true;
                else if (action == 2 && item.menu().station().equals(station)
                        && item.statusText().equals("รอปรุง"))
                    matches = true;
                else if (action == 3 && item.menu().station().equals(station)
                        && item.statusText().equals("กำลังปรุง"))
                    matches = true;
                else if (action >= 4 && item.statusText().equals("พร้อมเสิร์ฟ"))
                    matches = true;
                if (matches) {
                    queueOrders.add(order);
                    queueItems.add(item);
                    System.out.println(
                            "[" + queueItems.size() + "] " + order.orderId() + " - " + item.menu().name()
                                    + " x" + item.quantity() + " - " + displayStatus(item.statusText()));
                }
            }
        }
        if (queueItems.isEmpty())
            System.out.println("ไม่มีรายการในขั้นตอนนี้");
        if (action == 1 || action == 4)
            return;
        int choice = readInt("เลือกรายการครัว (0 กลับ)", 0, queueItems.size());
        if (choice == 0)
            return;
        Order order = queueOrders.get(choice - 1);
        OrderItem item = queueItems.get(choice - 1);
        System.out.println(
                "เผ็ด: " + item.spice() + " ตัวเลือก: " + item.optionNames() + " หมายเหตุ: " + item.note());
        if (action == 2) {
            SOP sop = findSop(item.menu());
            if (sop == null || !sop.statusText().equals("เผยแพร่")) {
                System.out.println("ยังไม่มี SOP เผยแพร่ เริ่มปรุงไม่ได้");
                return;
            }
            showSop(sop);
            if (confirm("เริ่มปรุงหรือไม่"))
                item.startCooking(cook, sop.getTotalMinutes());
        } else if (action == 3) {
            if (confirm("ปรุงเสร็จแล้วหรือไม่")) {
                item.markReady();
                System.out.printf("ใช้จริง %.2f นาที / มาตรฐาน %.2f นาที%n", item.cookingMinutes(),
```

```
                        item.standardMinutes());
                if (item.cookingMinutes() > item.standardMinutes())
                    System.out.println("เกินเวลามาตรฐาน");
            }
        } else if (action == 5) {
            if (confirm("แคชเชียร์นำไปเสิร์ฟแล้วหรือไม่"))
                item.serve(cashier);
        }
        order.refreshStatus();
        showOrder(order);
    }
```

### 3.5.4 UC04: ชำระเงินและออกใบเสร็จ (uc04)

**ขั้นตอนการทำงาน:** แสดงออเดอร์ที่เสิร์ฟครบทุกจานแล้ว (`isServed() == true`) → ดึงแต้มสมาชิกมาแสดง → สมาชิกขอแลกแต้มส่วนลด (ขั้นต่ำ 10 แต้ม) → เลือกจ่ายเงินสดหรือ QR → เรียก `payment.pay(...)` เตรียมบิล → กดยืนยัน `payment.confirm()` หักแต้มที่ใช้ แจกแต้มใหม่ ปิดออเดอร์ → พิมพ์ใบเสร็จ → ปล่อยโต๊ะว่าง

**src/Main.java → uc04()** — **Settlement & Receipt Workflow**

```java
    static void uc04() {
        screen("ข้าวต้มอารมณ์ดี - จุดชำระเงิน POS (Point of Sale)", "แคชเชียร์");
        System.out.println("เลือกโต๊ะรอชำระ ตรวจรายการ แลกแต้ม รับเงิน และออกใบเสร็จ");
        line();
        System.out.println("[1] เลือกออเดอร์ที่ต้องการชำระเงิน");
        System.out.println("[2] ตรวจสอบยอดรวมและสมาชิก");
        System.out.println("[3] ระบุแต้มที่ใช้แลกส่วนลด");
        System.out.println("[4] เลือกช่องทางชำระเงิน");
        System.out.println("[5] บันทึกการชำระและแต้ม");
        System.out.println("[6] แสดงใบเสร็จรับเงิน");
        System.out.println("[0] ย้อนกลับหน้าหลัก");
        if (readInt("เลือก 1 เพื่อเริ่มชำระเงิน", 0, 1) == 0)
            return;
        screen("เคาน์เตอร์แคชเชียร์: เรียกดูโต๊ะรอชำระเงิน", "แคชเชียร์");
        ArrayList<Order> unpaid = new ArrayList<Order>();
        for (int i = 0; i < orders.size(); i++) {
            Order order = orders.get(i);
            if (order.isServed() && order.payment() == null) {
                unpaid.add(order);
                System.out.printf("[%d] %s - โต๊ะ %s - %.2f บาท%n", unpaid.size(), order.orderId(),
                        order.table().tableId(), order.calculateTotal());
            }
        }
        int choice = readInt("เลือกบิลรอชำระ (0 กลับ)", 0, unpaid.size());
        if (choice == 0)
            return;
        Order order = unpaid.get(choice - 1);
        screen("รายละเอียดบิลค่าอาหาร", "แคชเชียร์");
        showOrder(order);
        screen("ตรวจสอบสมาชิกเพื่อสะสมหรือแลกแต้ม", "แคชเชียร์");
        String phone = readText("เบอร์สมาชิก (Enter ใช้สมาชิกเดิมถ้ามี / 0 ไม่ผูกสมาชิก)");
        if (phone.equals("0"))
            order.customer().linkMember(null);
        else if (!phone.isEmpty()) {
            Member member = findMember(phone);
            if (member == null) {
                System.out.println("ไม่พบสมาชิก");
                return;
            }
            order.customer().linkMember(member);
        }
        int points = 0;
        if (order.linkedMember() != null) {
            showMember(order.linkedMember());
            points = readInt("ใช้กี่แต้ม (0 ไม่ใช้)", 0, order.linkedMember().points());
        }
        Payment payment = new Payment(newPaymentId(), order, cashier);
        screen("เลือกช่องทางการชำระเงิน", "แคชเชียร์");
        int channel = readInt("1 เงินสด / 2 QR (จำลอง) / 0 ยกเลิก", 0, 2);
        if (channel == 0)
            return;
        if (channel == 1)
            payment.pay("เงินสด", readNumber("เงินที่รับ", 0, 10000000), points);
        else
            payment.pay("QR", 0, points);
        System.out.printf("สุทธิ %.2f / ส่วนลด %.2f / เงินทอน %.2f%n", payment.netAmount(), payment.discount(),
                payment.change());
        if (confirm("ยืนยันการชำระหรือไม่")) {
            payment.confirm();
            releaseTable(order.table());
            printReceipt(payment);
        }
    }
```

### 3.5.5 UC05: สต็อกและใบสั่งซื้อ (uc05)

**ขั้นตอนการทำงาน:** ตรวจสอบยอดสต็อกคงเหลือ แจ้งเตือนเมื่อต่ำกว่า `minQty` → ออกใบสั่งซื้อ PO เลือกซัพพลายเออร์ → เมื่อสินค้ามาส่ง เรียก `po.receive(qtys)` รับของเข้าคลังและบันทึก StockMovement อัตโนมัติ หรือปรับยอดตรวจนับด้วย `adjustStock`

**src/Main.java → uc05()** — **Inventory & PO Workflow**

```java
    static void uc05() {
        screen("ข้าวต้มอารมณ์ดี - คลังวัตถุดิบและจัดซื้อ (Inventory & Procurement)", "ผู้จัดการ");
        System.out.println("[1] ตรวจสอบคลังวัตถุดิบและแจ้งเตือนของใกล้หมด");
        System.out.println("[2] สร้างใบสั่งซื้อ PO");
        System.out.println("[3] ตรวจรับวัตถุดิบตาม PO");
        System.out.println("[4] ปรับยอดสต็อกจากการนับจริง");
        System.out.println("[5] เพิ่มวัตถุดิบใหม่");
        System.out.println("[6] แก้ข้อมูลวัตถุดิบและเกณฑ์ขั้นต่ำ");
        System.out.println("[7] ดูประวัติความเคลื่อนไหวสต็อก");
        System.out.println("[0] ย้อนกลับหน้าหลัก");
        int action = readInt("เลือกงานคลังวัตถุดิบ", 0, 7);
        if (action == 0)
            return;
        if (action == 1) {
            showStock();
            int low = 0;
            for (int i = 0; i < ingredients.size(); i++) {
                if (ingredients.get(i).stockQty() < ingredients.get(i).minQty())
                    low++;
            }
            System.out.println("พบวัตถุดิบ " + low + " รายการต่ำกว่าจุดสั่งซื้อขั้นต่ำ");
        } else if (action == 2)
            createPurchaseOrder();
        else if (action == 3)
            receivePurchaseOrder();
        else if (action == 5) {
            String name = readRequired("ชื่อวัตถุดิบ");
            String unit = readRequired("หน่วย");
            double minimum = readNumber("ขั้นต่ำ", 0, 100000);
            Ingredient ingredient = new Ingredient(newId("ING"), name, unit, minimum);
            ingredients.add(ingredient);
            for (int i = 0; i < suppliers.size(); i++) {
                if (confirm("ให้ " + suppliers.get(i).name() + " ส่งวัตถุดิบนี้หรือไม่"))
                    suppliers.get(i).addIngredient(ingredient);
            }
            System.out.println("เพิ่มแล้ว ยอดเริ่มต้น 0 รับเพิ่มผ่าน PO");
        } else {
            Ingredient ingredient = chooseIngredient();
            if (ingredient == null)
                return;
            if (action == 4) {
                double actual = readNumber("ยอดที่นับจริง", 0, 100000);
                String reason = readRequired("เหตุผล");
                if (confirm("ปรับยอดหรือไม่"))
                    ingredient.adjustStock(actual, reason, manager);
            } else if (action == 6) {
                String name = readRequired("ชื่อใหม่");
                String unit = readRequired("หน่วย");
                double minimum = readNumber("ขั้นต่ำ", 0, 100000);
                String reason = readRequired("เหตุผลการแก้ไข");
                if (confirm("บันทึกหรือไม่")) {
                    ingredient.update(name, unit, minimum);
                    System.out.println("บันทึกแล้ว: " + reason);
                }
            } else if (action == 7) {
                for (int i = 0; i < ingredient.movements().size(); i++) {
                    StockMovement movement = ingredient.movements().get(i);
                    String employee = "ระบบ";
                    if (movement.employee() != null)
                        employee = movement.employee().name();
                    System.out.println(movement.time() + " - " + movement.typeText() + " - " + movement.quantit
y()
                            + " - เหลือ " + movement.balance() + " - " + movement.reference() + " - "
                            + movement.reason() + " - " + employee);
                }
            }
        }
    }
```

### 3.5.6 UC06: ภาพรวมและรายงานยอดขาย (uc06 & printSales)

**ขั้นตอนการทำงาน:** รับช่วงวันที่เริ่มต้นและสิ้นสุด → วนลูปอ่าน Order ทั้งหมด กรองเฉพาะบิลที่จ่ายเงินแล้ว → รวมยอดสุทธิ แยกเงินสดและ QR → นับจำนวนจานที่ขายได้ของแต่ละเมนู → จัดอันดับเมนูยอดนิยมด้วย Simple Sort Algorithm

**src/Main.java → uc06() & printSales()** — **Sales Reporting & Ranking**

```
    static void uc06() {
        screen("ข้าวต้มอารมณ์ดี - แดชบอร์ดและรายงานวิเคราะห์ยอดขาย", "ผู้จัดการ");
        System.out.println("[1] ดูภาพรวมยอดขายวันนี้");
        System.out.println("[2] ดูรายงานยอดขายตามช่วงวันที่");
        System.out.println("[3] ดูประวัติออเดอร์และรายละเอียดบิล");
        System.out.println("[0] ย้อนกลับหน้าหลัก");
        int action = readInt("เลือกงานรายงาน", 0, 3);
        if (action == 0)
            return;
        LocalDate today = LocalDate.now();
        if (action == 1) {
            screen("ผู้จัดการ: แผงควบคุมภาพรวมร้าน", "ผู้จัดการ");
            System.out.println("วันที่ " + today);
            printSales(today, today);
            showStock();
        } else if (action == 2) {
            screen("รายงานยอดขายและอันดับเมนูอาหาร", "ผู้จัดการ");
            LocalDate start = readDate("วันเริ่มต้น");
            LocalDate end = readDate("วันสิ้นสุด");
            if (start.isAfter(end)) {
                System.out.println("วันเริ่มต้องไม่เกินวันสิ้นสุด");
                return;
            }
            printSales(start, end);
        } else {
            screen("ประวัติออเดอร์และสถานะบิล", "ผู้จัดการ");
            Order order = chooseOrder();
            if (order != null) {
                showOrder(order);
                if (order.payment() != null)
                    printReceipt(order.payment());
            }
        }
    }
```

**src/Main.java → printSales()** — **Sales Aggregation Logic**

```java
    static void printSales(LocalDate start, LocalDate end) {
        double total = 0;
        double cash = 0;
        double qr = 0;
        int paid = 0;
        int created = 0;
        int cookingCount = 0;
        // แต่ละตำแหน่งตรงกับเมนูใน menus เก็บจำนวนจานที่ขายในช่วงวันที่
        int[] sold = new int[menus.size()];
        for (int i = 0; i < orders.size(); i++) {
            Order order = orders.get(i);
            LocalDate createdDate = order.createdAt().toLocalDate();
            if (inRange(createdDate, start, end)) {
                created++;
                if (order.statusText().equals("กำลังปรุง"))
                    cookingCount++;
            }
            Payment payment = order.payment();
            if (payment != null && inRange(payment.time().toLocalDate(), start, end)) {
                paid++;
                total += payment.netAmount();
                if (payment.method().equals("เงินสด"))
                    cash += payment.netAmount();
                else
                    qr += payment.netAmount();
                System.out.printf("%s - %s - %.2f บาท%n", payment.time(), order.orderId(),
                        payment.netAmount());
                for (int j = 0; j < order.items().size(); j++) {
                    OrderItem item = order.items().get(j);
                    if (!item.statusText().equals("ยกเลิก")) {
                        int index = menus.indexOf(item.menu());
                        if (index >= 0)
                            sold[index] += item.quantity();
                    }
                }
            }
        }
        System.out.println("ช่วง " + start + " ถึง " + end);
        System.out.println("ออเดอร์ที่สร้าง " + created + " / กำลังปรุง " + cookingCount + " / ชำระในช่วงนี้ " + paid);
        System.out.printf("ยอดสุทธิ %.2f - เงินสด %.2f - QR %.2f บาท%n", total, cash, qr);
        // เรียงดัชนีด้วยลูปพื้นฐาน เพื่อเก็บลำดับ menus จริงไว้ตามเดิม
        int[] indexes = new int[menus.size()];
        for (int i = 0; i < indexes.length; i++)
            indexes[i] = i;
        for (int i = 0; i < indexes.length; i++) {
            for (int j = i + 1; j < indexes.length; j++) {
                if (sold[indexes[j]] > sold[indexes[i]]) {
                    int temporary = indexes[i];
                    indexes[i] = indexes[j];
                    indexes[j] = temporary;
                }
            }
        }
        System.out.println("อันดับขายดี (จำนวนจาน)");
        for (int i = 0; i < indexes.length && i < 3; i++) {
            System.out.println((i + 1) + " " + menus.get(indexes[i]).name() + " = " + sold[indexes[i]]);
        }
        System.out.println("อันดับขายต่ำ (จำนวนจาน)");
        for (int i = indexes.length - 1; i >= 0 && i >= indexes.length - 3; i--) {
            System.out.println(menus.get(indexes[i]).name() + " = " + sold[indexes[i]]);
        }
    }
```

### 3.5.7 UC07: จัดการเมนูและสูตรอาหาร (uc07)

**ขั้นตอนการทำงาน:** สร้างเมนูใหม่ (สถานะเริ่ม OFF_SALE) → สร้างสูตรฉบับร่าง DRAFT ผูกวัตถุดิบและปริมาณ → เรียก `complete()` ยืนยันสูตรสมบูรณ์ → สั่ง `openSale()` เปิดขาย → แตกเวอร์ชันสูตรใหม่ `createNewVersion()` → เพิ่ม Topping ตัวเลือกเสริม

**src/Main.java → uc07()** — **Menu Engineering Workflow**

```java
    static void uc07() {
        screen("ข้าวต้มอารมณ์ดี - จัดการเมนูและสูตรอาหาร (Menu & Recipe Management)", "ผู้จัดการ");
        System.out.println("เพิ่มเมนู ผูกสูตร ตรวจสอบ SOP และควบคุมสถานะการขาย");
        line();
        System.out.println("[1] ดูรายการเมนูทั้งหมด");
        System.out.println("[2] เพิ่มเมนูอาหารใหม่");
        System.out.println("[3] กำหนดสูตรอาหารหรือสร้างเวอร์ชันใหม่");
        System.out.println("[4] เปิดขายเมนู");
        System.out.println("[5] แก้ไขข้อมูลเมนูหรือปิดขายชั่วคราว");
        System.out.println("[6] ปรับปรุงสูตรอาหารเดิมเป็นเวอร์ชันใหม่");
        System.out.println("[7] ลบเมนูที่ไม่มีประวัติสั่งซื้อ");
        System.out.println("[8] กำหนดตัวเลือกเพิ่มเติม");
        System.out.println("[0] ย้อนกลับหน้าหลัก");
        int action = readInt("เลือกงานเมนูและสูตร", 0, 8);
        if (action == 0)
            return;
        if (action == 1) {
            chooseMenu();
            return;
        }
        if (action == 2) {
            String name = readRequired("ชื่อเมนู");
            double price = readNumber("ราคา", 0.01, 100000);
            String category = readRequired("หมวด");
            Menu menu = new Menu(newId("MNU"), name, price, category, chooseStation());
            menu.allowSpiceSelection(confirm("ให้เลือกความเผ็ดหรือไม่"));
            menus.add(menu);
            System.out.println("สร้างแล้ว ยังปิดขายจนกว่าจะมีสูตรสมบูรณ์");
            return;
        }
        Menu menu = chooseMenu();
        if (menu == null)
            return;
        if (action == 3 || action == 6)
            manageRecipe(menu);
        else if (action == 4)
            menu.openSale();
        else if (action == 5) {
            int edit = readInt("1 แก้ข้อมูล / 2 ปิดขายชั่วคราว", 1, 2);
            if (edit == 1) {
                String name = readRequired("ชื่อเมนูใหม่");
                double price = readNumber("ราคา", 0.01, 100000);
                String category = readRequired("หมวด");
                String station = chooseStation();
                if (confirm("บันทึกหรือไม่"))
                    menu.update(name, price, category, station);
            } else
                menu.closeSale();
        } else if (action == 7) {
            if (!menu.isDeletable()) {
                System.out.println("เคยยืนยันสั่งเมนูนี้แล้ว ให้ปิดขายแทน");
                return;
            }
            if (confirm("ลบเมนูและสูตรทั้งหมดหรือไม่")) {
                SOP sop = findSop(menu);
                if (sop != null) {
                    sop.destroy();
                    sops.remove(sop);
                }
                menu.destroy();
                menus.remove(menu);
            }
        } else if (action == 8) {
            String name = readRequired("ชื่อตัวเลือก เช่น เพิ่มไข่ดาว");
            double price = readNumber("ราคาเพิ่มต่อจาน", 0, 100000);
            menu.addOption(name, price);
            int index = menu.optionNames().size() - 1;
            while (confirm("ตัวเลือกนี้ใช้วัตถุดิบเพิ่มหรือไม่")) {
                Ingredient ingredient = chooseIngredient();
                if (ingredient == null)
                    break;
```

```
                menu.addOptionIngredient(index, ingredient, readNumber("ปริมาณเพิ่มต่อจาน", 0.001, 1000));
            }
        }
    }
```

### 3.5.8 UC08: สมาชิกและแต้มสะสม (uc08)

**ขั้นตอนการทำงาน:** สมัครสมาชิกใหม่ด้วยเบอร์โทร 10 หลัก → ค้นหาและดูประวัติ Statement แต้ม → ผู้จัดการพิจารณาคำขอยกเลิกอาหารที่กำลังปรุงจากครัว (`approveRequests`) อนุมัติหรือปฏิเสธ → ผู้จัดการปรับเปลี่ยนกฎแต้มกลางของร้าน (`Member.setRule`)

**src/Main.java → uc08()** — **Member & Manager Approval Workflow**

```
    static void uc08() {
        screen("ข้าวต้มอารมณ์ดี - สมาชิกและระบบสะสมแต้ม (Loyalty Program)", "แคชเชียร์ - ผู้จัดการ");
        System.out.println("กฎ: ทุก " + Member.bahtPerPoint() + " บาท = 1 แต้ม - 1 แต้ม = " + Member.pointValue()
                + " บาท - ขั้นต่ำ " + Member.minimumRedeem());
        System.out.println("[1] ค้นหาหรือสมัครสมาชิกใหม่");
        System.out.println("[2] จำลองการสะสมและแลกแต้มที่หน้าชำระเงิน");
        System.out.println("[3] ดูประวัติความเคลื่อนไหวแต้ม");
        System.out.println("[4] แก้ไขข้อมูลสมาชิก");
        System.out.println("[5] ผู้จัดการดูรายชื่อและกำหนดกฎแต้ม");
        System.out.println("[6] ผู้จัดการพิจารณาคำขออาหาร");
        System.out.println("[0] ย้อนกลับหน้าหลัก");
        int action = readInt("เลือกงานสมาชิก", 0, 6);
        if (action == 0)
            return;
        if (action == 1) {
            screen("เคาน์เตอร์แคชเชียร์: ค้นหาหรือสมัครสมาชิกใหม่", "แคชเชียร์");
            String phone = readRequired("เบอร์โทร 10 หลัก");
            Member existing = findMember(phone);
            if (existing != null) {
                showMember(existing);
                return;
            }
            Member member = new Member(newId("MBR"), readRequired("ชื่อสมาชิก"), phone);
            if (confirm("ยืนยันสมัครสมาชิกหรือไม่")) {
                members.add(member);
                System.out.println("สมัครสมาชิกสำเร็จ");
                showMember(member);
            }
        } else if (action == 2) {
            screen("จำลองการสะสมและแลกแต้มที่หน้าชำระเงิน", "แคชเชียร์");
            Member member = findMember(readRequired("เบอร์โทร"));
            if (member == null) {
                System.out.println("ไม่พบสมาชิก");
                return;
            }
            showMember(member);
            double amount = readNumber("ยอดสุทธิของบิล (บาท)", 0, 10000000);
            int earned = (int) (amount / Member.bahtPerPoint());
            System.out.println(
                    "คำนวณแต้มสะสม: " + amount + " / " + Member.bahtPerPoint() + " (ปัดลง) = +" + earned + " แต้
m");
            int points = readInt("จำนวนแต้มที่ใช้แลก (0 = ไม่ใช้)", 0, member.points());
            if (points > 0) {
                if (points < Member.minimumRedeem())
                    System.out.println("ต้องแลกอย่างน้อย " + Member.minimumRedeem() + " แต้ม");
                else if (points * Member.pointValue() > amount)
                    System.out.println("ส่วนลดเกินยอดบิล");
                else
                    System.out.printf("ส่วนลด: %.2f บาท - หักแต้มเมื่อชำระเงินจริง%n", points * Member.pointValue());
            }
            System.out.println("นี่เป็นตัวอย่างคำนวณ - จ่ายเงินจริงที่เมนู 4 ชำระเงิน");
        } else if (action == 3) {
            screen("ตรวจสอบประวัติและสถิติแต้มสมาชิก", "แคชเชียร์");
            Member member = findMember(readRequired("เบอร์โทร"));
            if (member == null) {
                System.out.println("ไม่พบสมาชิก");
                return;
            }
            showMember(member);
            for (int i = 0; i < member.transactions().size(); i++) {
                PointTransaction transaction = member.transactions().get(i);
                String reference = "ยอดเริ่มต้น";
                if (transaction.payment() != null)
                    reference = transaction.payment().receiptNo();
                System.out.println(transaction.time() + " - " + transaction.typeText() + " " + transaction.poin
ts()
                        + " - เหลือ " + transaction.balance() + " - " + reference);
            }
        } else if (action == 4) {
            screen("แก้ไขข้อมูลสมาชิก", "แคชเชียร์");
            Member member = findMember(readRequired("เบอร์โทร"));
```

```
            if (member == null) {
                System.out.println("ไม่พบสมาชิก");
                return;
            }
            showMember(member);
            String name = readRequired("ชื่อใหม่");
            String phone = readRequired("เบอร์ใหม่");
            Member duplicate = findMember(phone);
            if (duplicate != null && duplicate != member) {
                System.out.println("เบอร์ซ้ำ");
                return;
            }
            if (confirm("บันทึกข้อมูลสมาชิกหรือไม่")) {
                member.changePhone(phone);
                member.rename(name);
                System.out.println("แก้ข้อมูลแล้ว");
            }
        } else if (action == 5) {
            screen("รายชื่อสมาชิกทั้งหมดในระบบ", "ผู้จัดการ");
            for (int i = 0; i < members.size(); i++)
                showMember(members.get(i));
            if (confirm("กำหนดนโยบายแต้มใหม่หรือไม่")) {
                screen("ผู้จัดการ: กำหนดเงื่อนไขและนโยบายแต้มสะสมทั้งร้าน", "ผู้จัดการ");
                int baht = readInt("บาทต่อ 1 แต้ม", 1, 100000);
                double value = readNumber("มูลค่า 1 แต้ม", 0.01, 10000);
                int minimum = readInt("แต้มขั้นต่ำ", 1, 100000);
                Member.setRule(baht, value, minimum);
                System.out.println("เปลี่ยนกฎสำหรับการชำระครั้งต่อไปแล้ว");
            }
        } else if (action == 6)
            approveRequests();
    }
```

# บทที่ 4: การเดินทางของข้อมูลแบบครบวงจร (Life of an Order)

## 4.1 จำลองสถานการณ์จริง: โต๊ะ T01 สั่งอาหารและคิดเงิน

เพื่อความเข้าใจที่แจ่มชัดที่สุด ให้เราพิจารณากรณีศึกษาตัวอย่าง P4 ตามเอกสาร: ลูกค้าโต๊ะ T01 (ซึ่งเป็นสมาชิก คุณอารมณ์ดี มี 46 แต้ม) สั่งอาหาร 2 รายการ:

- **ผัดกะเพราปลา (M01):** 2 จาน @ 69 บาท พร้อมตัวเลือก "เพิ่มไข่ดาว" ทั้ง 2 จาน @ 15 บาท = 2 × (69 + 15) = 168 บาท
- **ยำหมูยอ/แหนม (M02):** 1 จาน @ 65 บาท = 65 บาท
- **ยอดรวมทั้งสิ้น:** 168 + 65 = 233 บาท

### การไหลของข้อมูลในระบบทีละสเต็ป (Step-by-Step Flow):

**สเต็ปที่ 1: สแกนโต๊ะและผูกสมาชิก (UC01)**
`Table t01` ถูกสแกน (`qrScanned = true`) → สร้าง `Customer cus` → ค้นพบสมาชิก `cus.linkMember(member)` → สร้าง `Order ord = new Order("ORD-1", t01, cus)` ในสถานะ CART

**สเต็ปที่ 2: หยิบอาหารใส่ตะกร้า (UC01)**
เรียก `ord.addItem(kaprao, 2, "เผ็ดกลาง", "ไม่ใส่ชูรส")` → สร้าง `OrderItem item1` พร้อมคัดลอกสูตรปลา 0.15 กก. ต่อจาน → เรียก `item1.addOption(0)` เพิ่มไข่ดาว บวกราคาเพิ่ม 15 บาท และเพิ่มไข่ไก่ 1 ฟองในสูตร Snapshot
เรียก `ord.addItem(yam, 1, "เผ็ดน้อย", "")` → สร้าง `OrderItem item2` คัดลอกสูตรหมูยอ 0.5 แท่ง, แหนม 0.5 แท่ง, น้ำยำ 0.1 ลิตร

**สเต็ปที่ 3: ตรวจสอบสต็อกและยืนยันออเดอร์ (Two-Pass Confirmation)**
เมื่อผู้ใช้กดยืนยัน เรียก `ord.confirm()`:
• *Pass 1 (ตรวจสอบ):* รวมวัตถุดิบทั้งหมด: ปลาต้องใช้ 0.15 × 2 = 0.30 กก. (มี 5.3 กก. พอ), ไข่ไก่ต้องใช้ 1 × 2 = 2 ฟอง (มี 3 ฟอง พอ), หมูยอ 0.5 แท่ง (พอ), แหนม 0.5 แท่ง (พอ), น้ำยำ 0.1 ลิตร (พอ)
• *Pass 2 (ตัดสต็อก):* เรียก `deduct()` ของทุกวัตถุดิบ: ปลาเหลือ 5.0 กก., ไข่ไก่เหลือ 1 ฟอง, หมูยอเหลือ 1.5 แท่ง → แต่ละตัวสร้าง `StockMovement` ประเภท "ขาย"
• ออเดอร์เปลี่ยนสถานะเป็น `CONFIRMED` และโต๊ะ T01 เปลี่ยนสถานะเป็น **ใช้งาน**

**สเต็ปที่ 4: ครัวปรุงอาหาร (UC03)**
คนครัวรับ `item1` เข้าสถานีครัวร้อน เรียก `item1.startCooking(cook, 5.0)` จดเวลาเริ่ม → ปรุงเสร็จเรียก `item1.markReady()` จดเวลาจบ
สถานีครัวยำทำ `item2` จนพร้อมเสิร์ฟเช่นเดียวกัน

**สเต็ปที่ 5: เสิร์ฟอาหาร (UC03)**
พนักงานเสิร์ฟ `item1.serve(cashier)` และ `item2.serve(cashier)` → เรียก `ord.refreshStatus()` พบว่าอาหารเสิร์ฟครบทุกจาน ออเดอร์เปลี่ยนสถานะเป็น SERVED

**สเต็ปที่ 6: ชำระเงิน คิดแต้ม และปิดบิล (UC04)**
แคชเชียร์เลือกออเดอร์ ORD-1 (ยอดรวม 233 บาท) → ลูกค้าใช้ 10 แต้มแลกส่วนลด 10 บาท (ยอดสุทธิเหลือ 223 บาท) → ลูกค้าจ่ายเงินสด 250 บาท (เงินทอน 27 บาท)
เมื่อกดยืนยันชำระเงิน `payment.confirm()`:
• แต้มสมาชิกถูกหัก 10 แต้ม: 46 - 10 = เหลือ 36 แต้ม (สร้าง PointTransaction ประเภท "แลก")
• คำนวณแต้มสะสมใหม่จากยอดสุทธิ: 223 ÷ 50 = 4 แต้ม (ปัดเศษทิ้ง)
• แต้มสะสมเพิ่ม 4 แต้ม: 36 + 4 = **ยอดสุทธิสุดท้าย 40 แต้ม** (สร้าง PointTransaction ประเภท "สะสม")
• ออเดอร์เปลี่ยนสถานะเป็น PAID และโต๊ะ T01 เปลี่ยนสถานะกลับเป็น ว่าง

**สเต็ปที่ 7: รายงานผลการขาย (UC06)**
เมื่อผู้จัดการเปิดรายงานยอดขาย ยอดเงินสด 223 บาทจะถูกบันทึกเข้ารายรับของวัน และผัดกะเพราปลาจะถูกนับยอดขาย 2 จาน, ยำหมูยอถูกนับ 1 จานในการจัดอันดับเมนูยอดนิยม

## 4.2 แผนภาพสถานะ (State Transition Diagram)

| เอนทิตี | สถานะที่เป็นไปได้ | เหตุการณ์ที่ทำให้เปลี่ยน (Trigger) | เงื่อนไขสำคัญที่ต้องผ่าน (Guards) |
|---|---|---|---|
| **Order** | CART → CONFIRMED → COOKING → READY → SERVED → PAID | สแกนโต๊ะ → confirm() → ครัวเริ่มปรุงจานแรก → ปรุงเสร็จครบ → เสิร์ฟครบ → payment.confirm() | ตะกร้าต้องมีอาหาร, สต็อกต้องพอทุกตัว, การชำระเงินต้องทำได้เมื่ออาหารเสิร์ฟครบทุกจานแล้วเท่านั้น |
| **OrderItem** | WAITING → COOKING → READY → SERVED (หรือ CANCELLED) | confirm ออเดอร์ → startCooking() → markReady() → serve() (หรือ cancel() / decideRequest()) | ต้องเคยตัดสต็อกจริง, ต้องไม่มีคำขอค้างที่รอผู้จัดการอนุมัติ, หากกำลังปรุงอยู่ต้องผ่านผู้จัดการเท่านั้น |
| **PurchaseOrder** | DRAFT → ORDERED → RECEIVED | สร้างเอกสาร → markOrdered() → receive(qtys) | ต้องมีรายการสั่ง, ซัพพลายเออร์ต้องส่งวัตถุดิบนั้นได้, จำนวนรับจริงต้องเป็นตัวเลขครบทุกรายการ |
| **Table** | ว่าง → ใช้งาน → ว่าง | เปิดร้าน → order.confirm() → payment.confirm() | โต๊ะต้องผ่านการสแกน QR ก่อนนำรหัสไปเปิดบิล |

# บทที่ 5: คลังข้อสอบและแนวทางตอบคำถามสำหรับนำเสนอ (Defense Cheatsheet)

รวบรวม 15 คำถามสำคัญที่คณะกรรมการสอบ อาจารย์ผู้ทรงคุณวุฒิ หรือผู้ตรวจประเมินโปรเจกต์มักจะถาม พร้อมแนวทางการตอบที่ถูกต้องตามหลักการวิเคราะห์และออกแบบเชิงวัตถุ (OOAD) และมาตรฐานสถาปัตยกรรมซอฟต์แวร์:

**คำถามที่ 1: ทำไมระบบนี้จึงแยก Main ออกจาก 16 คลาสในแพ็กเกจ model?**

**แนวทางการตอบ:** เป็นไปตามหลักการ Separation of Concerns (SoC) และ Single Responsibility Principle (SRP) คลาสใน `model` มีหน้าที่ดูแลข้อมูล กฎเกณฑ์ทางธุรกิจ และความถูกต้องของตนเอง โดยไม่ขึ้นกับว่าผู้ใช้จะติดต่อผ่านหน้าจอแบบใด ส่วน `Main` ทำหน้าที่เป็น Presentation และ Orchestrator หากในอนาคตเราต้องการเปลี่ยนหน้าจอจากคอนโซล Text Mode ไปเป็นหน้าเว็บ หรือแอปมือถือ เราสามารถนำคลาสทั้ง 16 ใน `model` ไปใช้งานต่อได้ทันทีโดยไม่ต้องแก้ไขโค้ดตรรกะธุรกิจเลย

**คำถามที่ 2: ทำไมจึงไม่มีการใช้ Inheritance (extends) ระหว่าง 16 คลาสนี้เลย?**

**แนวทางการตอบ:** เพราะคลาสทั้ง 16 ในระบบร้านอาหารนี้มีความสัมพันธ์เชิงธุรกิจแบบ *has-a (เป็นเจ้าของหรือรู้จักกัน)* ไม่ใช่ความสัมพันธ์แบบ *is-a (เป็นชนิดย่อยของกันและกัน)* เช่น โต๊ะไม่ได้เป็นลูกค้า, ลูกค้าไม่ได้เป็นออเดอร์, และออเดอร์ไม่ได้เป็นอาหาร การฝืนใช้ Inheritance จะทำให้เกิดสถาปัตยกรรมที่ผิดเพี้ยนและเกิด Coupling สูง เราจึงยึดหลักการสากล "Composition over Inheritance" ตามแนวทางของ Clean OOP

**คำถามที่ 3: ทำไมใน Order.confirm() ถึงต้องตรวจสต็อก 2 รอบ (Two-Pass Loop)?**

**แนวทางการตอบ:** เพื่อรักษาความเป็น Atomic Transaction (All-or-Nothing) หากออเดอร์ต้องใช้เนื้อปลาและไข่ไก่ แต่เราวนลูปตรวจเนื้อปลาแล้วตัดเนื้อปลาทันที ก่อนจะไปพบว่าไข่ไก่หมด จะทำให้คำสั่งซื้อล้มเหลวแต่เนื้อปลาถูกตัดไปแล้ว เกิดความเสียหายต่อสต็อกสินค้า ดังนั้นเราจึงต้องใช้ Loop ที่ 1 ตรวจสอบวัตถุดิบทุกชนิดให้มั่นใจว่าพอครบ 100% ก่อน เมื่อผ่านหมดแล้วจึงค่อยเข้า Loop ที่ 2 เพื่อตัดสต็อกจริง

**คำถามที่ 4: ทำไม OrderItem ต้องคัดลอกราคาและสูตรมาจาก Menu มาเก็บไว้เอง (Snapshot Pattern)?**

**แนวทางการตอบ:** เพื่อรักษาประวัติศาสตร์และความถูกต้องทางบัญชี หากร้านเปิดมา 1 ปีแล้วผู้จัดการขึ้นราคาผัดกะเพราจาก 69 บาทเป็น 80 บาท หรือเชฟปรับสูตรลดเนื้อปลาลง บิลและประวัติออเดอร์ของลูกค้าเก่าที่เคยสั่งไปเมื่อสัปดาห์ก่อนจะต้องไม่เปลี่ยนราคาและสูตรตามไปด้วย การทำ Snapshot ไว้ใน `OrderItem` ทำให้ข้อมูลในอดีตคงเดิมอย่างถาวร

**คำถามที่ 5: ในภาษา Java มี Garbage Collection อยู่แล้ว ทำไมเรายังต้องเขียนเมธอด destroy() ขึ้นมาเอง?**

**แนวทางการตอบ:** Garbage Collector (GC) ของ Java จะลบอ็อบเจกต์ออกจากหน่วยความจำก็ต่อเมื่อ *ไม่มีใครอ้างถึงอ็อบเจกต์นั้นแล้ว* แต่ในระดับการออกแบบเชิงตรรกะ (Domain Modeling) เมธอด `destroy()` ที่เราเขียนขึ้นทำหน้าที่เป็น **Lifecycle Cascade Hook** เพื่อสะท้อนกฎ Composition อย่างชัดเจน เช่น เมื่อ `Order.destroy()` ถูกเรียก มันจะสั่ง `item.destroy()` ทุกตัว และเคลียร์ลิงก์ต่าง ๆ ทิ้ง ทำให้ไม่มีการอ้างอิงค้างอยู่ และช่วยให้ GC สามารถเก็บกวาดหน่วยความจำได้อย่างรวดเร็วและปลอดภัย

**คำถามที่ 6: ทำไมฟิลด์กฎแต้มใน Member ถึงประกาศเป็น static แต่ยอดแต้มสะสมเป็นธรรมดา?**

**แนวทางการตอบ:** ยอดแต้มสะสม (`points`) เป็นของสมาชิกแต่ละคน สมาชิก A มี 40 แต้ม สมาชิก B มี 10 แต้ม จึงต้องเป็น Instance Variable แต่กฎเกณฑ์การคำนวณ เช่น อัตรา 50 บาทได้ 1 แต้ม (`bahtPerPoint`) หรือ 1 แต้มเท่ากับ 1 บาท (`pointValue`) เป็นนโยบายการตลาดของร้านที่สมาชิกทุกคนต้องปฏิบัติตามเหมือนกันหมด จึงประกาศเป็น `static` (Class Variable) ทำให้เมื่อผู้จัดการแก้ไขกฎผ่าน `Member.setRule()` จะมีผลต่อสมาชิกทุกคนทั้งร้านทันที

**คำถามที่ 7: ทำไมใน Recipe ถึงใช้ List 2 ชุด (ingredients และ quantities) แทนที่จะใช้ Map?**

**แนวทางการตอบ:** ตามหลักสูตรการสอนและการอธิบาย OOP ขั้นพื้นฐาน การใช้ List สองชุดที่ตรงกันด้วย Index (Parallel Lists) เช่น ตำแหน่งที่ 0 คือวัตถุดิบปลา และตำแหน่งที่ 0 ของอีก List คือปริมาณปลา 0.15 กก. ช่วยให้ผู้เรียนสามารถไล่ติดตามค่าด้วยลูป `for (int i = 0; i < size; i++)` ได้อย่างตรงไปตรงมา และไม่ก่อให้เกิดปัญหาเรื่องการเรียงลำดับของ Key เหมือนใน โครงสร้าง Map ทั่วไป

**คำถามที่ 8: มี Constructor แล้ว ทำไมคลาสต่าง ๆ ยังต้องมีเมธอดอ่านข้อมูล (เช่น name(), stockQty())?**

**แนวทางการตอบ:** เพราะ Constructor ทำงานเพียง *ครั้งเดียวตอนสร้างอ็อบเจกต์ (ตอน new)* แต่หลังจากสร้างอ็อบเจกต์ขึ้นมาแล้ว โปรแกรมยังต้องมีการทำงานต่อเนื่องอีกยาวนาน เช่น หน้าจอ `Main` ต้องการนำชื่ออาหาร ราคา ยอดเงิน สถานะ และสต็อกคงเหลือไปแสดงผลให้ผู้ใช้ดู จึงจำเป็นต้องมีเมธอดอ่านข้อมูลเพื่อเปิดช่องทางให้ภายนอกอ่านค่าฟิลด์ที่เป็น `private` ได้โดยไม่ละเมิด Encapsulation

**คำถามที่ 9: ทำไมในคลาสของแพ็กเกจ model ถึงไม่มีคำสั่ง System.out.println() แม้แต่บรรทัดเดียว?**

**แนวทางการตอบ:** เพราะคลาสใน `model` เป็น Business Entity ที่ไม่ควรผูกติดกับอุปกรณ์แสดงผล (Output Device) หากคลาสโมเดลมีคำสั่งพิมพ์หน้าจอคอนโซล เมื่อนำคลาสนี้ไปใช้งานบนเว็บหรือแอปพลิเคชัน ข้อความเหล่านั้นจะกลายเป็นข้อความขยะในเซิร์ฟเวอร์ หน้าที่การแสดงผลและการจัดรูปแบบข้อความจึงเป็นหน้าที่ของคลาส `Main` เพียงผู้เดียว

**คำถามที่ 10: enum ที่อยู่ในคลาส เช่น OrderStatus หรือ Role ทำให้มีจำนวนคลาสเกิน 16 หรือไม่?**

**แนวทางการตอบ:** ไม่เกินครับ เพราะโครงสร้างไฟล์ในระบบมี 16 ไฟล์ Java ใน `src/model` พอดี โดย `enum` ถูกประกาศเป็นสมาชิกภายในคลาสของตนเอง (Nested Enum) เช่น `Order.OrderStatus` หรือ `Employee.Role` ทำให้ขอบเขตของ enum ผูกอยู่กับคลาสนั้นอย่างสมเหตุสมผล และไม่มีไฟล์คลาสสนับสนุนแยกย่อยภายนอก

**คำถามที่ 11: ทำไมลูกค้าถึงกดยกเลิกอาหารเองไม่ได้เมื่ออาหารเริ่มปรุงแล้ว?**

**แนวทางการตอบ:** เป็นการสะท้อน Business Logic จริงของร้านอาหาร เมื่ออาหารอยู่ในสถานะ `WAITING` (รอปรุง) พนักงานยังไม่ได้นำวัตถุดิบไปปรุง ลูกค้าจึงยกเลิกได้ทันทีและสต็อกจะถูกคืนเข้าคลัง แต่หากอาหารเปลี่ยนเป็น `COOKING` (กำลังปรุง) วัตถุดิบได้ถูกใช้ลงกระทะไปแล้ว หากลูกค้ายกเลิกโดยพลการร้านจะขาดทุน จึงต้องออกแบบให้ส่งคำขอพิเศษ (`requestChange`) ไปยังหน้าจอของผู้จัดการเพื่อให้ผู้จัดการพิจารณาอนุมัติหรือปฏิเสธเป็นกรณีพิเศษ

**คำถามที่ 12: การสแกน QR Code โต๊ะใน Terminal ทำงานอย่างไร ทั้งที่ไม่มีกล้องจริง?**

**แนวทางการตอบ:** เป็นการจำลอง (Simulation) โดยในคลาส `Table` จะมีเมธอด `scanTableQRCode()` ซึ่งทำหน้าที่ตั้งค่าแฟล็ก `qrScanned = true` เมื่อผู้ใช้เลือกโต๊ะในหน้าจอ UC01 ระบบจะจำลองว่าลูกค้าได้นำกล้องมาสแกนแล้ว และเมธอด `identifyTableNumber()` จะยอมคืนรหัสโต๊ะให้เปิดออเดอร์ได้เฉพาะเมื่อแฟล็กนี้เป็นจริงเท่านั้น หากไม่ได้สแกนจะโยน `IllegalStateException` ออกมาทันที

**คำถามที่ 13: ทำไม Customer จึงไม่ถูกลบเมื่อ Order ถูกลบ?**

**แนวทางการตอบ:** เพราะความสัมพันธ์ระหว่าง `Order` กับ `Customer` เป็นแบบ Association ลูกค้าเป็นบุคคลภายนอกที่เข้ามาใช้บริการ ในขณะที่ Order เป็นเพียงเอกสารการสั่งอาหาร 1 ครั้ง เมื่อบิลอาหารสิ้นสุดลง ข้อมูลตัวแทนลูกค้ายังคงสามารถคงอยู่ได้เพื่อใช้วิเคราะห์หรือเชื่อมโยงกับบัญชีสมาชิก

**คำถามที่ 14: ปัญหา Floating Point ป้องกันอย่างไรตอนตัดสต็อกและคำนวณเงิน?**

**แนวทางการตอบ:** ในการเปรียบเทียบสต็อกของ `Ingredient.hasEnough(qty)` มีการบวกค่าเผื่อความคลาดเคลื่อน (Epsilon) `0.000001` เข้าไปก่อนเทียบ เพื่อป้องกันข้อผิดพลาดการปัดเศษของเลขฐานสองในชนิดข้อมูล double ส่วนในการคำนวณเงินทอนและยอดสต็อก มีการใช้ฟังก์ชัน `Math.round(...)` ปัดทศนิยมให้ตรงกับหน่วยสตางค์และหน่วยกรัมอย่างแม่นยำ

**คำถามที่ 15: หากระบบไฟดับหรือปิดโปรแกรม ข้อมูลในร้านจะหายหรือไม่ เพราะเหตุใด?**

**แนวทางการตอบ:** ข้อมูลจะหายและกลับไปเริ่มต้นที่ข้อมูลตัวอย่างใหม่ครับ เพราะระบบฉบับนี้เป็นการจำลองสถาปัตยกรรม OOP เชิงตรรกะแบบ In-Memory Collections (เก็บใน RAM ผ่าน ArrayList) เพื่อวัตถุประสงค์ในการศึกษาโครงสร้างอ็อบเจกต์ โดยยังไม่ได้เชื่อมต่อกับระบบจัดเก็บข้อมูลถาวร (Persistent Storage) เช่น ฐานข้อมูล SQL ซึ่งเป็นสเต็ปการต่อยอดในระยะต่อไป

# บทที่ 6: บทสรุปและแนวทางการพัฒนาต่อยอดสู่ระดับ Enterprise

## 6.1 สรุปสาระสำคัญของโครงสร้างระบบ (Project Summary)

ระบบบริหารจัดการร้านอาหาร **"ข้าวต้มอารมณ์ดี"** ในฉบับ Java OOP 17 คลาสนี้ ได้บรรลุเป้าหมายสำคัญทางวิศวกรรมซอฟต์แวร์และการออกแบบเชิงวัตถุอย่างสมบูรณ์แบบ:

- **ความสอดคล้องกับระเบียบวิธี OOAD:** สามารถแปลงโมเดลเชิงแนวคิดจากสไลด์และ Use Cases (P1–P8) สู่อ็อบเจกต์จริงในภาษา Java ได้อย่างถูกต้องครบถ้วนทั้ง 8 งาน
- **การปฏิบัติตามหลักการ Encapsulation:** ข้อมูลทั้งหมดถูกปกป้องและเปลี่ยนแปลงผ่านเมธอดที่มีการตรวจสอบกฎเกณฑ์ทางธุรกิจอย่างเข้มงวด
- **ความชัดเจนของ Composition ทั้ง 5 คู่:** มีหลักฐานการทำลายวงจรชีวิต (destroy cascade) ที่พิสูจน์ได้จริงในระดับซอร์สโค้ด
- **ความปลอดภัยในการทำธุรกรรม:** ใช้รูปแบบ Two-Pass Validation ในการตัดสต็อกและการตรวจรับสินค้า เพื่อป้องกันสภาวะข้อมูลไม่สอดคล้องกัน

## 6.2 แผนที่การพัฒนาต่อยอดสู่ระบบจริง (Enterprise Modernization Roadmap)

หากต้องการนำระบบนี้ไปพัฒนาต่อยอดเพื่อใช้งานในร้านอาหารจริงในเชิงพาณิชย์ สามารถดำเนินตามขั้นตอนวิศวกรรมซอฟต์แวร์สมัยใหม่ได้ดังนี้:

### ระยะที่ 1: การเปลี่ยน In-Memory สู่ Relational Database (JPA / PostgreSQL)

เปลี่ยน `ArrayList` ใน `Main` ไปเป็นตารางในฐานข้อมูลเชิงสัมพันธ์ โดยใช้ **Spring Data JPA / Hibernate**:

- คลาสโมเดลทั้ง 16 จะถูกใส่คำอธิบายประกอบ (Annotations) เช่น `@Entity`, `@Table`, `@Id`
- ความสัมพันธ์ Composition จะถูกแปลงเป็น `@OneToMany(cascade = CascadeType.ALL, orphanRemoval = true)` ซึ่งจะทำหน้าที่เหมือนเมธอด `destroy()` โดยอัตโนมัติในระดับฐานข้อมูล
- ความสัมพันธ์ Association จะถูกแปลงเป็น `@ManyToOne(fetch = FetchType.LAZY)` เพื่อเพิ่มประสิทธิภาพในการสืบค้น

### ระยะที่ 2: การพัฒนา RESTful API และ Layered Architecture

ปรับเปลี่ยนสถาปัตยกรรมจากไฟล์ `Main` เดี่ยว ให้เป็น **3-Tier Layered Architecture**:

- **Controller Layer:** สร้าง `@RestController` เช่น `OrderController`, `MenuController` เพื่อให้บริการรับส่งข้อมูลในรูปแบบ JSON
- **Service Layer:** ย้ายตรรกะการประสานงาน Use Cases จาก `uc01()` ถึง `uc08()` ไปอยู่ใน `OrderService`, `PaymentService`, `StockService` พร้อมใส่ `@Transactional` เพื่อควบคุมความปลอดภัยของธุรกรรม
- **Repository Layer:** อินเทอร์เฟซ `JpaRepository` สำหรับจัดการบันทึกและสืบค้นข้อมูลจากฐานข้อมูล

### ระยะที่ 3: ส่วนติดต่อผู้ใช้สมัยใหม่ (Modern UI / UX)

สร้างเว็บแอปพลิเคชันและแอปพลิเคชันบนอุปกรณ์พกพา:

- **Customer Mobile Web (React / Vue):** ให้ลูกค้าสแกน QR Code จริงบนโต๊ะผ่านกล้องมือถือ ดูเมนูรูปภาพสวยงาม สั่งอาหาร และติดตามสถานะแบบเรียลไทม์ผ่าน WebSocket
- **Kitchen Display System (KDS):** หน้าจอแท็บเล็ตสำหรับคนครัวในแต่ละสถานี แสดงคิวอาหาร จับเวลา และกดแจ้งพร้อมเสิร์ฟ
- **POS Cashier Terminal:** หน้าจอสัมผัสสำหรับแคชเชียร์ คิดเงิน สแกนจ่าย Thai QR PromptPay และพิมพ์ใบเสร็จผ่านเครื่องพิมพ์ความร้อน (Thermal Printer)

---

--- สิ้นสุดคู่มือการเรียนรู้และเจาะลึกโค้ด Java OOP 17 คลาส ระบบร้านข้าวต้มอารมณ์ดี ---